



Mahidol University

*Wisdom of the Land*

# ITCS215
# Introduction to Interactive Multimedia Technology

**Class: 1-2026**
**Module III: Introduction to Computer Graphics, Generative AI, and Audio**
**Lecture: TUE 13:00-16:00**

## Unit 2: Pixel Capturing, 2D Transformation, & AI Application Development

**Instructor:** Lecturer Dr. Kanoksak Wattanachote, PhD (金功勳博士)
**Contact:** kanoksak.wat@mahidol.ac.th

# Unit 2: Content

## Unit 2: Pixel Capturing, 2D Transformation, AI Application Development

- Pixel Capturing
- Webcam and AI Vision:
    - Face Detection using Haar and LBP Cascades for Interactive Multimedia
    - Hand Detection using MediaPipe Hands for Interactive Multimedia
- Drawing II: Closed Shape
- 2D Transformation
- In-class Workshop I: 2D Transformation
- Animation/Video Concepts
    - Motion Vector Field for AI Vision Applications

Concepts
of

# Pixel Capturing



# Pixels

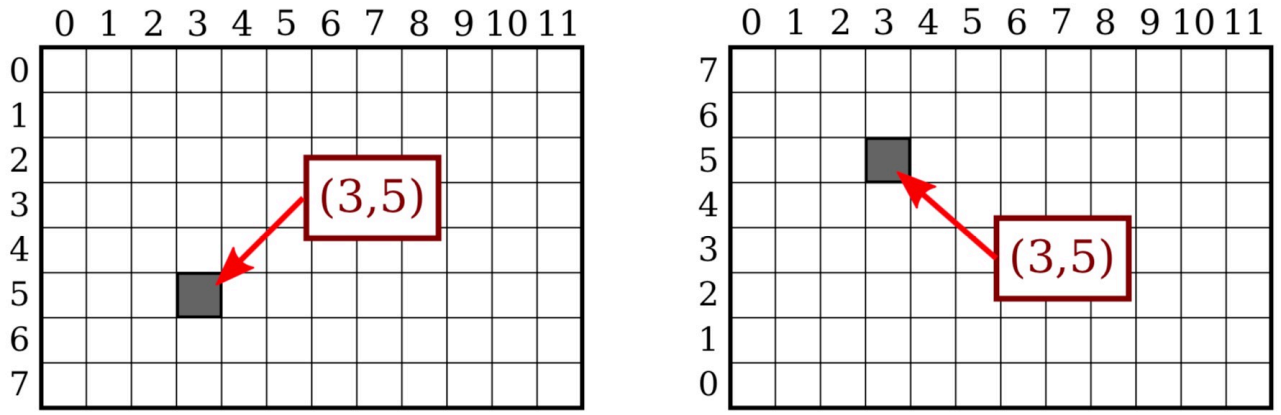
## Module III: Pixel

- A point in 2D can be identified by a pair of numerical coordinates, and it is assigned a color — ‘pixel’.
- In terms of coordinates, a pixel can be identified by a pair of integers giving the column number and row number.

# Pixel Coordinates

## Module III: Pixels

- A digital image is made up of rows and columns of pixels.
- A pair of integers giving the column number and row number, for example, the pixel with coordinates (3, 5) would lie in column number 3 and row number 5.

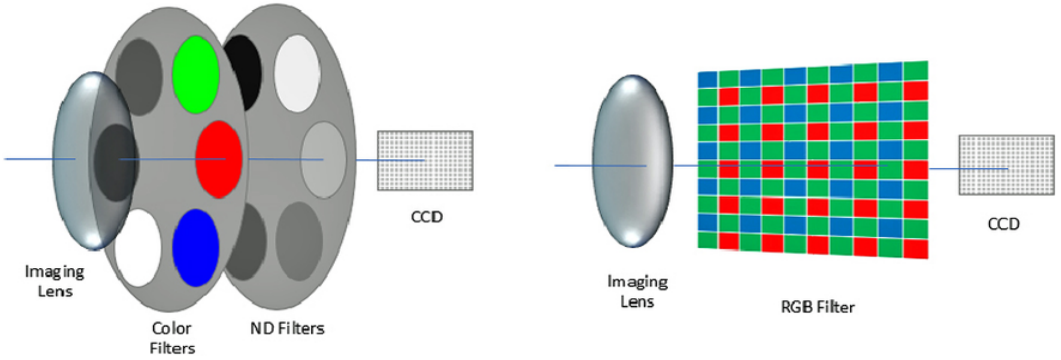



12-by-8 pixel grids, shown with row and column numbers.
On the left, rows are numbered from top to bottom,
on the right, they are numberd bottom to top.

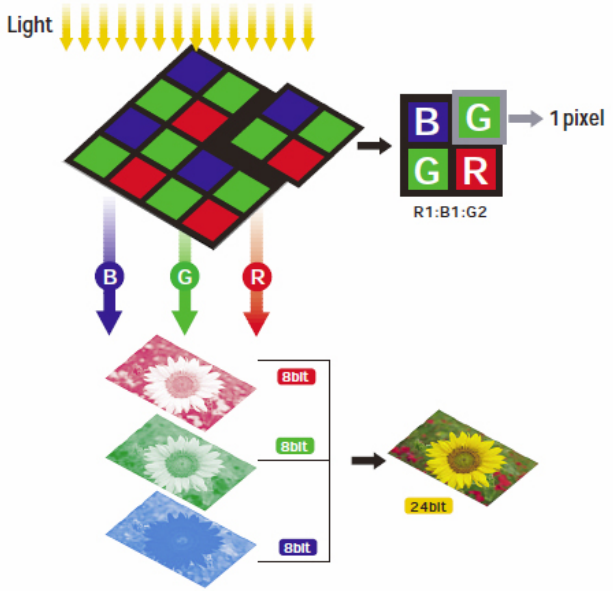
# Pixel Capturing

## Module III: Pixels

- Pixels are captured in a camera sensor using CCDs (Charge-Coupled Devices), which are light-sensitive integration circuits that convert photons into electrons, creating a digital representation of an image (A/D (Analog-to-Digital) conversion).
- Uses Bayer filters for color.
- Limited by dynamic range, noise, and resolution.

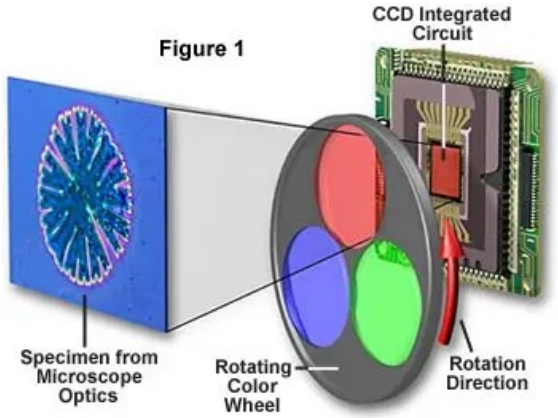



source: https://www.fierceelectronics.com/components/understanding-ccd-camera-resolution-and-dynamic-range




source: https://av.jpn.support.panasonic.com/support/global/cs/dsc/knowhow/knowhow29.html

**Sequential Color Three-Pass CCD Imaging System**

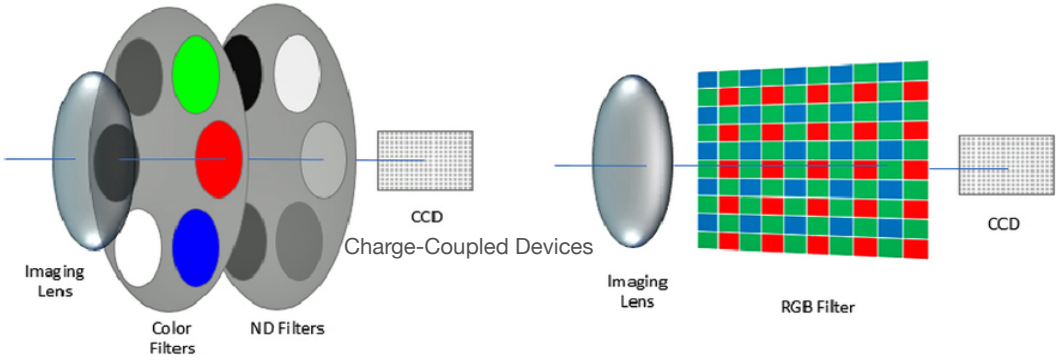



source: https://evidentscientific.com/en/microscope-resource/knowledge-hub/digital-imaging/concepts/threepass

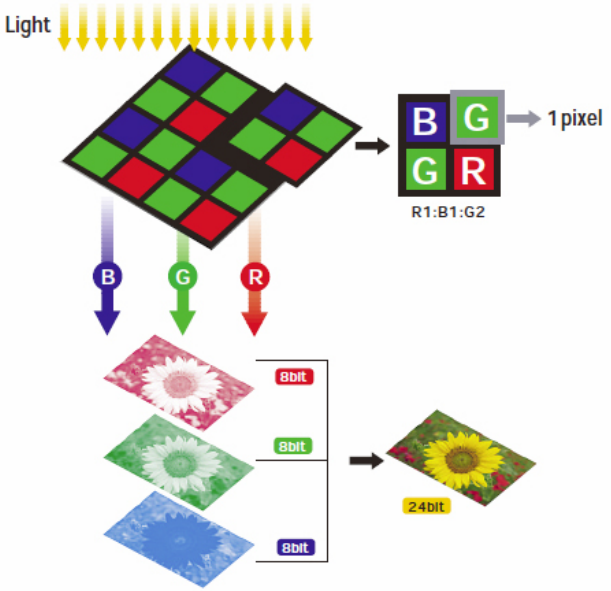
# Pixel Capturing

## Module III: Pixels

- The resolution of an imaging colorimeter is defined as the number of individual horizontal and vertical pixels captured by the imager (typically a CCD).
- The Bayer array represents that two out of every four pixels are green, one is red, and one is blue.
  - Since the green signal contributes the most to the luminance channel.

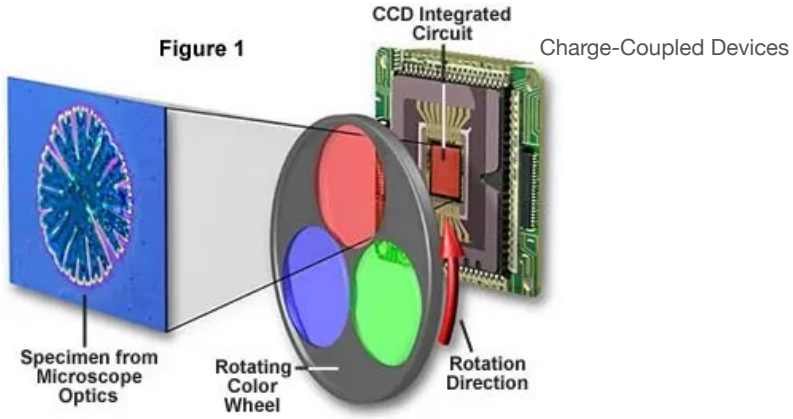



source: https://www.fierceelectronics.com/components/understanding-ccd-camera-resolution-and-dynamic-range




source: https://av.jpn.support.panasonic.com/support/global/cs/dsc/knowhow/knowhow29.html

**Sequential Color Three-Pass CCD Imaging System**



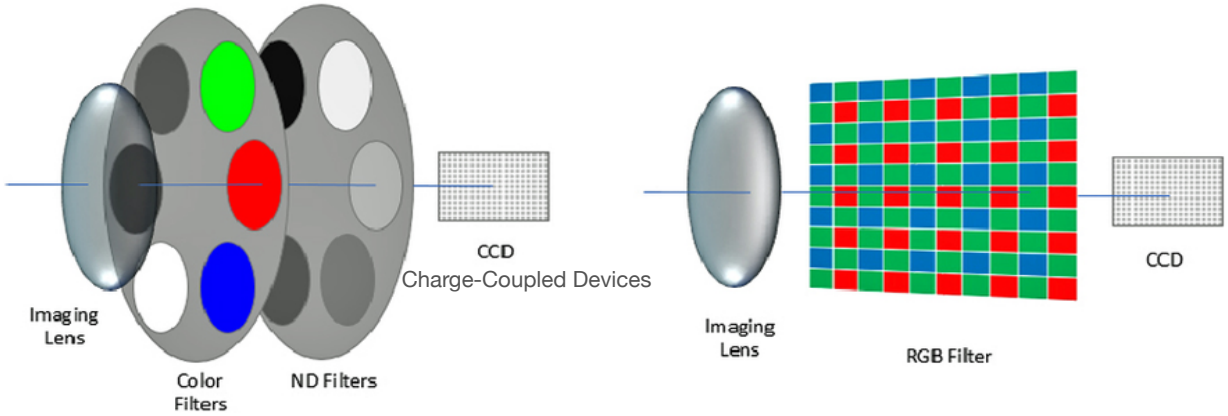

source: https://evidentscientific.com/en/microscope-resource/knowledge-hub/digital-imaging/concepts/threepass

# Dynamic Range

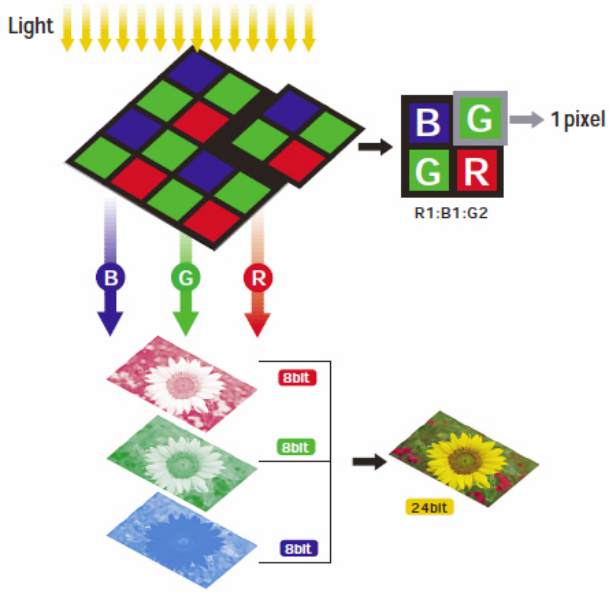
## Module III: Color Perception

- For CCD-based measurements, Dynamic Range is commonly measured in decibels (dB) to describe the ratio of maximum possible signal-to-read noise (signal-to-noise ratio, or SNR).
- The mathematical expression used for the dynamic range of a CCD is:

$$Dynamic\ Range = 20 \times log_{10} \frac{Well\ Size}{Read\ Noise}$$




source: https://www.fierceelectronics.com/components/understanding-ccd-camera-resolution-and-dynamic-range




source: https://av.jpn.support.panasonic.com/support/global/cs/dsc/knowhow/knowhow29.html

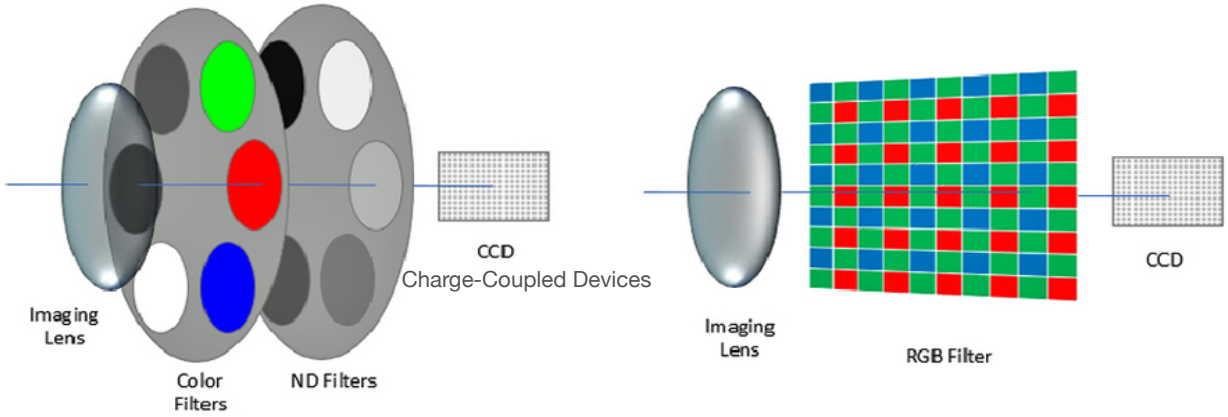


# Dynamic Range

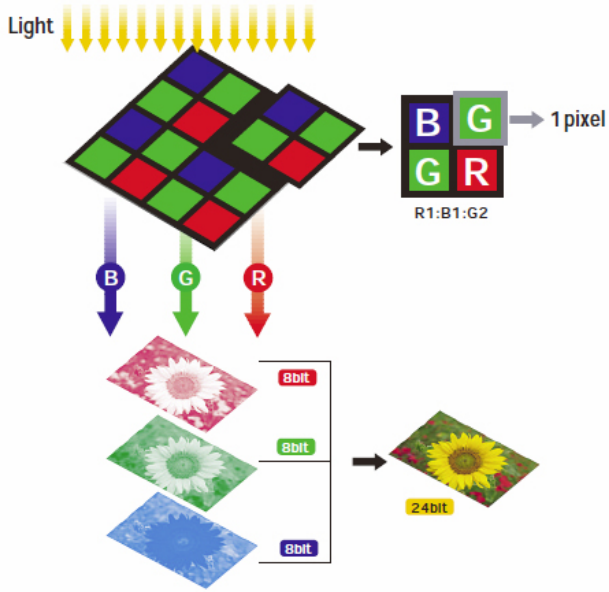
## Module III: Color Perception

- Well size is the number of electrons available in a pixel, i.e., the number of electrons available before CCD saturation.
- Read noise is the electronic noise, expressed in electrons, introduced into the signal when transferring the data from the CCD into a digital format.

$$Dynamic\ Range = 20 \times log_{10} \frac{Well\ Size}{Read\ Noise}$$




source: https://www.fierceelectronics.com/components/understanding-ccd-camera-resolution-and-dynamic-range




source: https://av.jpn.support.panasonic.com/support/global/cs/dsc/knowhow/knowhow29.html

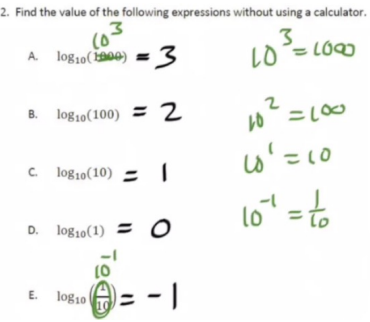


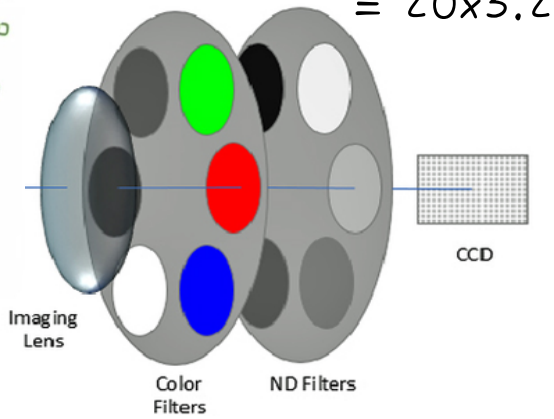
# Dynamic Range

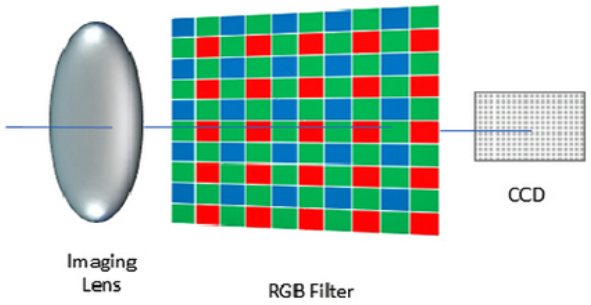
## Module III: Color Perception

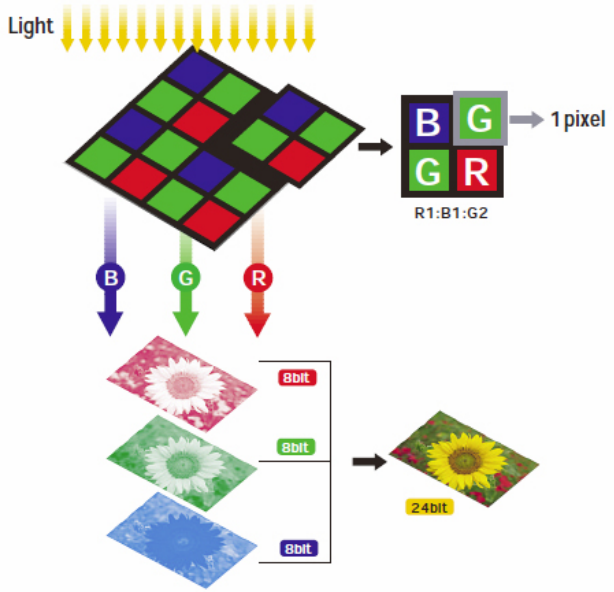
- For example, a CCD with a well size of 20ke- (20 kiloelectrons) per pixel and a system read noise of 12e-.
- Using these values, we obtain a value of 64.4 dB.

$$\text{Dynamic Range} = 20 \times \log_{10}\left(\frac{20000}{12}\right)$$
$$= 20 \times \log_{10}(1666.667)$$
$$= 20 \times 3.22185 = 64.437$$




source: https://av.jpn.support.panasonic.com/support/global/cs/dsc/knowhow/knowhow29.html

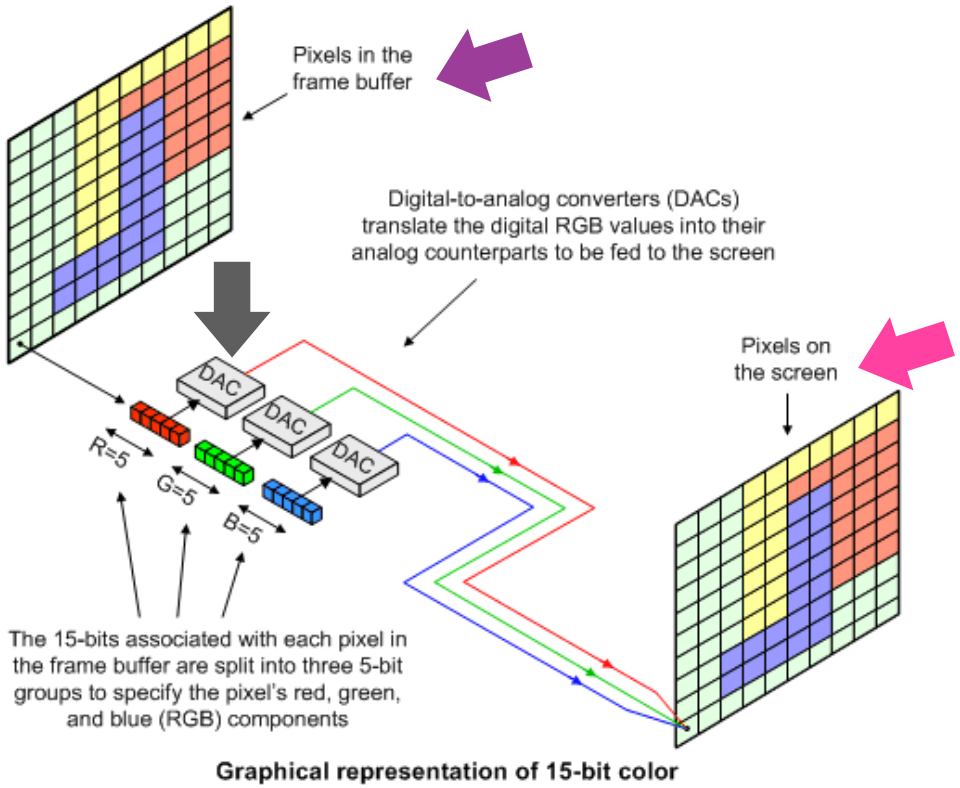


source: https://www.fierceelectronics.com/components/understanding-ccd-camera-resolution-and-dynamic-range

# Pixel Capturing

## Module III: Pixels

- The display (monitor/screen) emits light and converts digital signals back to photons (D/A (Digital-to-Analog) conversion) using Digital-to-Analog Converters (DACs).
- A modern graphics subsystem contains a block of RAM (memory) known as the frame buffer.




**Graphical representation of 15-bit color**

**emit** (v) - to send out a beam, noise, smell, or gas.

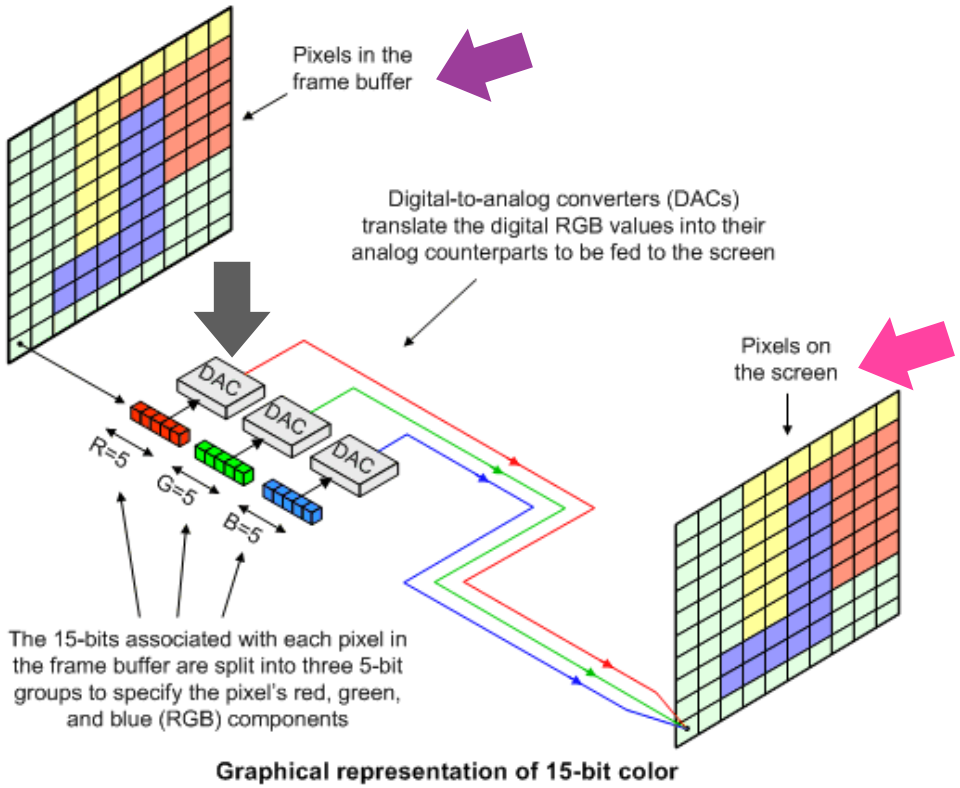
**photon** (n) - a single unit of light.

source: https://www.clivemaxfield.com/diycalculator/popup-h-console.shtml

# Pixel Capturing

## Module III: Pixels

- The frame buffer stores a copy of whatever image is currently being displayed on the computer's screen, where this image is constructed by the graphics processor.
- Another term of which we need to be aware is color depth, which refers to the number of bits we use to represent the color of each pixel in an image.



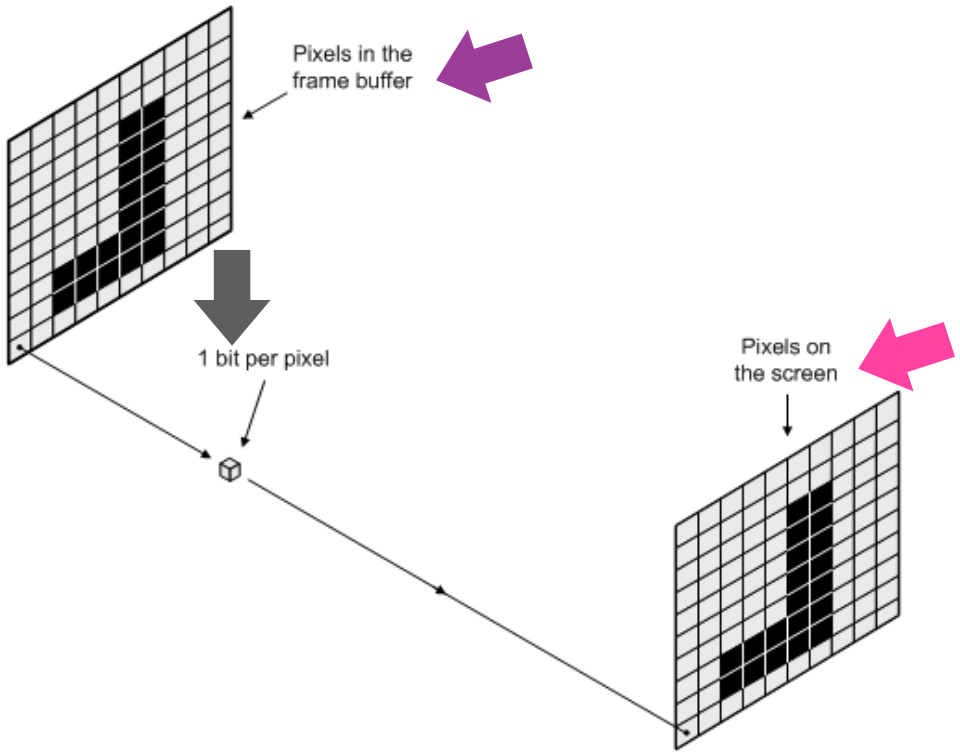

**Graphical representation of 15-bit color**

source: https://www.clivemaxfield.com/diycalculator/popup-h-console.shtml

# Pixel Capturing

## Module III: Pixels

- If we used only a single bit to represent each pixel, for example, then each bit could be in only one of two states – either 'Off' or 'On' (logic 0 or logic 1) – which would allow us to represent only two colors; for instance, black and white.



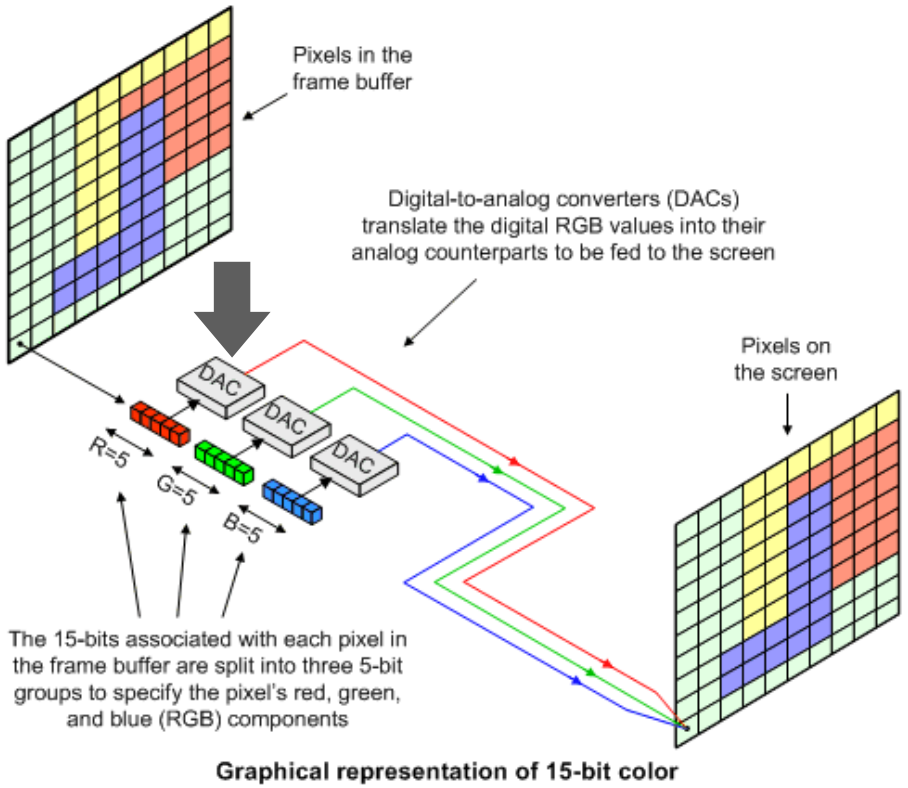

**Graphical representation of 1-bit "color"**

source: https://www.clivemaxfield.com/diycalculator/popup-h-console.shtml

# Pixel Capturing

## Module III: Pixels

- **15-Bit Color:** One technique used in some mid-range graphics cards is to store 15 bits per pixel in the frame buffer.
- This requires almost twice the amount of memory as an 8-bit (256 color) approach, but in return it offers $2^{15}$ = 32,768 colors.



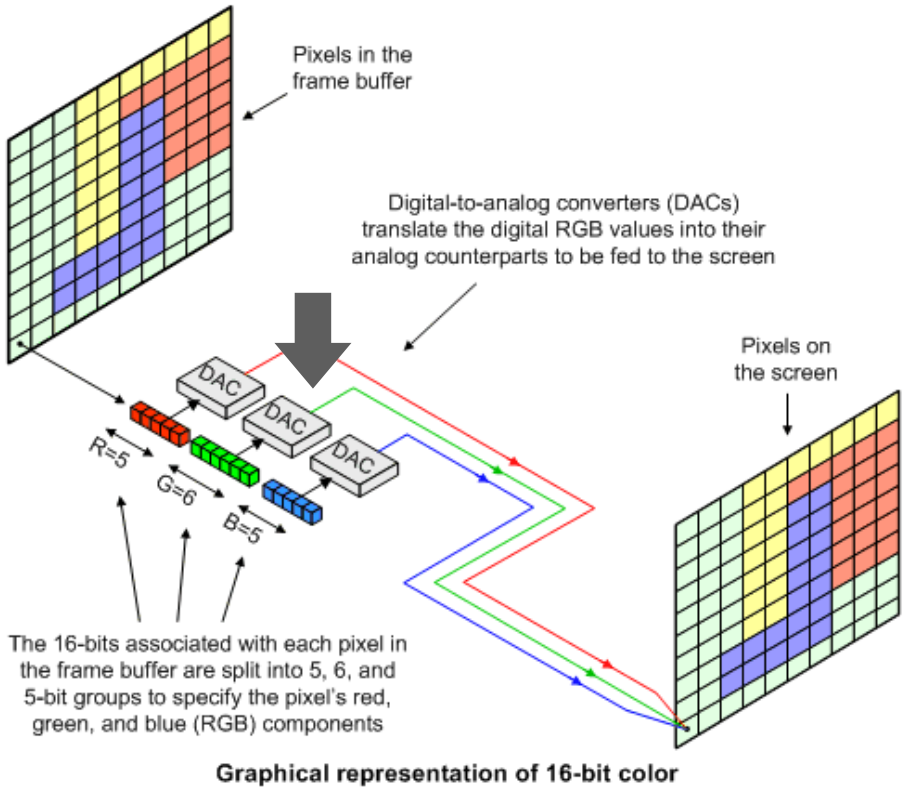

**Graphical representation of 15-bit color**

source: https://www.clivemaxfield.com/diycalculator/popup-h-console.shtml

# Pixel Capturing

## Module III: Pixels

- **16-Bit Color:** Another technique used by mid-range graphics cards is to store 16 bits per pixel in the frame buffer.
- This requires twice the amount of memory as an 8-bit (256 color) approach, but in return it offers $2^{16}$ = 65,536 colors.



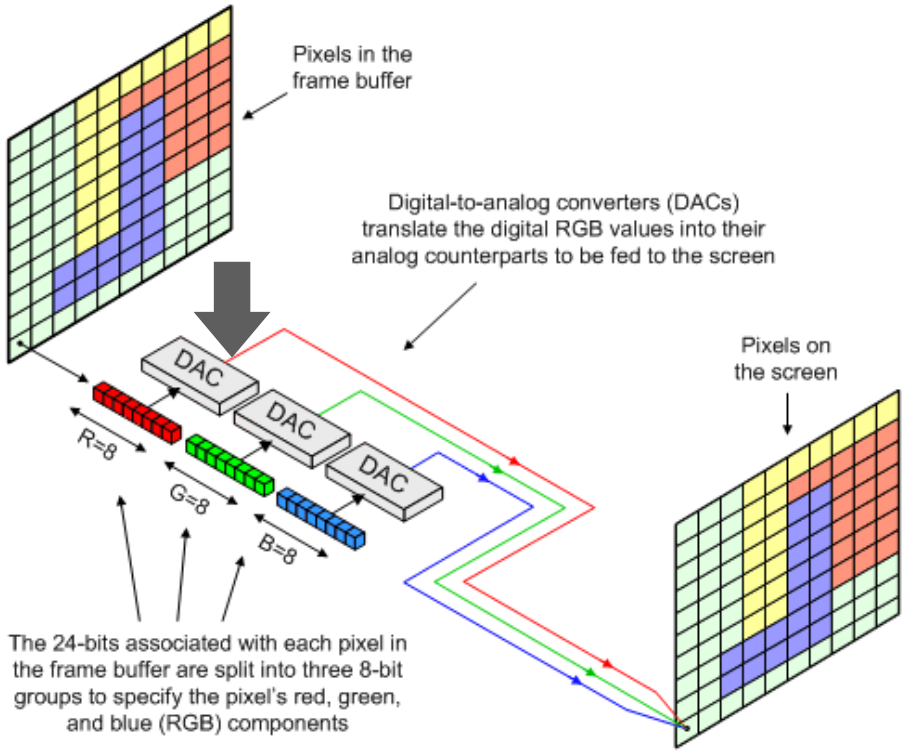

**Graphical representation of 16-bit color**

source: https://www.clivemaxfield.com/diycalculator/popup-h-console.shtml

# Pixel Capturing

## Module III: Pixels

- **24-Bit Color:** Graphics cards that store 24 bits per pixel in the frame buffer can represent $2^{24}$ = 16,777,216 different colors.
- This is more than enough to accurately portray true-to-life images, so 24-bit color is often referred to as "true color."



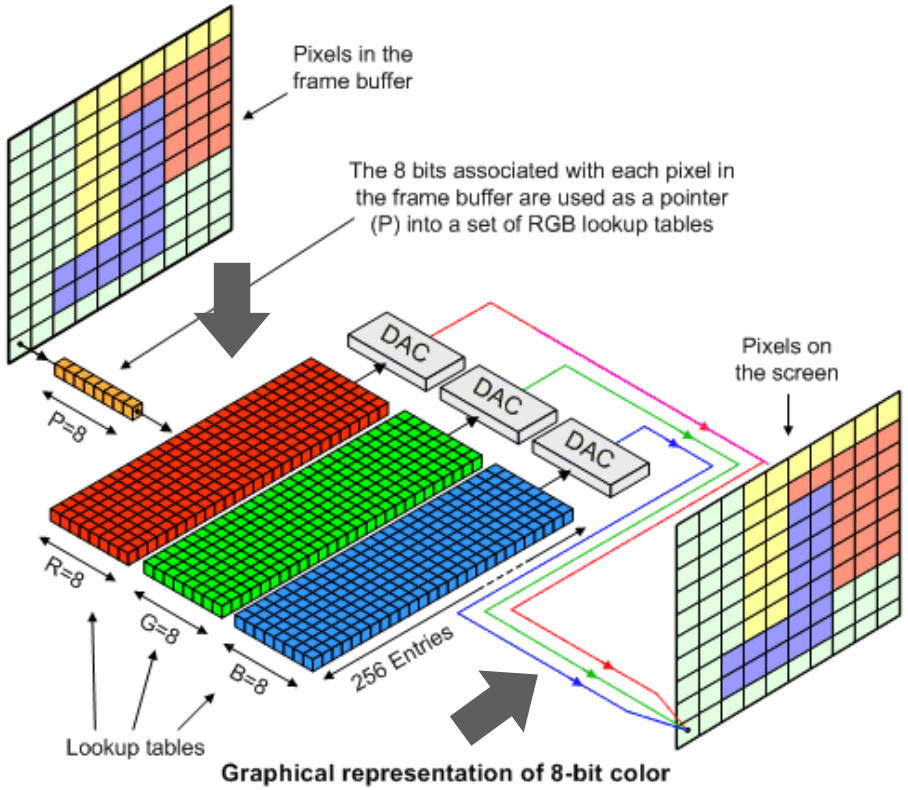

**Graphical representation of 24-bit "true color"**

source: https://www.clivemaxfield.com/diycalculator/popup-h-console.shtml

# Pixel Capturing

## Module III: Pixels

- **8-Bit Color (Redux):** And so we return to considering 8-bit color.
- As we previously noted, the eight bits associated with each pixel in the frame buffer can be used to represent $2^8 = 256$ different patterns of 0s and 1s.
- The problem is that a palette of only 256 colors is really quite restrictive.
  - One common way to mitigate this limitation is to use the 8-bit fields in the frame buffer to index (point) into a set of lookup tables (color palettes).




Graphical representation of 8-bit color

source: https://www.clivemaxfield.com/diycalculator/popup-h-console.shtml

Work
with

# Camera Capture

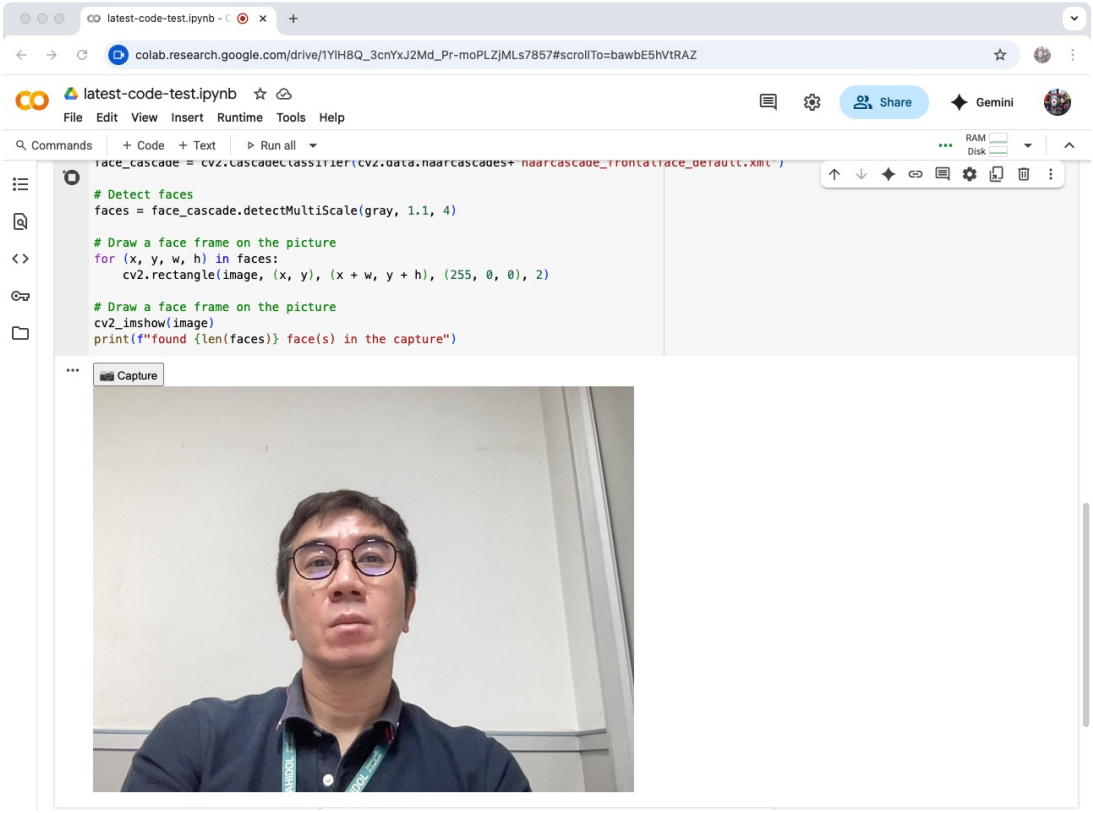


# Webcam with Google Colab

## Module III: Camera Capturing

1. **Install and import the library**

```
!pip install opencv-python-headless
```



# Webcam with Google Colab

## Module III: Camera Capturing

### 2. Install the headless OpenCV package (no GUI) in Google Colab

```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow
from IPython.display import display, Javascript
from google.colab.output import eval_js
from base64 import b64decode
```

- cv2: OpenCV for image processing
- numpy: for working with arrays
- cv2_imshow: for displaying images in Colab
- Javascript, eval_js: for running JavaScript in Colab
- b64decode: for decoding image data

# Webcam with Google Colab

## Module III: Camera Capturing

### 3. Webcam Capture Function

- Defines the `take_photo()` function to capture images via webcam.
- Parameters:
  - filename: Filename to save (default: 'photo.jpg')
  - quality: Image quality (0-1)

```
# Camera opening and photo taking function
def take_photo(filename='photo.jpg', quality=0.8):
    js = Javascript('''
      async function takePhoto(quality) {
        const div = document.createElement('div');
        const capture = document.createElement('button');
        capture.textContent = '📷 Capture';
        div.appendChild(capture);

        const video = document.createElement('video');
        video.style.display = 'block';
        const stream = await navigator.mediaDevices.getUserMedia({video: true});

        document.body.appendChild(div);
        div.appendChild(video);
        video.srcObject = stream;
        await video.play();

        google.colab.output.setIframeHeight(document.documentElement.scrollHeight, true);

        await new Promise((resolve) => capture.onclick = resolve);

        const canvas = document.createElement('canvas');
        canvas.width = video.videoWidth;
        canvas.height = video.videoHeight;
        canvas.getContext('2d').drawImage(video, 0, 0);
        stream.getVideoTracks()[0].stop();
        div.remove();

        return canvas.toDataURL('image/jpeg', quality);
      }
    ''')
```

# Webcam with Google Colab

## Module III: Camera Capturing

### 3. Webcam Capture Function

- Create and execute JavaScript to open the webcam and take a picture.
- Convert image data from base64 to binary.
- Save the image to a file.
- Return the image filename.

```
# Camera opening and photo taking function
def take_photo(filename='photo.jpg', quality=0.8):
    js = Javascript('''

        ...

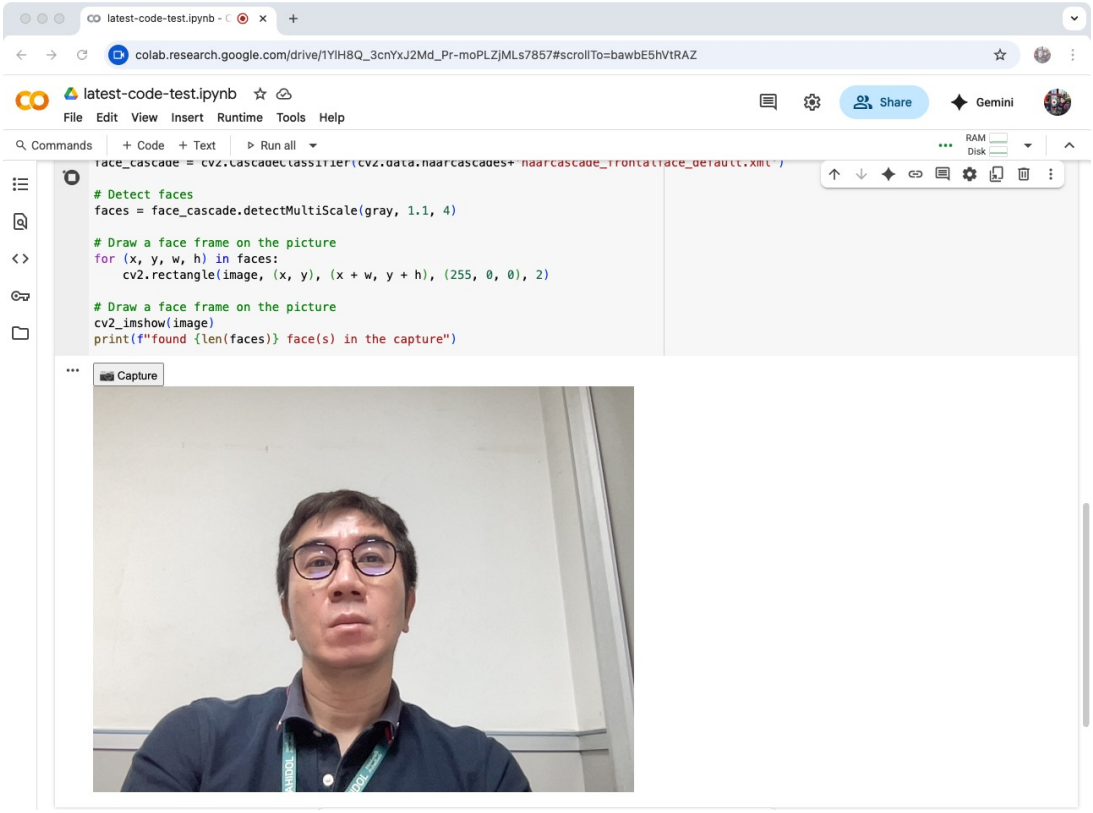
    ''')
    display(js)
    data = eval_js('takePhoto({})'.format(quality))
    binary = b64decode(data.split(',')[1])
    with open(filename, 'wb') as f:
        f.write(binary)
    return filename
```

# Webcam with Google Colab

## Module III: Camera Capturing

4. **Capture (Take a Photo)**
   - Call the take_photo() function to take a photo.
   - Click the 📷 “Capture” button to take a photo.
   - After capturing the photo, the system will analyze your face and immediately display an image with a frame around it.

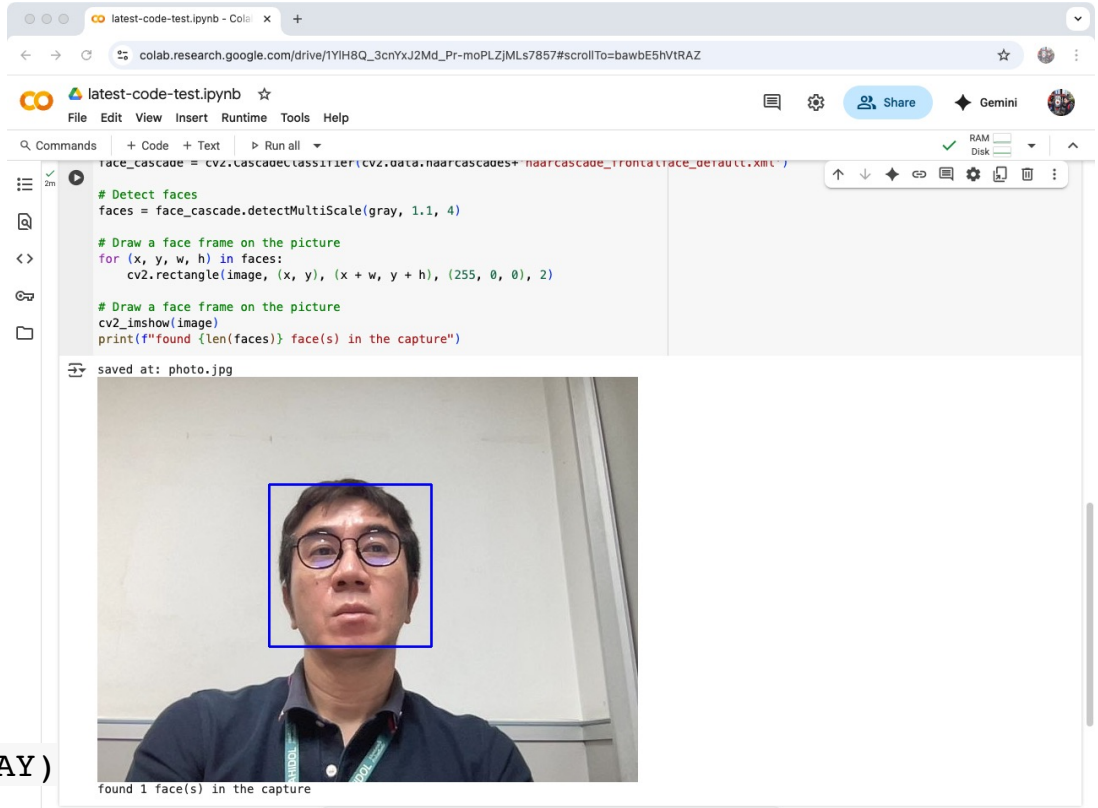
```
# capture
image_path = take_photo()
print(f"saved at: {image_path}")
```



# Webcam with Google Colab

## Module III: Camera Capturing

5. **Load images and detect faces**
   - Read images from files.
   - Convert images to grayscale for easier processing.
   - Load the Haar Cascade model for frontal face detection.



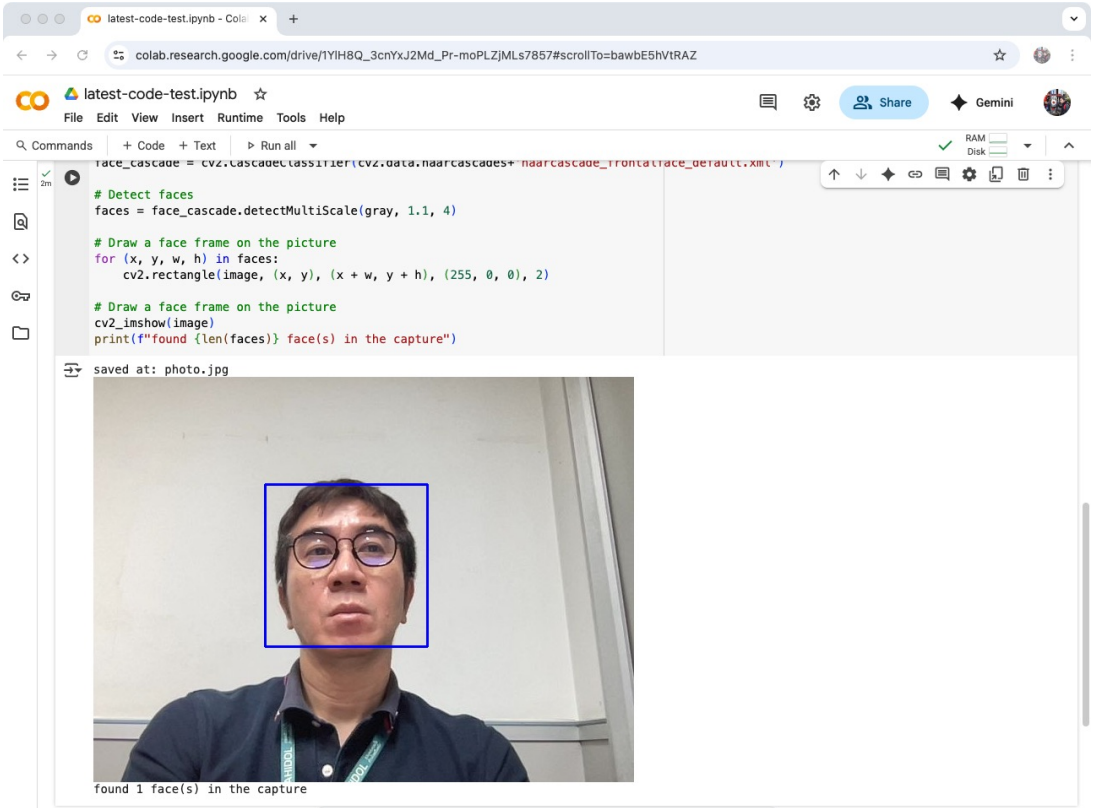

```
# Load images and detect faces
image = cv2.imread(image_path)
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Load Haar cascade for face
face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades+'haarcascade_frontalface_default.xml')
```

# Webcam with Google Colab

## Module III: Camera Capturing

5. Load images and detect faces
   - Grayscale Face Detection
   - `scaleFactor = 1.1`: The size of each image reduction step.
   - `minNeighbors = 4`: The number of neighbors required to confirm a face.

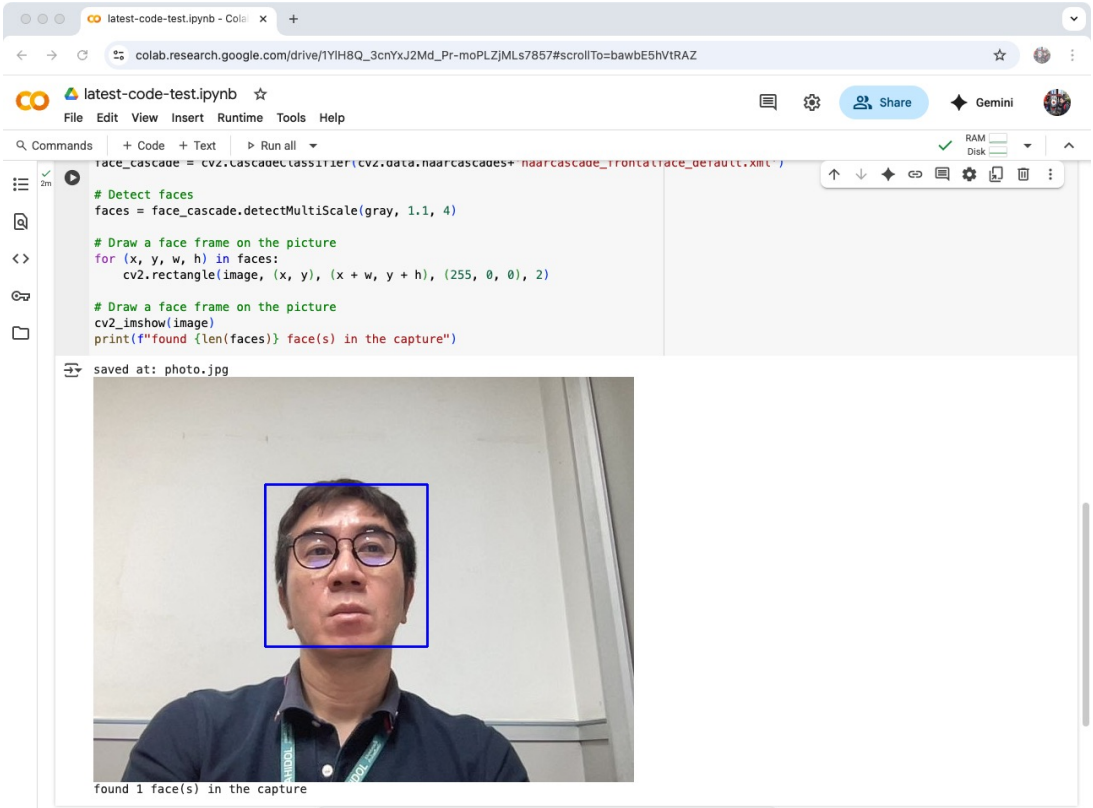



```
# Detect faces
faces = face_cascade.detectMultiScale(gray, 1.1, 4)
```

# Webcam with Google Colab

## Module III: Camera Capturing

6. **Draw a frame around the face you find**
   - Loop through the found face positions (x,y = origin, w,h = width-height).
   - Draw a blue rectangle (BGR: 255,0,0) around each face.



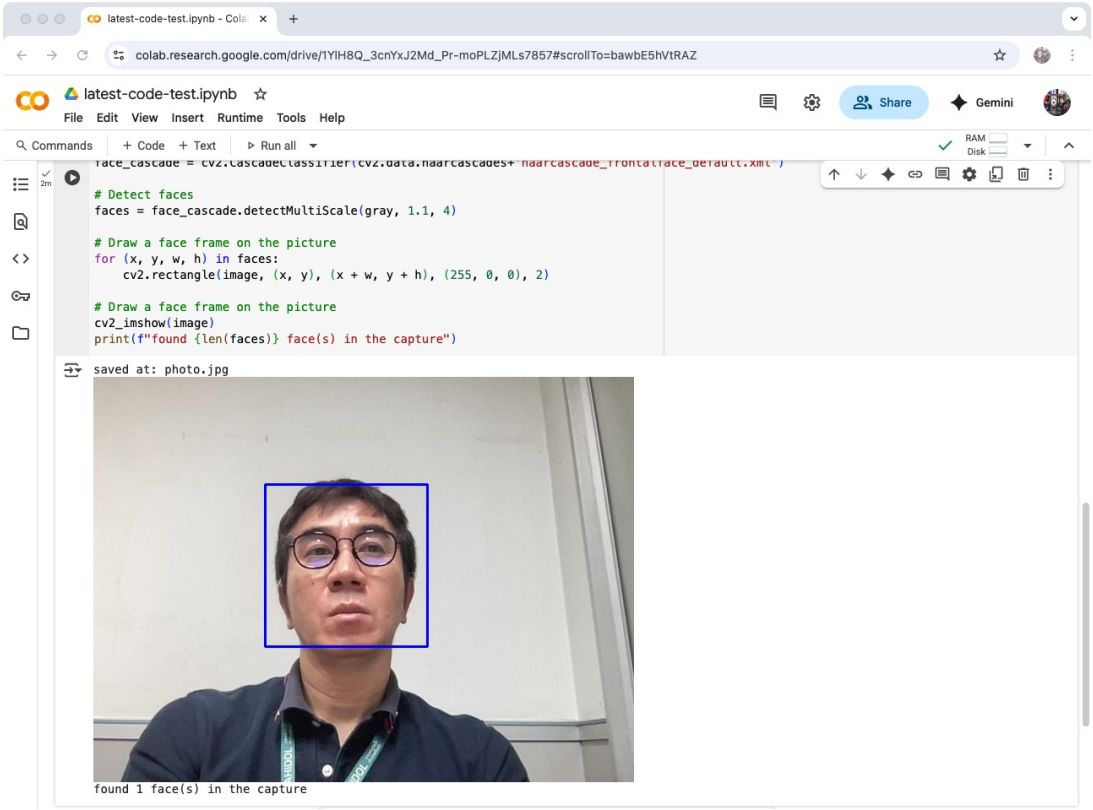

```
# Draw a face frame on the picture
for (x, y, w, h) in faces:
    cv2.rectangle(image, (x, y), (x + w, y + h), (255, 0, 0), 2)
```

# Webcam with Google Colab

## Module III: Camera Capturing

### 7. Show Results

- Show images with frames around faces
- Enter the number of faces found




```
# Draw a face frame on the picture
cv2_imshow(image)
print(f"found {len(faces)} face(s) in the capture")
```

# Webcam with Google Colab

## Module III: Camera Capturing

```
!pip install opencv-python-headless
```

```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow
from IPython.display import display, Javascript
from google.colab.output import eval_js
from base64 import b64decode
```

```
# Camera opening and photo taking function
def take_photo(filename='photo.jpg', quality=0.8):
    js = Javascript('''
      async function takePhoto(quality) {
        const div = document.createElement('div');
        const capture = document.createElement('button');
        capture.textContent = '📷 Capture';
        div.appendChild(capture);

        const video = document.createElement('video');
        video.style.display = 'block';
        const stream = await navigator.mediaDevices.getUserMedia({video: true});

        document.body.appendChild(div);
        div.appendChild(video);
        video.srcObject = stream;
        await video.play();

        google.colab.output.setIframeHeight(document.documentElement.scrollHeight, true);

        await new Promise((resolve) => capture.onclick = resolve);

        const canvas = document.createElement('canvas');
        canvas.width = video.videoWidth;
        canvas.height = video.videoHeight;
        canvas.getContext('2d').drawImage(video, 0, 0);
        stream.getVideoTracks()[0].stop();
        div.remove();

        return canvas.toDataURL('image/jpeg', quality);
      }
    ''')

    display(js)
    data = eval_js('takePhoto({})'.format(quality))
    binary = b64decode(data.split(',')[1])
    with open(filename, 'wb') as f:
        f.write(binary)
    return filename
```

```
# capture
image_path = take_photo()
print(f"saved at: {image_path}")

# Load images and detect faces
image = cv2.imread(image_path)
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Load Haar cascade for face
face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades+'haarcascade_frontalface_default.xml')

# Detect faces
faces = face_cascade.detectMultiScale(gray, 1.1, 4)

# Draw a face frame on the picture
for (x, y, w, h) in faces:
    cv2.rectangle(image, (x, y), (x + w, y + h), (255, 0, 0), 2)

# Draw a face frame on the picture
cv2_imshow(image)
print(f"found {len(faces)} face(s) in the capture")
```

Introduction
to

# Face Detection Concepts



# Face Detection

## Module III: Face Detection

- Face detection has been one of the hottest topics in computer vision for the past few years.
- This technology has been available for some years now and is being used all over the place.
  - From cameras that make sure faces are focused before you take a picture.
  - Facebook tags people automatically once you upload a picture.
  - Face Detection and Face Recognition work when unlocking your phone.




source: https://imgur.com/fHxbKnU

# Face Detection Classifiers

## Module III: Face Detection

- A computer program that decides whether an image is a positive image (face image) or a negative image (non-face image) is called a **classifier**.
- A classifier was trained on hundreds of thousands of face and non-face images to learn how to classify a new image correctly.
- **OpenCV** provides us with two pre-trained and ready to be used for face detection classifiers:
    - Haar Classifier
    - LBP (Local Binary Patterns) Classifier

# Face Detection Classifiers

## Module III: Face Detection

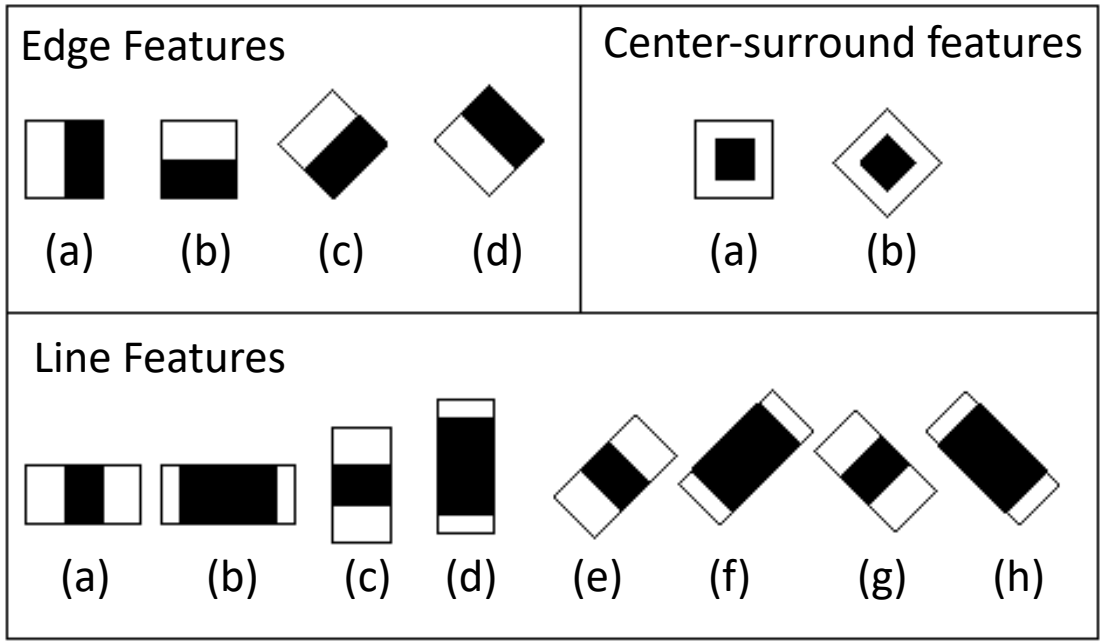
- Both the Haar classifier and the LBP classifier process images in gray scale.
- As these are pre-trained in OpenCV, their learned knowledge files also come bundled with OpenCV opencv/data/ or can be downloaded.
- To run a classifier, we need to load the knowledge files first.
- Each file starts with the name of the classifier it belongs to.
  - For example, a **Haar cascade classifier** starts as
    - `haarcascade_frontalface_alt.xml`.

# Haar Cascade Classifier

# Haar Classifier

## Module III: Face Detection

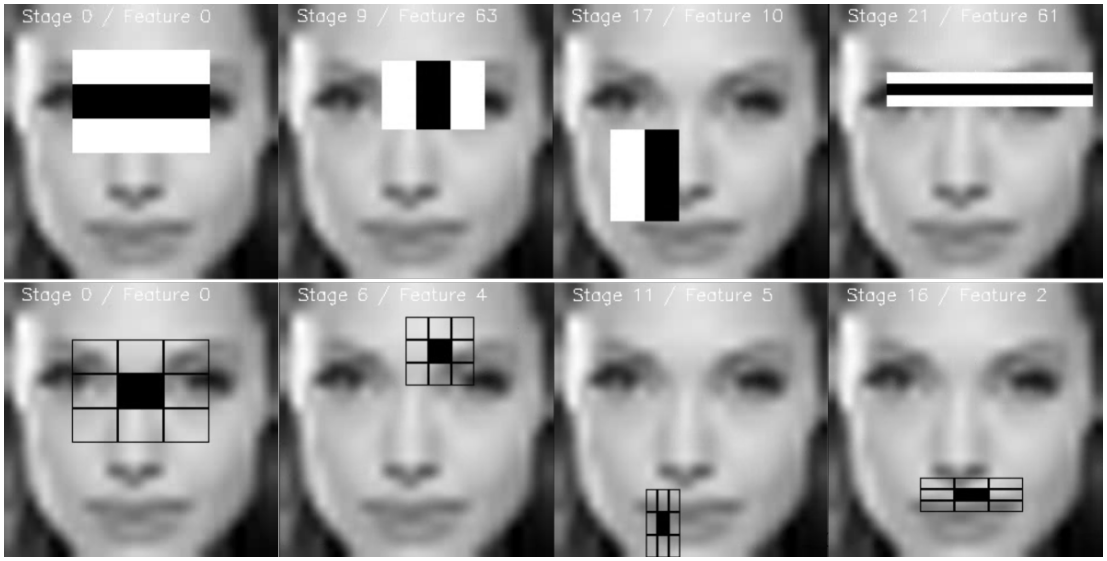
- The Haar Classifier is a machine learning based approach, an algorithm created by Paul Viola and Michael Jones, which (as mentioned before) is trained from many positive images (with faces) and negative images (without faces).
- It starts by extracting Haar features from each image, as shown by the windows below:

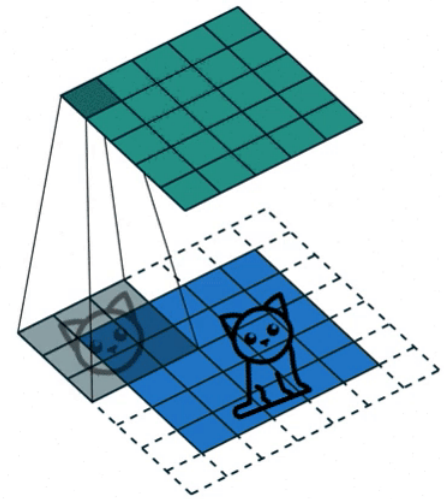


# Haar Classifier

## Module III: Face Detection

- Each window is placed on the picture to calculate a single feature.
- This feature is a single value obtained by **subtracting** the sum of pixels under the white part of the window from the sum of the pixels under the black part of the window.
- All possible sizes of each window are placed on all possible locations of each image to calculate plenty of features.



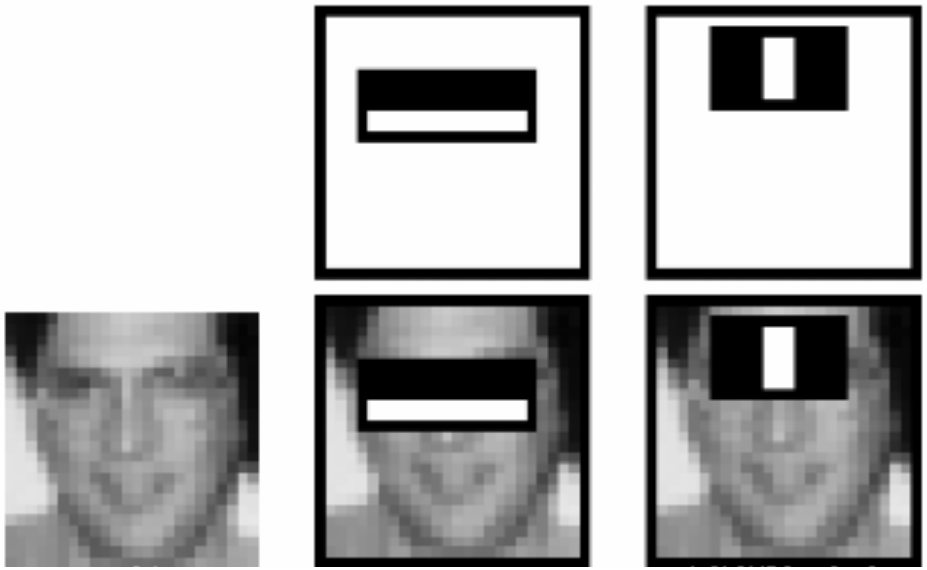



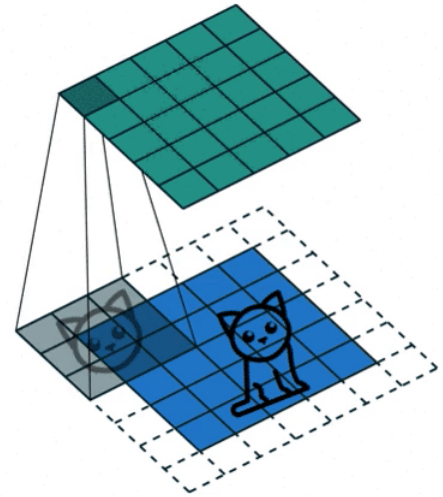
source: https://wikidocs.net/177768

# Haar Classifier

## Module III: Face Detection

- For example, in this image, we are extracting two features.
- The first one focuses on the property that the region of the eyes is often darker than the area of the nose and cheeks.
- The second feature relies on the property that the eyes are darker than the bridge of the nose.



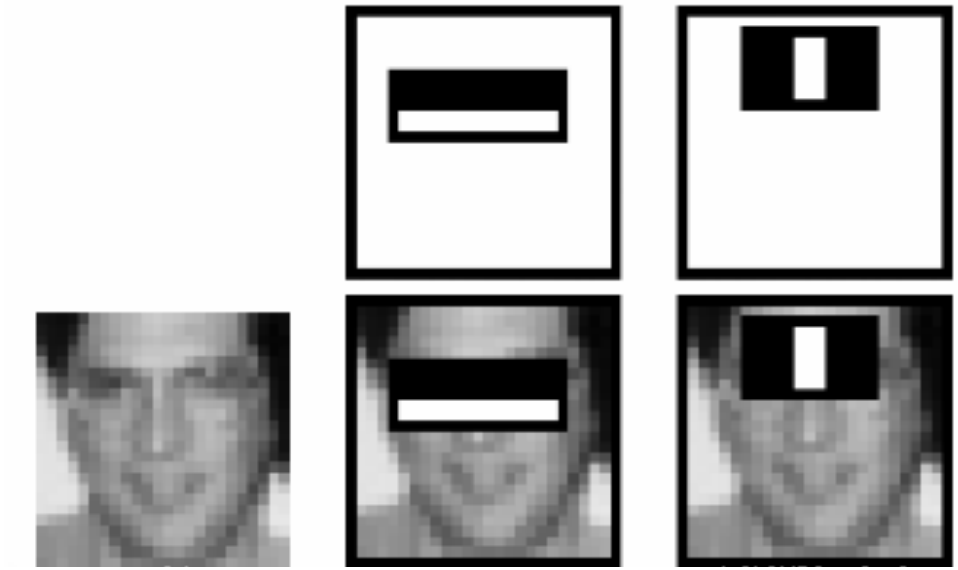


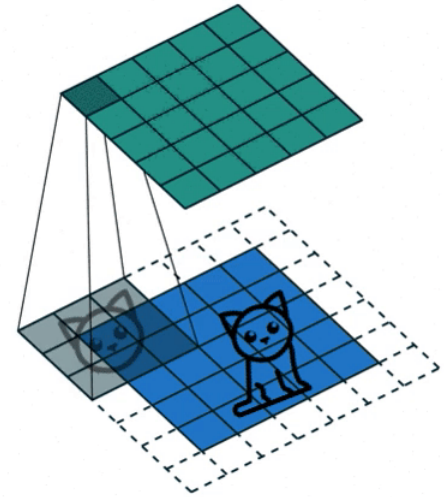
source: https://wikidocs.net/177768

# Haar Classifier

## Module III: Face Detection

- But among all these features calculated, some of them are irrelevant.
- For example, when used on the cheek, the windows become irrelevant because none of these areas are darker or lighter than other regions on the cheeks; all sectors here are the same.





source: https://wikidocs.net/177768

# AdaBoost (Adaptive Boosting)

## Module III: Face Detection

- So we promptly discard irrelevant features and keep only those relevant with a fancy technique called AdaBoost.
- AdaBoost is a training process for face detection, which selects only those features known to improve the classification (face/non-face) accuracy of our classifier.
- In the end, the algorithm considers the fact that generally, most of the region in an image is a non-face region.
- Considering this, it's a better idea to have a simple method to check if a window is a non-face region, and if it's not, discard it right away and don't process it again.
- So we can focus mostly on the area where the face is.

# AdaBoost (Adaptive Boosting)

## Module III: Face Detection

- AdaBoost (Adaptive Boosting) Summary
  - AdaBoost is a machine learning algorithm that selects and combines several important Haar-like features.
  - It improves accuracy by prioritizing difficult-to-classify examples.
  - In Viola-Jones' work, AdaBoost was used to select approximately 200 important Haar features from thousands of possible candidates.

# Haar Classifier

## Module III: Face Detection

- Summary of the Collaboration (Haar Classifier and Adaptive Boosting Collaboration)
- The Viola-Jones face detection system uses Haar-like features, AdaBoost, and Cascade classifier techniques to create a system that:
    - detects quickly
    - has high accuracy
    - can operate in real-time

- A well-known example of this is in OpenCV, using the `cv2.CascadeClassifier` function, which uses a pre-trained Haar cascade model.

# LBP Cascade Classifier

# LBP Cascade Classifier

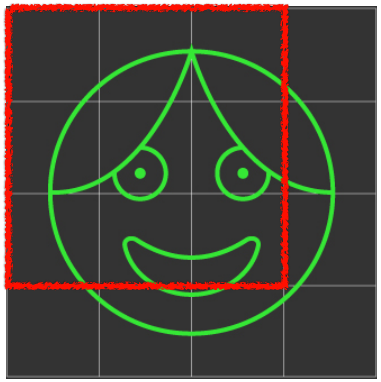
## Module III: Face Detection

- As any other classifier, the Local Binary Patterns, or LBP in short, also needs to be trained on hundreds of images.
- LBP is a visual/texture descriptor, and thankfully, our faces are also composed of microvisual patterns.
- So, LBP features are extracted to form a feature vector that classifies a face from a non-face.
- But how are LBP features found?

# LBP Cascade Classifier

## Module III: Face Detection

- Each training image is divided into some blocks, as shown in the picture below.
- For each block, LBP looks at 9 pixels (3×3 window) at a time, and with a particular interest in the pixel located in the center of the window.

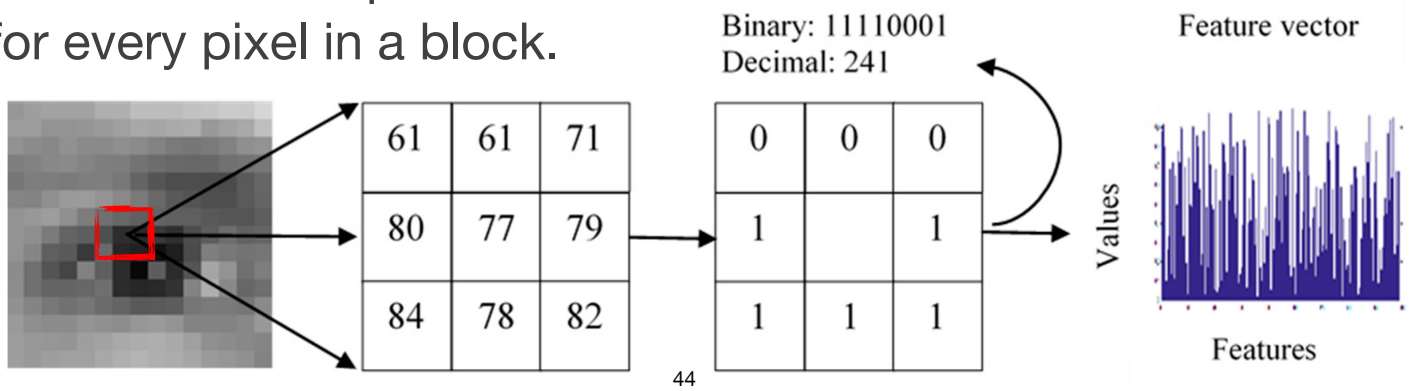


source: https://www.superdatascience.com/blogs/opencv-face-detection

# LBP Cascade Classifier

## Module III: Face Detection

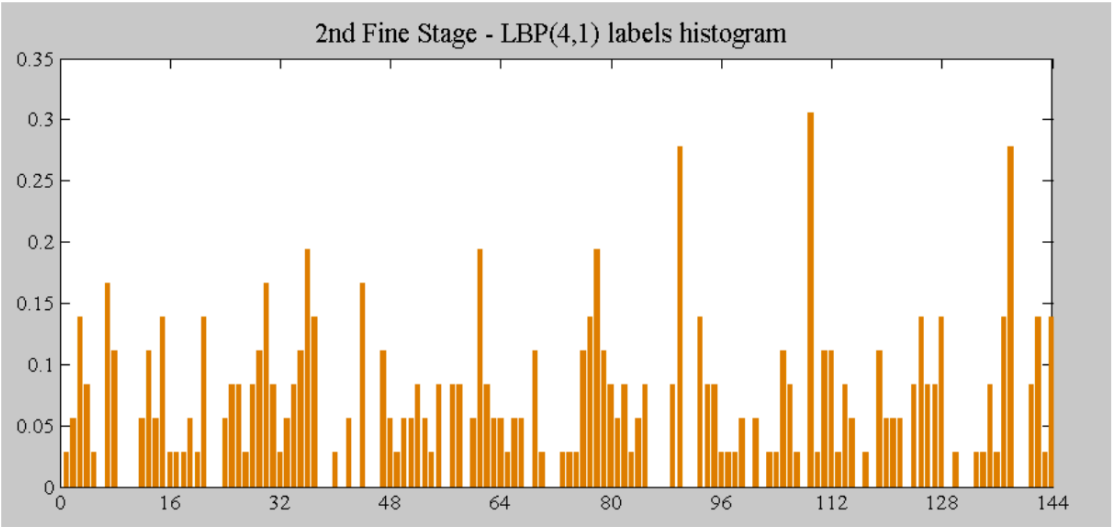
- Then, it compares the central pixel value with every neighbour's pixel value under the 3×3 window.
- For each neighbour pixel that is greater than or equal to the center pixel, it sets its value to 1, and for the others, it sets them to 0.
- After that, it reads the updated pixel values (which can be either 0 or 1) in a clockwise order and forms a binary number.
- Next, it converts the binary number into a decimal number, and that decimal number is the new value of the center pixel.
- We do this for every pixel in a block.



# LBP Cascade Classifier

## Module III: Face Detection

- Then, it converts each block value into a histogram, so now we have one histogram for each block in an image, like this:




- Finally, it concatenates these block histograms to form a one feature vector for one image, which contains all the features we are interested in.
- So, this is how we extract LBP features from a picture.

# Haar vs. LBP

## Module III: Face Detection

- Each OpenCV face detection classifier has its pros and cons, but the major differences are in accuracy and speed.

| Algorithm | Advantages | Disadvantages |
|---|---|---|
| Haar | 1. High detection accuracy<br>2. Low false positive rate | 1. Computationally complex and slow<br>2. Longer training time<br>3. Less accurate on black faces<br>4. Limitations in difficult lightening conditions<br>5. Less robust to occlusion |
| LBP | 1. Computationally simple and fast<br>2. Shorter training time<br>3. Robust to local illumination changes<br>4. Robust to occlusion | 1. Less accurate<br>2. High false positive rate |

Introduction
to

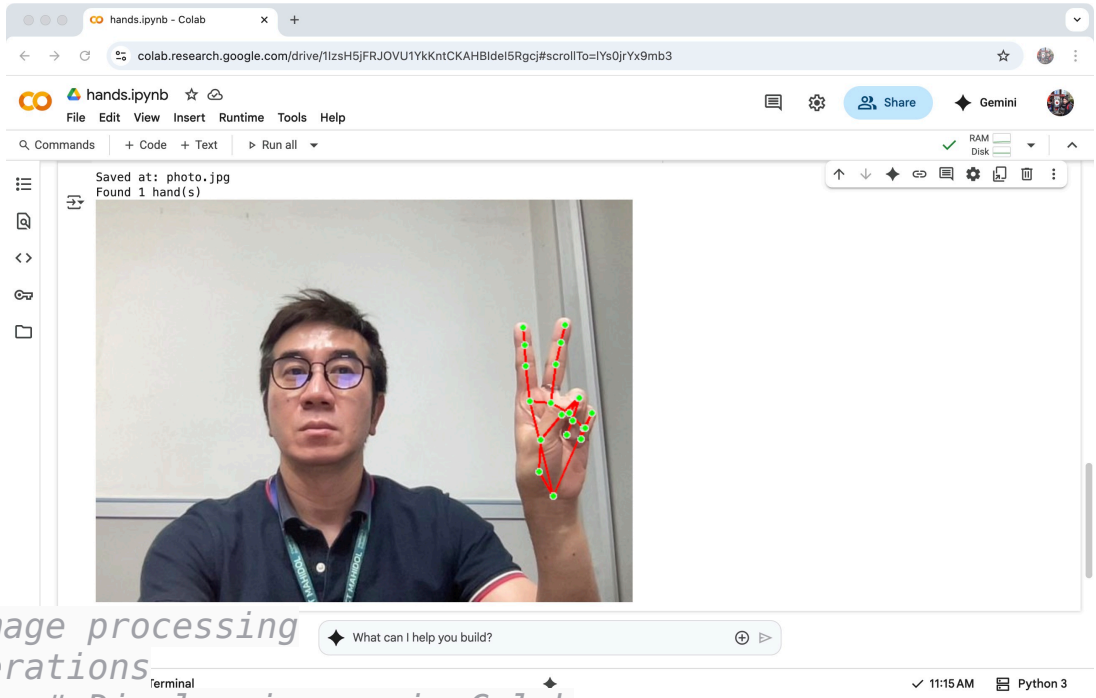
# Hands Detection Concepts



# Hands Detection

## Module III: Hand Detection

- Hands use MediaPipe Hands to detect and visualize hand landmarks in an image captured from a webcam in Google Colab. (see in detail next unit)




1. Install MediaPip Hands

```
!pip install mediapipe
```

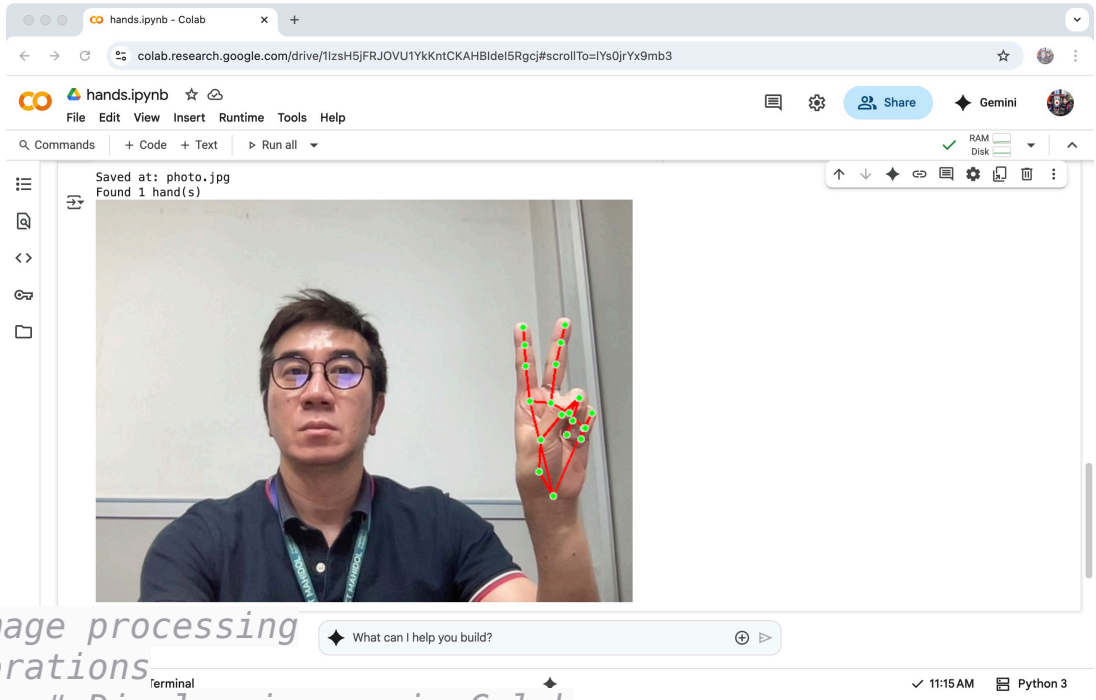
2. Import Required Libraries

```
import cv2                    # OpenCV for image processing
import numpy as np            # Numerical operations
from google.colab.patches import cv2_imshow  # Display images in Colab
from IPython.display import display, Javascript  # Run JavaScript in Colab
from google.colab.output import eval_js      # Evaluate JavaScript
from base64 import b64decode                 # Decode base64 images
import mediapipe as mp      # MediaPipe for hand detection
```

# Hands Detection

## Module III: Hand Detection

2. Import Required Libraries
   - OpenCV (cv2) → Processes images and draws landmarks.
   - MediaPipe (mp) → Detects hand landmarks.
   - Google Colab Utilities → Enable webcam access and image display.




```python
import cv2                      # OpenCV for image processing
import numpy as np              # Numerical operations
from google.colab.patches import cv2_imshow  # Display images in Colab
from IPython.display import display, Javascript  # Run JavaScript in Colab
from google.colab.output import eval_js      # Evaluate JavaScript
from base64 import b64decode                 # Decode base64 images
import mediapipe as mp      # MediaPipe for hand detection
```

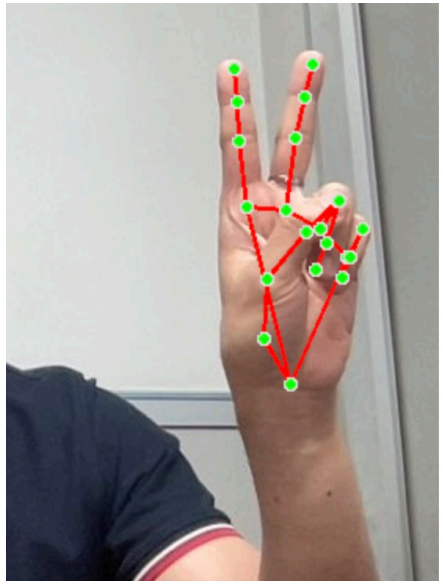
# Hands Detection

## Module III: Hand Detection

3. Initialize MediaPipe Hands

```
mp_hands = mp.solutions.hands
hands = mp_hands.Hands(max_num_hands=2)  # Detect up to 2 hands
mp_drawing = mp.solutions.drawing_utils  # For drawing landmarks
```

- `mp_hands.Hands()`
  - `max_num_hands=2` → Detects two hands (can be adjusted).
  - Uses machine learning models to find 21 key points per hand.
- `mp_drawing` → Helps visualize landmarks and connections.



# Hands Detection

## Module III: Hand Detection

4. Webcam Capture Function
   - Creates a Webcam Stream
     - Uses `getUserMedia` to access the webcam.
     - Displays a “Capture” button to take a photo.
   - Captures and Saves the Image
     - Converts the video frame into a canvas image.
     - Stops the webcam stream after capture.
     - Saves the image as `photo.jpg`.

```
def take_photo(filename='photo.jpg', quality=0.8):
    js = Javascript('''
      async function takePhoto(quality) {
        const div = document.createElement('div');
        const capture = document.createElement('button');
        capture.textContent = '📷 Capture';
        div.appendChild(capture);

        const video = document.createElement('video');
        video.style.display = 'block';
        const stream = await navigator.mediaDevices.getUserMedia({video: true});

        document.body.appendChild(div);
        div.appendChild(video);
        video.srcObject = stream;
        await video.play();

        google.colab.output.setIframeHeight(document.documentElement.scrollHeight, true);

        await new Promise((resolve) => capture.onclick = resolve);

        const canvas = document.createElement('canvas');
        canvas.width = video.videoWidth;
        canvas.height = video.videoHeight;
        canvas.getContext('2d').drawImage(video, 0, 0);
        stream.getVideoTracks()[0].stop();
        div.remove();

        return canvas.toDataURL('image/jpeg', quality);
      }
    ''')
    display(js)
    data = eval_js('takePhoto({})'.format(quality))
    binary = b64decode(data.split(',')[1])
    with open(filename, 'wb') as f:
        f.write(binary)
    return filename
```

# Hands Detection

## Module III: Hand Detection

5. Capture an Image

```
image_path = take_photo()
print(f"Saved at: {image_path}")
```

- Calls `take_photo()` to capture an image.
- Prints the saved file path.

# Hands Detection

## Module III: Hand Detection

6. Process the Image with MediaPipe

```
image = cv2.imread(image_path)  # Read the captured image
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)  # Convert BGR to RGB
results = hands.process(image_rgb)  # Detect hands
```

- `cv2.imread()` → Loads the image.
- `cv2.cvtColor()` → Converts from BGR (OpenCV default) to RGB (MediaPipe requirement).
- `hands.process()` → Runs hand detection and returns landmarks.

# Hands Detection

## Module III: Hand Detection

7. Draw Hand Landmarks
   - Check if Hands are Detected (`results.multi_hand_landmarks`).
   - Loop Through Each Detected Hand:
     - `mp_drawing.draw_landmarks()` → Draws:
     - Landmarks (`green dots`, (`0`, `255`, `0`)).
     - Connections (`red lines`, (`0`, `0`, `255`)).
   - Print Detection Status (`number of hands found`).

```
if results.multi_hand_landmarks:
    for hand_landmarks in results.multi_hand_landmarks:
        mp_drawing.draw_landmarks(
            image, hand_landmarks, mp_hands.HAND_CONNECTIONS,
            mp_drawing.DrawingSpec(color=(0, 255, 0), thickness=2, circle_radius=2),
            mp_drawing.DrawingSpec(color=(0, 0, 255), thickness=2))
    print(f"Found {len(results.multi_hand_landmarks)} hand(s)")
else:
    print("No hands detected")
```

# Hands Detection

## Module III: Hand Detection

7. Display the Result

```
cv2_imshow(image)  # Show the image with landmarks
```

- Summary of Workflow
  - Access Webcam → Capture an image.
  - Detect Hands → MediaPipe processes the image.
  - Visualize Landmarks → Draws key points and connections.
  - Display Output → Shows the annotated image.

Python Programming
on

# Colab



# Colab: User-defined Functions

## Module III: Straight Line

Step 1: Import IPython and output

```
import IPython
from google.colab import output
```

Step 2: Create embedded html code for <head></head>

```
html_code = """
<!DOCTYPE html>
<html>

<head>
<title>Interactive Bresenham's Line Algorithm</title>
<style>
  body {
    margin: 0;
    overflow: hidden;
  }
  canvas {
    border: 1px solid black;
    cursor: crosshair;
  }
</style>
</head>
```

# Colab: User-defined Functions

## Module III: Straight Line

Step 3: Create embedded html code for <body><canvas>…</canvas></body>

```
#… continue

<body>

<canvas id="myCanvas"></canvas>

<script>
const canvas = document.getElementById('myCanvas');
const ctx = canvas.getContext('2d');

canvas.width = 500;
canvas.height = 300;

let startPoint = null;

#… continue
```

# Colab: User-defined Functions

## Module III: Straight Line

Step 3: Create embedded html code for <body><canvas>…</canvas></body>

```
#… continue

canvas.addEventListener('click', (event) => {
  const rect = canvas.getBoundingClientRect();
  const x = event.clientX - rect.left;
  const y = event.clientY - rect.top;

  if (!startPoint) {
    startPoint = { x, y };
    ctx.clearRect(0, 0, canvas.width, canvas.height);
    ctx.fillRect(x - 2, y - 2, 4, 4); // Mark the start point
  } else {
    bresenhamLine(startPoint.x, startPoint.y, x, y);
    startPoint = null;
  }
});

#… continue
```

# Colab: User-defined Functions

## Module III: Straight Line

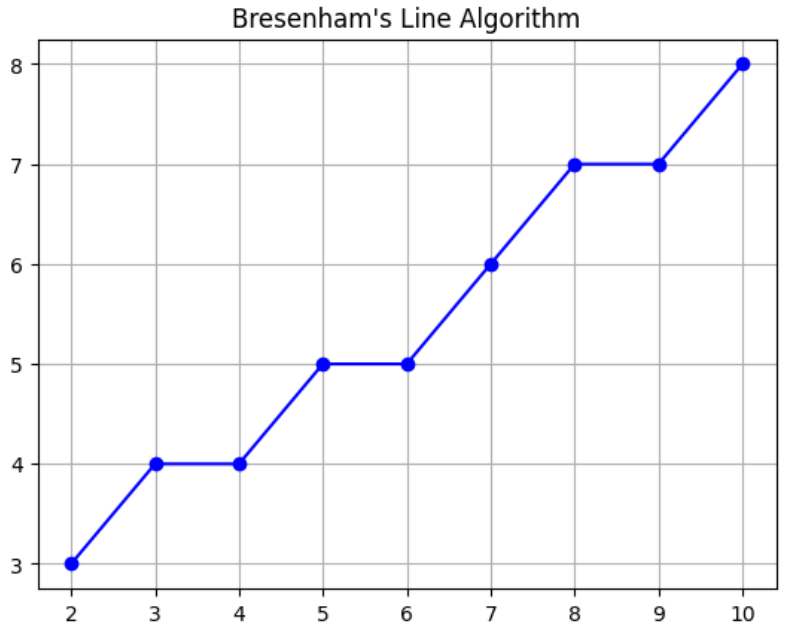
Step 3: Create embedded html code for <body><canvas>…</canvas></body>

```
#… continue

function bresenhamLine(x0, y0, x1, y1) {
    let dx = Math.abs(x1 - x0);
    let dy = Math.abs(y1 - y0);
    let sx = (x0 < x1) ? 1 : -1;
    let sy = (y0 < y1) ? 1 : -1;
    let err = dx - dy;

    function draw() {
        ctx.fillRect(x0, y0, 2, 2);
        if ((x0 === x1) && (y0 === y1)) return;
        let e2 = 2 * err;
        if (e2 > -dy) { err -= dy; x0 += sx; }
        if (e2 < dx) { err += dx; y0 += sy; }
        requestAnimationFrame(draw);
    }
    draw();
}

#… continue
```



# Colab: User-defined Functions

## Module III: Straight Line

Step 4: Ending HTML with </html>

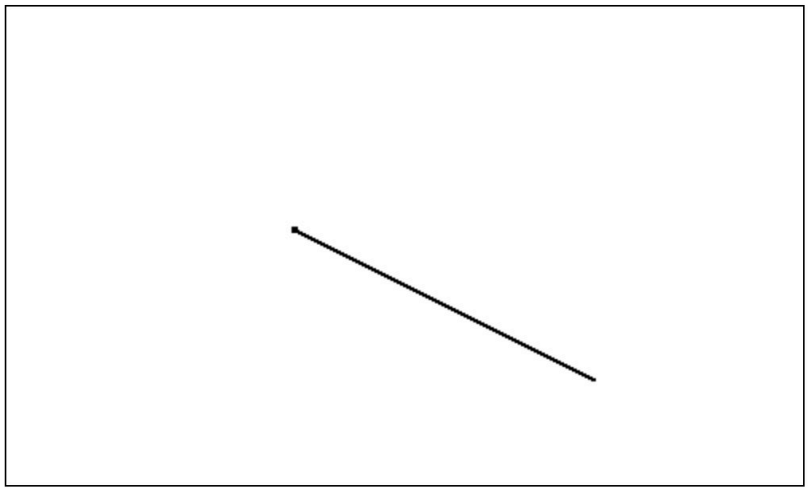
```
#… continue

</script>

</body>
</html>
"""
```

Step 5: Display an interactive screen

```
display(IPython.display.HTML(html_code))
```



Concepts
of

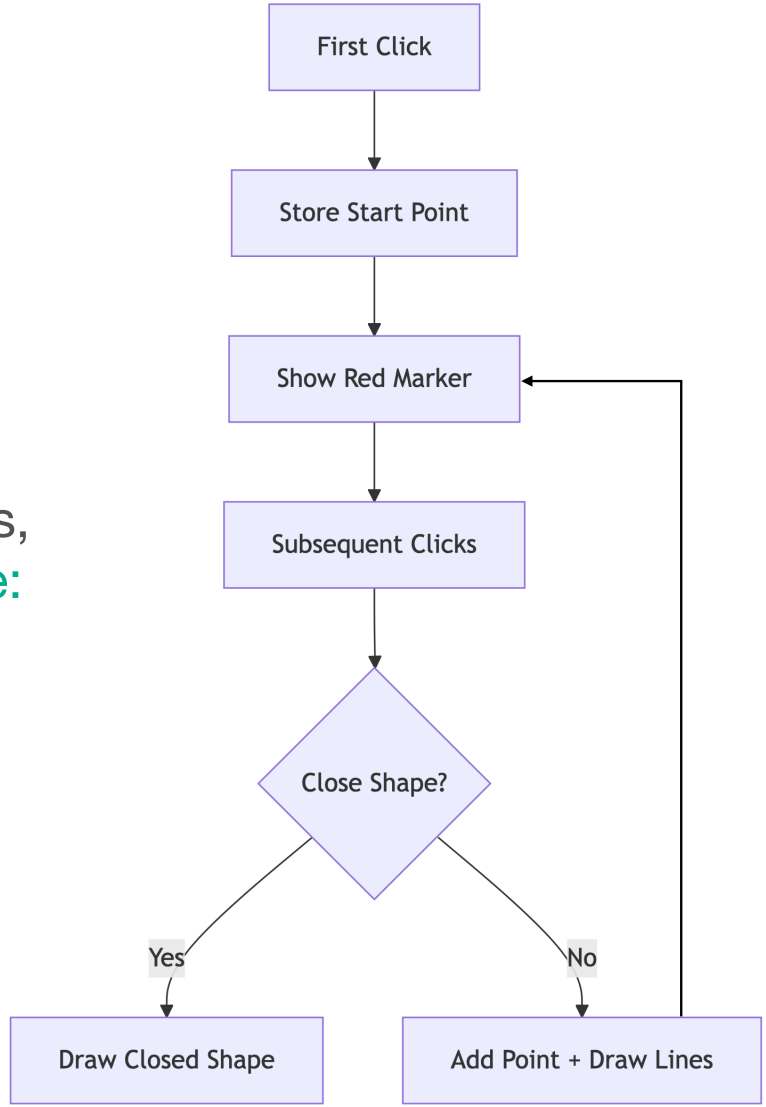
# Closed Shape Drawing



# Closed Shape Drawing

## Module III: Close Shape Drawing

- This creates an interactive tool where users can:
  - Draw free-form shapes
  - Get automatic shape closing
  - Visualize both open and closed paths
  - All with clean visual feedback (red points, blue lines, green closed shapes) — use the code in myCourse: Closed Shape Code 02 (in class Workshop 1)



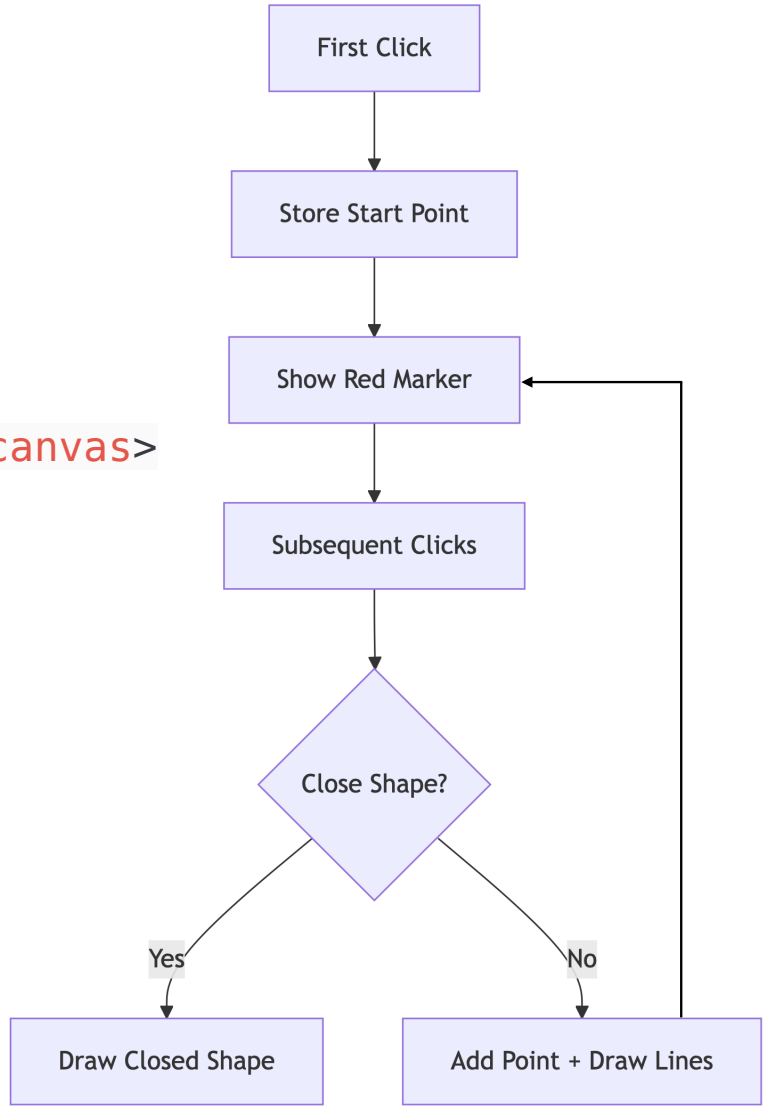
# Closed Shape Drawing

## Module III: Close Shape Drawing

- Step 1: HTML Canvas Setup
  - Creates a 500x300 pixel drawing canvas with a black border.

```
<canvas id="myCanvas" width="500" height="300"></canvas>
```

  - Uses crosshair cursor for precise pointing.



# Closed Shape Drawing

## Module III: Close Shape Drawing

- Step 2: JavaScript Core Components

  **Initialization**

  - Gets the canvas element and 2D drawing context.
  - Initializes an empty array to store points.
  - The `isDrawing` flag indicates whether the user is actively drawing.

```
const canvas = document.getElementById('myCanvas');
const ctx = canvas.getContext('2d');
let points = []; // Stores all drawn points
let isDrawing = false; // Tracks drawing state
```

# Closed Shape Drawing

## Module III: Close Shape Drawing

- Step 2: JavaScript Core Components

  **Mouse Click Handler**

  - Calculates the exact click position relative to the canvas.
  - Gets (x,y) coordinates adjusted for canvas offset.

```
canvas.addEventListener('click', (event) => {
  const rect = canvas.getBoundingClientRect();
  const x = event.clientX - rect.left;
  const y = event.clientY - rect.top;

  // ... drawing logic ...

});
```

# Closed Shape Drawing

## Module III: Close Shape Drawing

- Step 3: Drawing Logic

  **Starting a New Shape**

  - First click starts a new shape.
  - Stores the initial point and draws a 4x4 red square marker.

```
if (!isDrawing) {
  points = [{x, y}]; // Initialize with first point
  isDrawing = true;
  ctx.fillRect(x - 2, y - 2, 4, 4); // Draw red start point
}
```

# Closed Shape Drawing

## Module III: Close Shape Drawing

- Step 3: Drawing Logic

  **Adding Points & Auto-Closing**

  - Calculates the distance to the starting point.
  - If within 10 pixels and at least 2 points exist:
    - Closes the shape by adding the start point again.
    - Calls `drawShape()` for closed shape rendering.
  - Otherwise, adds a new point and draws connecting lines.

```
const distance = Math.sqrt((x - firstPoint.x) ** 2 + (y - firstPoint.y) ** 2);
if (distance <= 10 && points.length >= 2) {
  points.push({x: firstPoint.x, y: firstPoint.y}); // Close shape
  drawShape();
  isDrawing = false;
} else {
  points.push({x, y}); // Add intermediate point
  drawLines();
}
```

# Closed Shape Drawing

## Module III: Close Shape Drawing

- Step 4: Rendering Functions

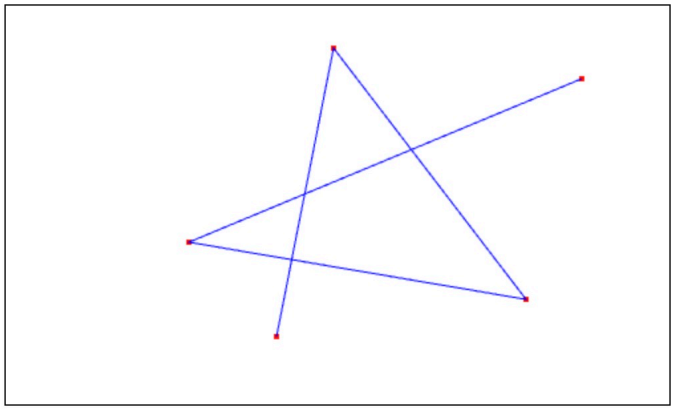
  **`drawLines()` Function**

  - Clears the canvas
  - Draws all recorded points as red squares
  - Connects points with blue lines

```
function drawLines() {
  ctx.clearRect(0, 0, canvas.width, canvas.height); // Clear canvas

  // Draw all points as red squares
  points.forEach(point => {
    ctx.fillRect(point.x - 2, point.y - 2, 4, 4);
  });

  // Connect points with blue lines
  ctx.beginPath();
  ctx.moveTo(points[0].x, points[0].y);
  for (let i = 1; i < points.length; i++) {
    ctx.lineTo(points[i].x, points[i].y);
  }
  ctx.strokeStyle = 'blue';
  ctx.stroke();
}
```



# Closed Shape Drawing

## Module III: Close Shape Drawing

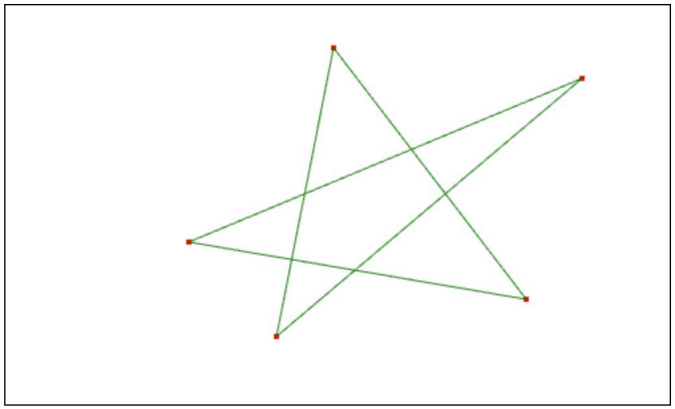
- Step 4: Rendering Functions

  **`drawShape()` Function**

  - First draws open shape (blue lines)
  - If closed (≥3 points), adds a green outline connecting the last point to the first.

```
function drawShape() {
  drawLines(); // Draw open shape first

  // For closed shapes (≥3 points)
  if (points.length >= 3) {
    ctx.beginPath();
    ctx.moveTo(points[0].x, points[0].y);
    // Draw connecting lines
    for (let i = 1; i < points.length; i++) {
      ctx.lineTo(points[i].x, points[i].y);
    }
    ctx.closePath(); // Automatically connects last to first point
    ctx.strokeStyle = 'green';
    ctx.stroke(); // Draw closed shape in green
  }
}
```



# Closed Shape Drawing

## Module III: Close Shape Drawing

- Step 5: Rendering Functions

  **Bresenham's Line Algorithm**

  - Efficient pixel-perfect line drawing algorithm
  - Not actively used in main flow (commented out in original)
  - Demonstrates how line rendering could be optimized

```
function bresenhamLine(x0, y0, x1, y1) {
  // Algorithm setup
  let dx = Math.abs(x1 - x0);
  let dy = Math.abs(y1 - y0);

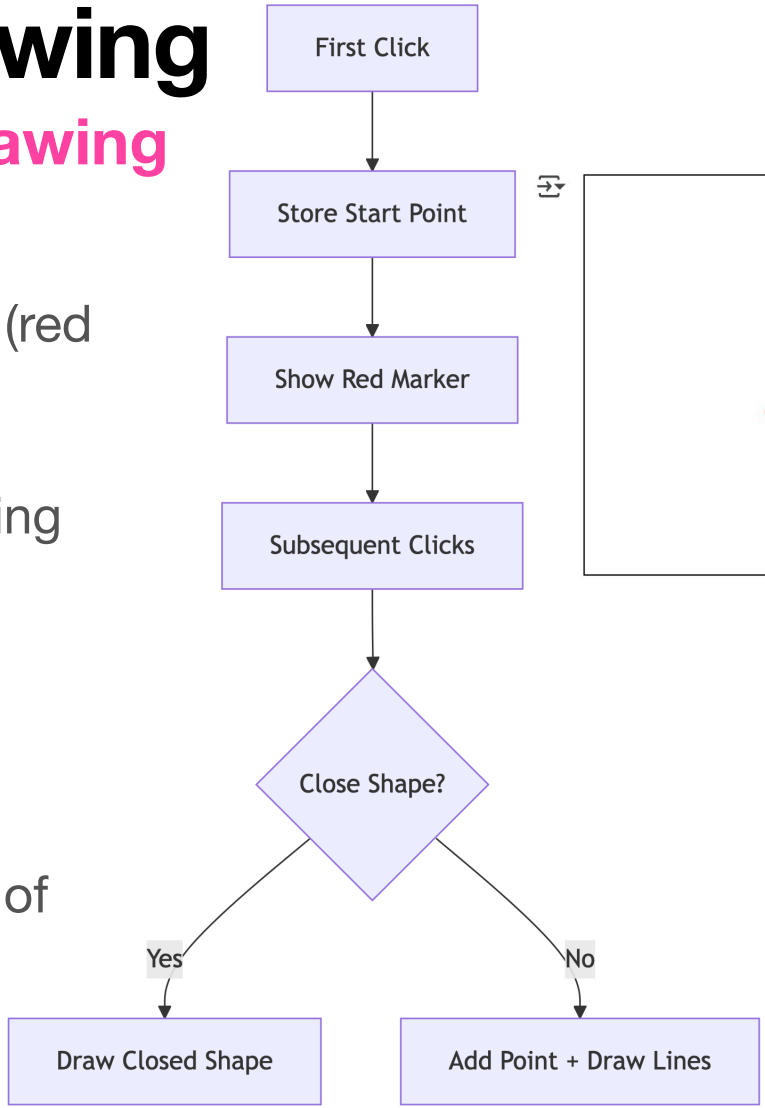
  // ... algorithm steps ...

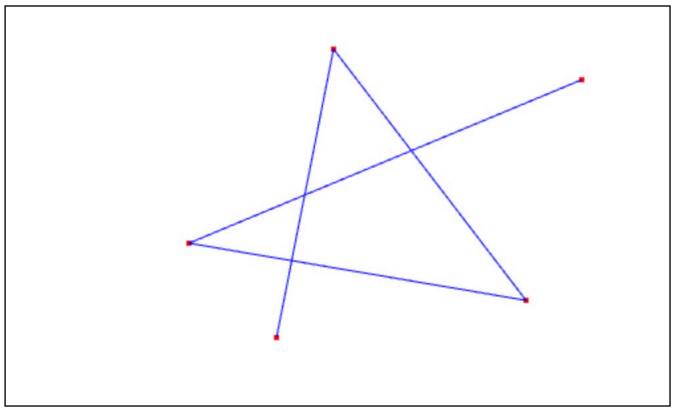
  function draw() {
    ctx.fillRect(x0, y0, 2, 2); // Draw pixel
    if ((x0 === x1) && (y0 === y1)) return;
    // ... update coordinates ...
    requestAnimationFrame(draw); // Animate
  }
  draw();
}
```

# Closed Shape Drawing

## Module III: Close Shape Drawing

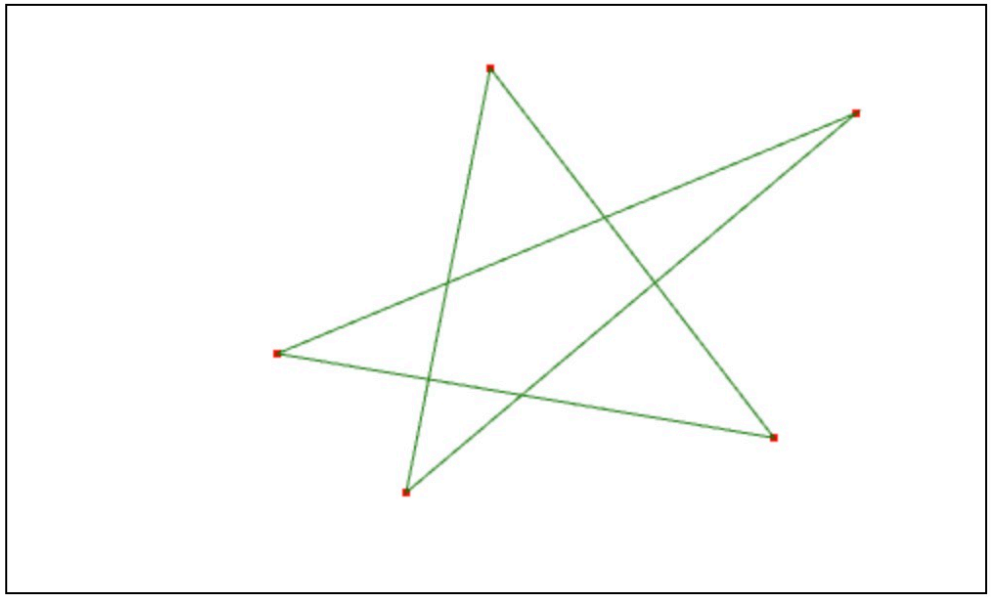
- User Workflow
  1. First Click: Starts new shape (red marker appears).
  2. Subsequent Clicks:
     - Adds points (blue connecting lines).
     - Automatically closes when near the start point (green outline).
  3. Auto-Close:
     - Triggers when within 10px of the start.
     - Requires a minimum of 3 points for a closed shape.

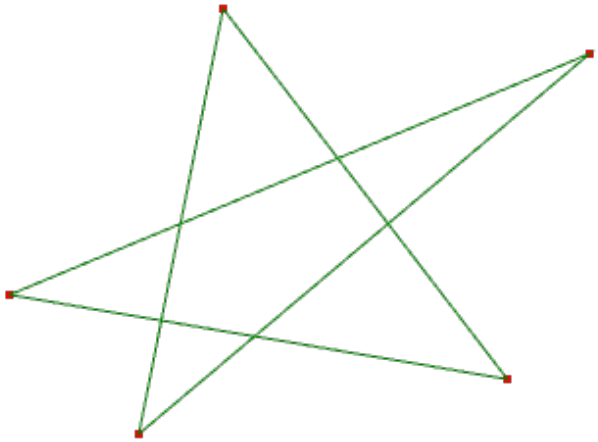


# Closed Shape Drawing

## Module III: Close Shape Drawing





Introduction
to

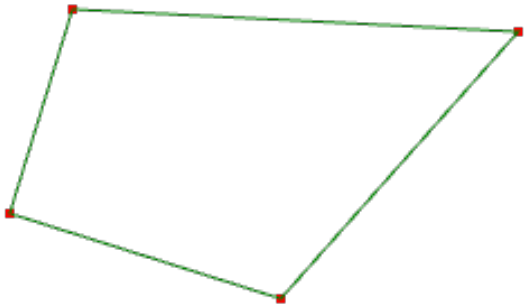
# 2D Transformation



# 2D Object

## Module III: 2D Transformation

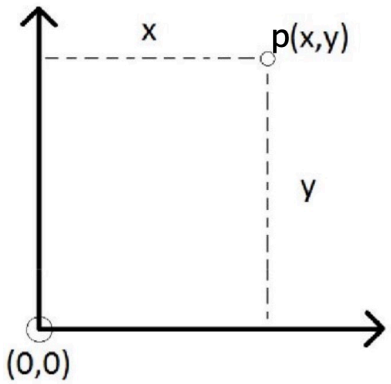
- 2D Object
  - Points / Vertices
  - Line Segments / Edges
  - Shape / Polygon (Face)



# 2D Transformations

## Module III: 2D Transformations

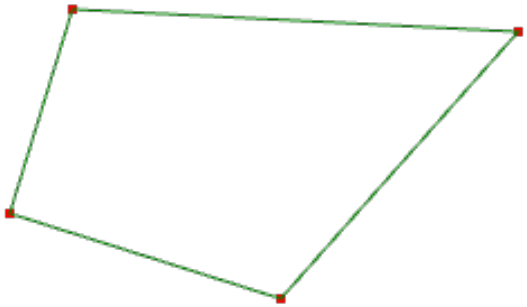
- Representation using points, such as p(x, y)
  - We usually present it as a 2x1 matrix
  - Concerning
    - the origin (Point)
    - and the x- and y-coordinate axes (Basis vectors)



# 2D Transformations

## Module III: 2D Transformations

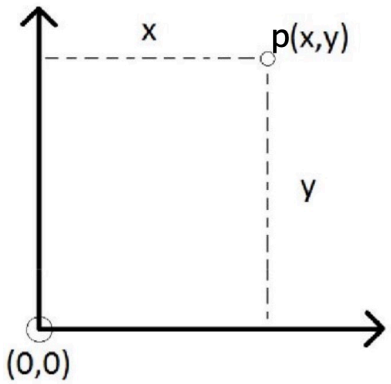
- Transformation
  - Position (Translation)
  - Size (Scaling)
  - Orientation (Rotation)
  - Shape (Shear)



# 2D Transformations

## Module III: 2D Transformations

- Representation using points, such as p(x, y)
  - We usually present it as a 2x1 matrix
  - Concerning
    - the origin (Point)
    - and the x- and y-coordinate axes (Basis vectors)



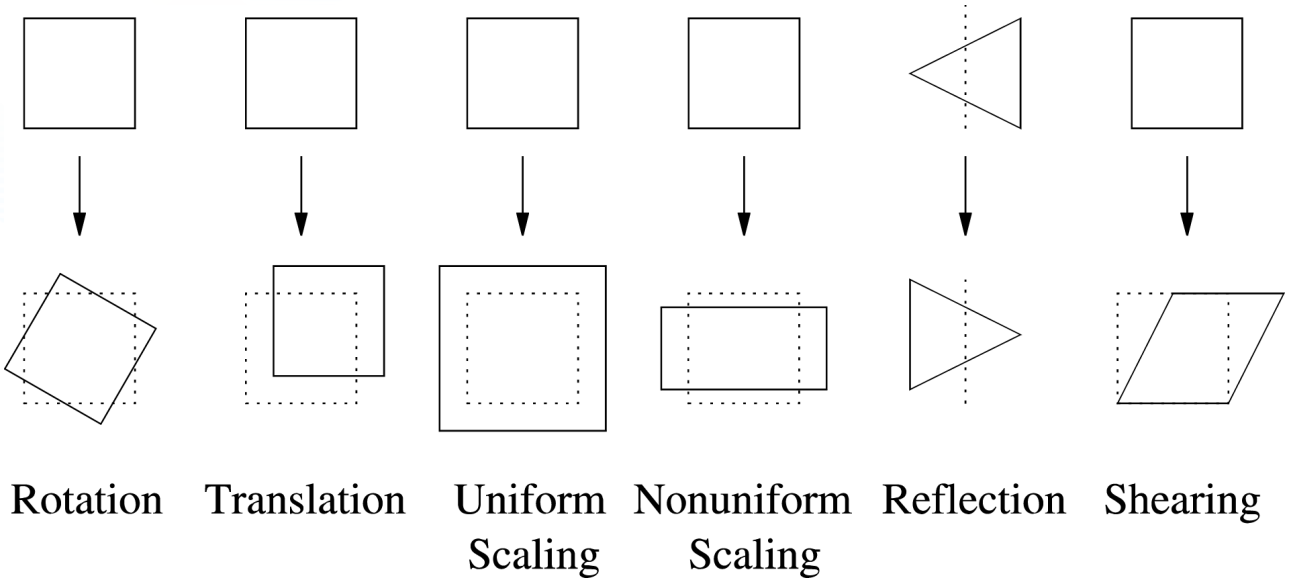
# 2D Transformations

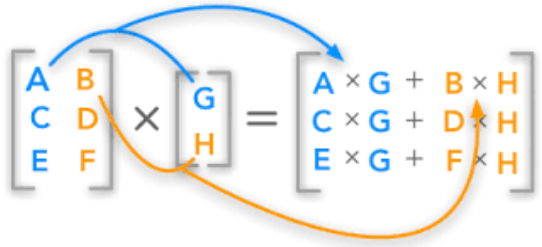
## Module III: 2D Transformations

- Perform Matrix Multiplication

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} ax + by \\ cx + dy \end{bmatrix}$$

$$\begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} ax + by + c \\ dx + ey + f \\ 1 \end{bmatrix}$$

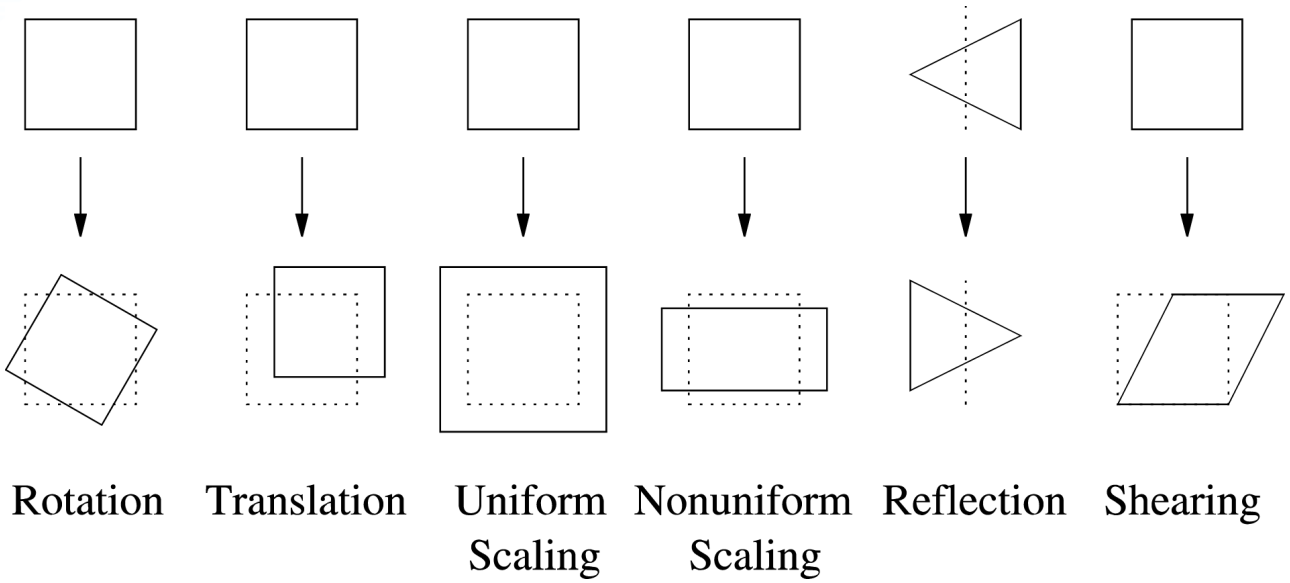


# 2D Affine Transformation

## Module III: 2D Transformations

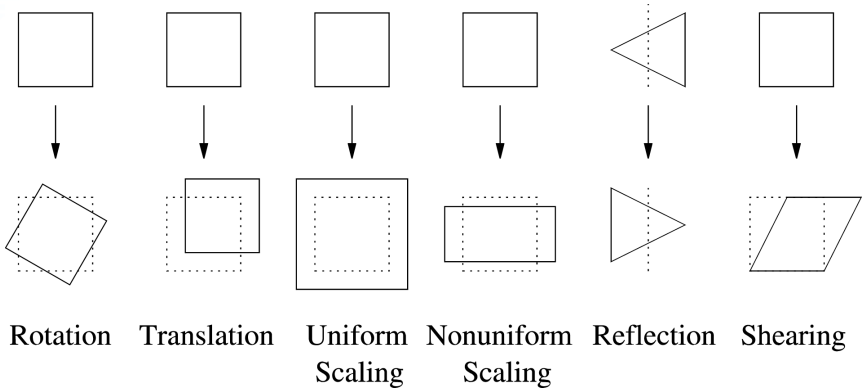
- All represented as matrix operations on vectors:
  - Translate
  - Scale
  - Rotate
  - Reflect
  - Shear

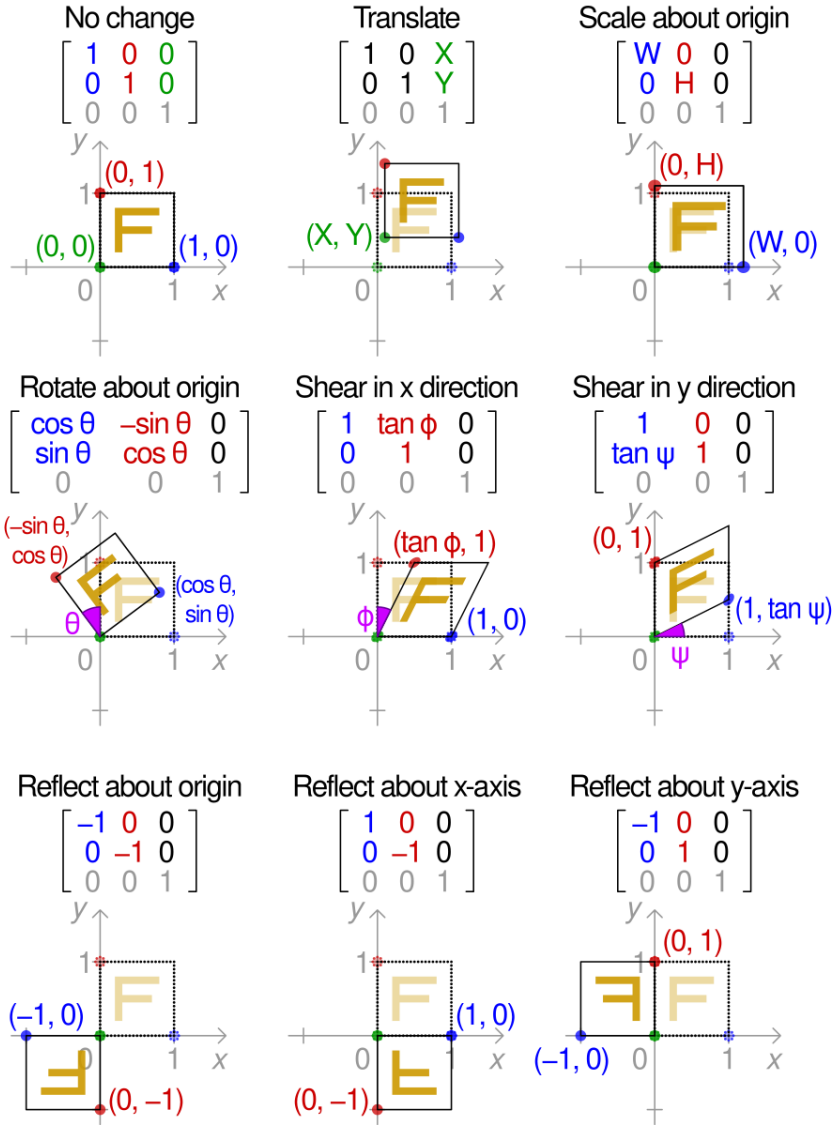


# 2D Affine Transformation

## Module III: 2D Transformations

- All represented as matrix operations on vectors:
  - Translate
  - Scale
  - Rotate
  - Reflect
  - Shear




source: https://commons.wikimedia.org/wiki/File:2D_affine_transformation_matrix.svg

Introduction
to

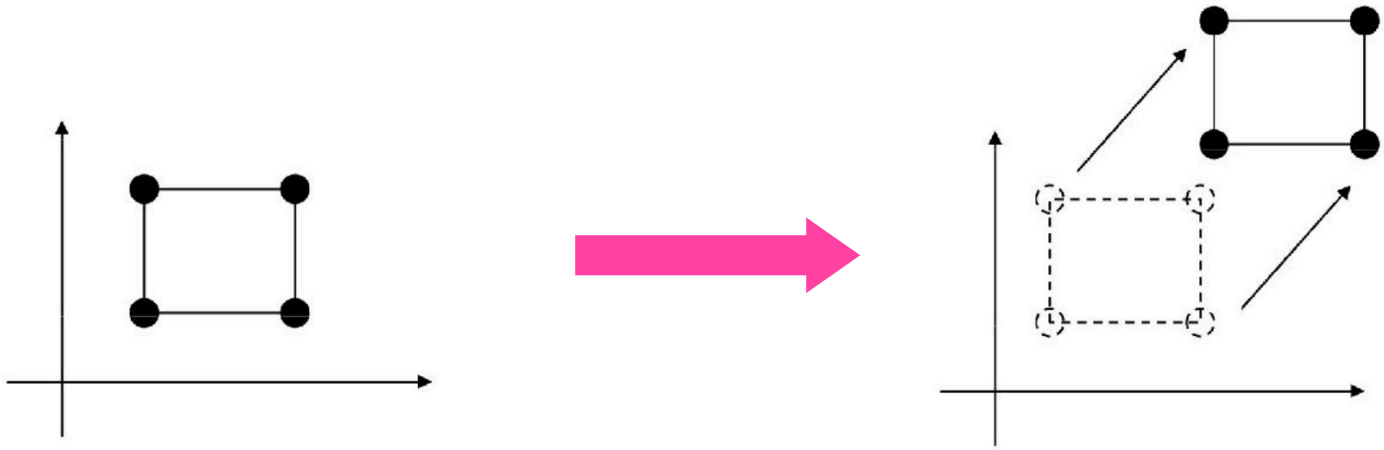
# 2D Translation



# 2D Translation Transformation

## Module III: 2D Translation

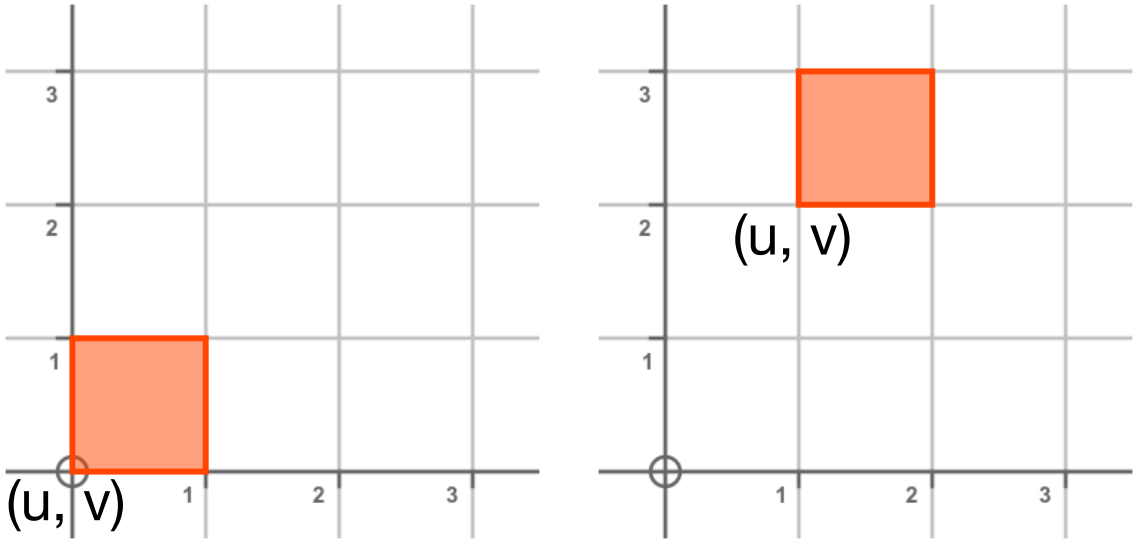
- Translation of multiple vertices
  - This involves moving the positions of individual points that make up a 2D object.



# 2D Translation Transformation

## Module III: 2D Translation

- Translating a shape means moving it to a different position, without changing its shape or orientation in any way.
- This diagram illustrates a translation by (u, v) = (1, 2).
- The square moves by 1 unit in the x-direction and 2 units in the y-direction:




source: https://graphicmaths.com/pure/matrices/matrix-2d-transformations/

# 2D Translation Transformation

## Module III: 2D Translation

- We can represent this by adding 2 vectors, the original position (x, y) and a displacement vector (u, v):

$$\begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} u \\ v \end{pmatrix} = \begin{pmatrix} x + u \\ y + v \end{pmatrix}$$

- But this isn't ideal.
- In all the other cases, we used matrix multiplication to represent the transform.
- It is a bit inconvenient to have to use a different calculation for translation.
- And in a future article, you will see that we can combine multiple transforms into a single matrix.
  - To do that, we need a consistent way to represent all transformations.

# 2D Translation Transformation

## Module III: 2D Translation

- It turns out that we can do this by using a 3 by 3 matrix.
- We extend our transformation matrix by adding an extra column to the right, containing the transformation values u and v.
- To keep the matrix square, we add an extra row that always contains 0, 0, 1:

$$\begin{pmatrix} a & b & u \\ c & d & v \\ 0 & 0 & 1 \end{pmatrix}$$

# 2D Translation Transformation

## Module III: 2D Translation

- We also need to extend our position vector to be 3 elements long.
- We do this by adding an extra row element that is always equal to 1.

$$\begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

- For a translation formula, we set the a, b, c, d values to a unit matrix, and set the u, v elements to contain the translation values.

$$\begin{pmatrix} a & b & u \\ c & d & v \\ 0 & 0 & 1 \end{pmatrix}$$

# 2D Translation Transformation

## Module III: 2D Translation

- When we apply this matrix to a position vector, we do indeed get a translation:

transformation matrix  position vector  translated position

$$\begin{pmatrix} 1 & 0 & u \\ 0 & 1 & v \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} x+u \\ y+v \\ 1 \end{pmatrix}$$

- The third element of the transformed vector will always be 1 and is discarded after the calculation to leave us with a 2-vector result.

$$\begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} ax+by+c \\ dx+ey+f \\ 1 \end{bmatrix}$$

# 2D Translation

## Module III: 2D Transformations

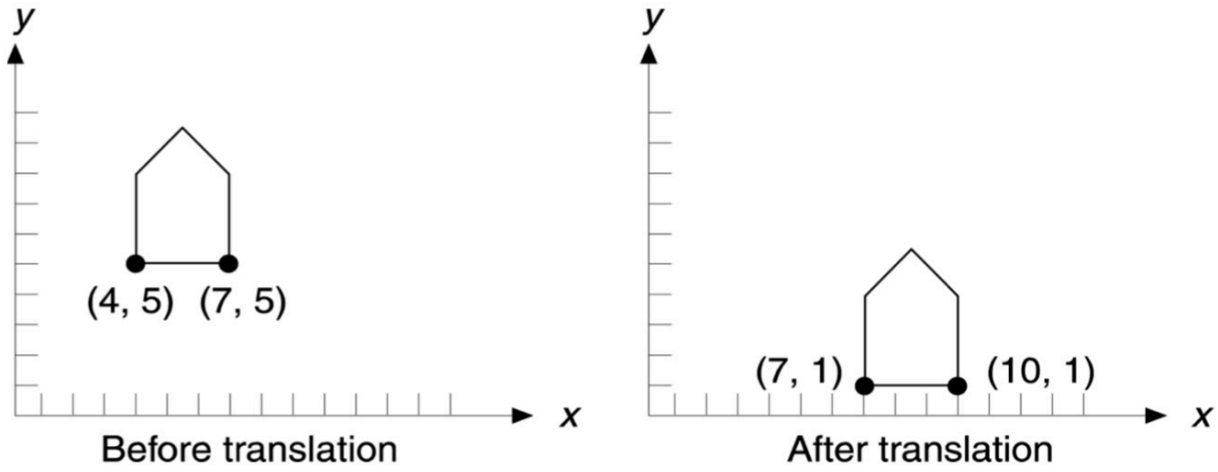
- A translation transformation changes or performs a rigid motion of points to new locations.
- That means we calculate:

$$x' = x + d_x$$

$$y' = y + d_y$$

- Defined with column vectors:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} d_x \\ d_y \end{bmatrix}$$



# 2D Scaling

## Module III: 2D Transformations

- A scaling transformation alters the size of an object.
- In the scaling process, we either compress or expand the dimension of the object.
- That means we calculate:

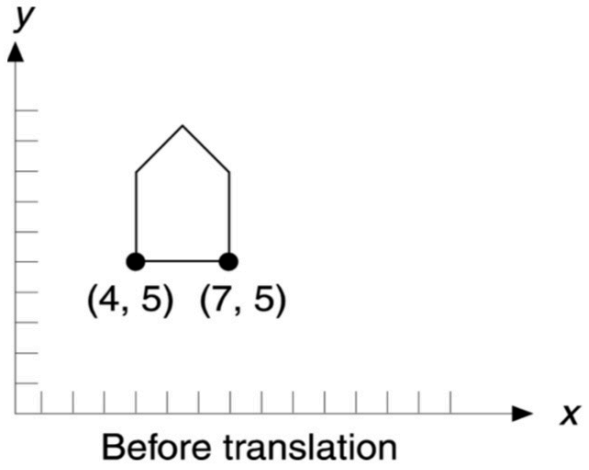
$$x' = x \cdot s_x$$

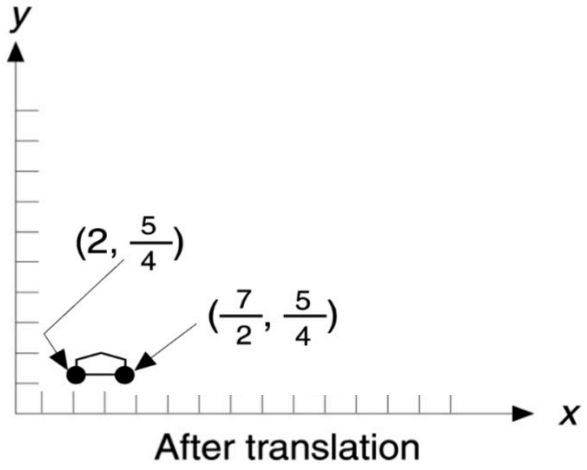
$$y' = y \cdot s_y$$

- In matrix form:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} -s_x & 0 \\ 0 & -s_y \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$

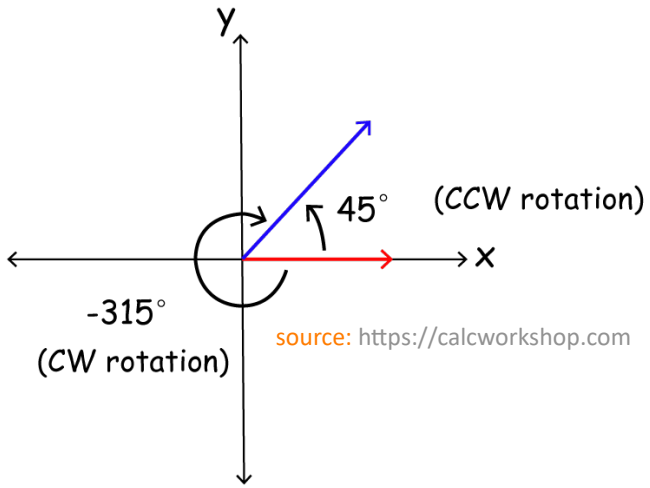
$$\begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix}$$



# 2D Rotation

## Module III: 2D Transformations



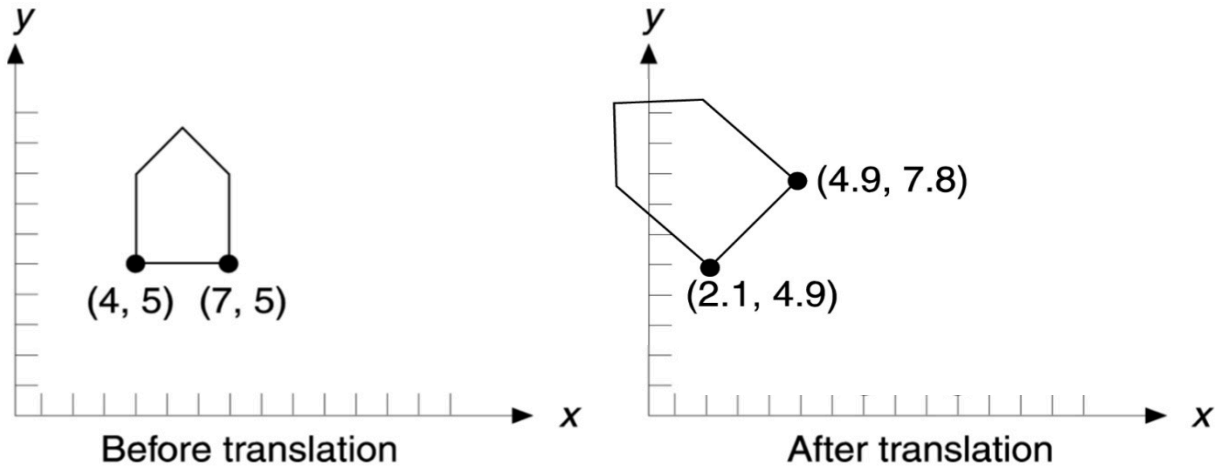

source: https://calcworkshop.com

- A rotation of points is done about the origin.
- There are two directions of rotation:
  - positive angle: CCW
  - negative angle: CW
- That means we calculate:

$$x' = x \cdot cos\,\theta - y \cdot sin\,\theta$$

$$y' = x \cdot sin\,\theta + y \cdot cos\,\theta$$

- In matrix form:




$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} cos\ \theta & -sin\ \theta \\ sin\ \theta & cos\ \theta \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$

$$\begin{bmatrix} -sin\ \theta & cos\ \theta \\ cos\ \theta & sin\ \theta \end{bmatrix}$$

# 2D Affine Matrix Representation

## Module III: 2D Transformations

- Translation:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & d_x \\ 0 & 1 & d_y \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- Scaling:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- Rotation:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} cos\ \theta & -sin\ \theta & 0 \\ sin\ \theta & cos\ \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} ax + by \\ cx + dy \end{bmatrix}$$

$$\begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} ax + by + c \\ dx + ey + f \\ 1 \end{bmatrix}$$

# Composition of 2D Transforms

## Module III: 2D Transformations

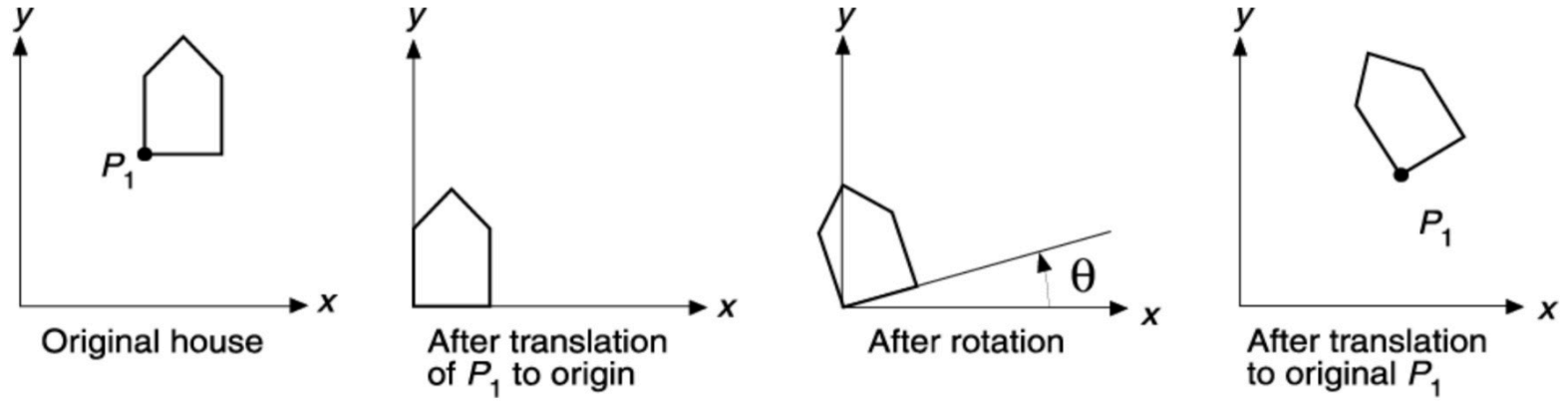
- Rotate about a point $P_1$ steps:
  - Translate $P_1$ to the origin

$$\begin{bmatrix} 1 & 0 & -x \\ 0 & 1 & -y \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

  - Rotate
  - Translate back to $P_1$

$$T(x_1, y_1) \cdot R(\theta) \cdot T(-x_1, -y_1)$$

$$\text{RM} = \begin{bmatrix} 1 & 0 & x_1 \\ 0 & 1 & y_1 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} cos\ \theta & -sin\ \theta & 0 \\ sin\ \theta & cos\ \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & -x_1 \\ 0 & 1 & -y_1 \\ 0 & 0 & 1 \end{bmatrix}$$



# Composition of 2D Transforms

## Module III: 2D Transformations

- Rotate about a point $P_1$ steps:
  - Translate $P_1$ to the origin
  - Rotate
  - Translate back to $P_1$

$$T(x_1, y_1) \cdot R(\theta) \cdot T(-x_1, -y_1)$$

$$\text{RM} = \begin{bmatrix} cos\ \theta & -sin\ \theta & x_1(1 - cos\ \theta) + y_1 sin\ \theta \\ sin\ \theta & cos\ \theta & y_1(1 - cos\ \theta) - x_1 sin\ \theta \\ 0 & 0 & 1 \end{bmatrix}$$

- Then, we got the Rotation Matrix (RM) as above.

# Composition of 2D Transforms

## Module III: 2D Transformations

- Scale about a point $P_1$ steps:
  - Translate $P_1$ to the origin

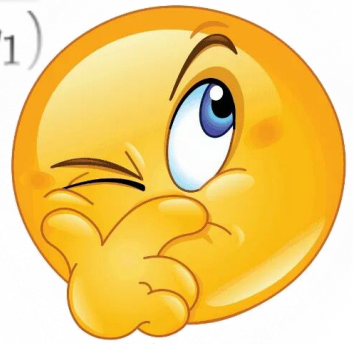
$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

  - Scale
  - Translate back to P1

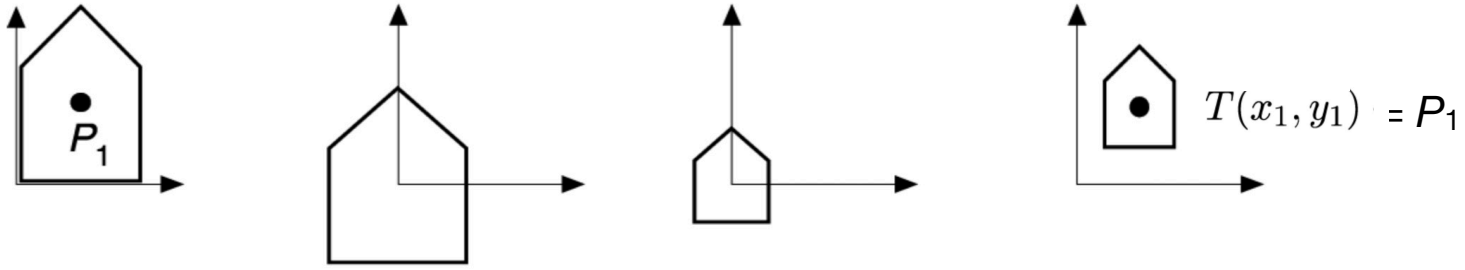
$$T(x_1, y_1) \cdot S(S_x, S_y) \cdot T(-x_1, -y_1)$$

SM = ?



- Hence, what is SM (Scaling Matrix)?




Original house | Translate $P_1$ to origin | Scale | Translate to final position $P_1$

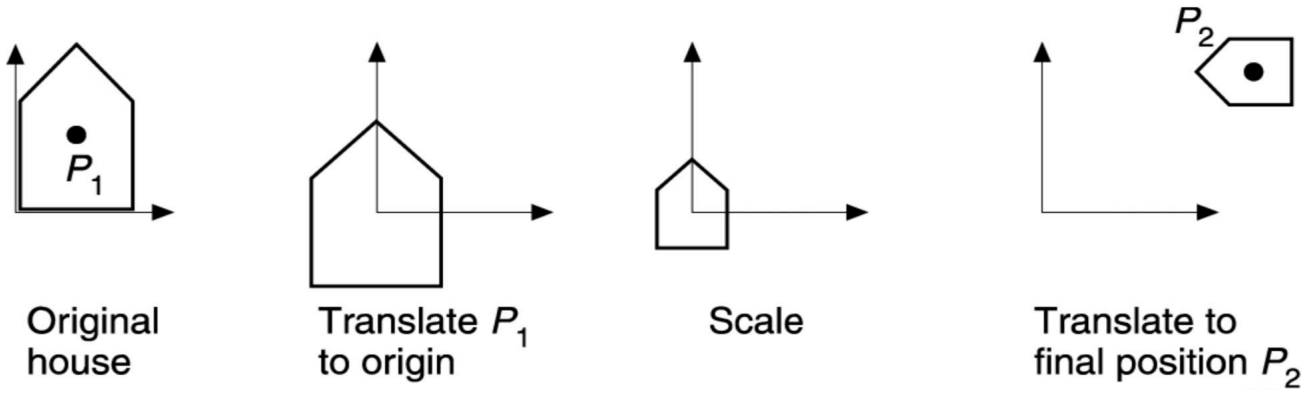
# Composition of 2D Transforms

## Module III: 2D Transformations

- To perform multiple transformations in the proper order.

$$P' = (T \cdot (R \cdot (S \cdot (T \cdot P))))$$

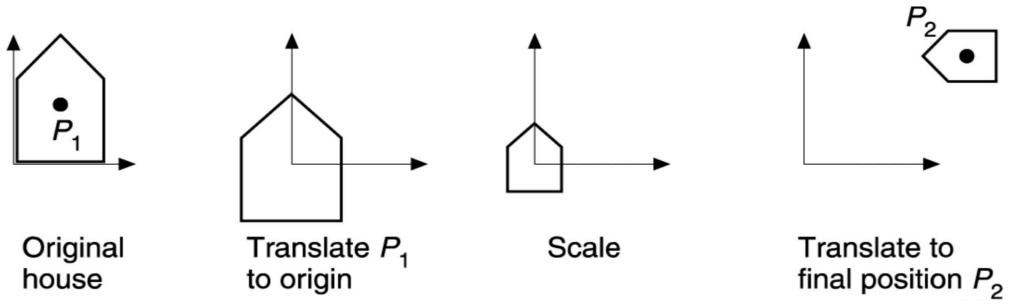
$$P' = ((T \cdot (R \cdot (S \cdot T))) \cdot P)$$



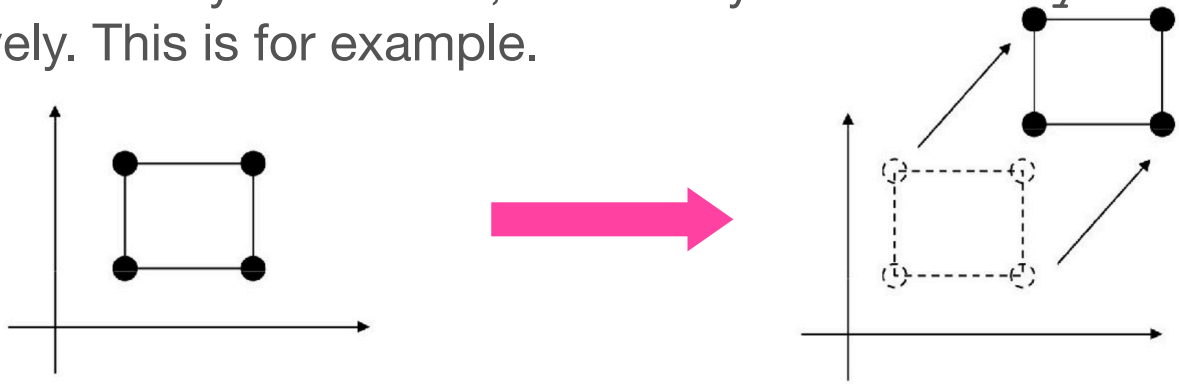
# Composition of 2D Transforms

## Module III: 2D Transformations

- For the next assignment, your transformation matrix needs to combine all the Affine transformation matrices.




- When you want to do only translation, it means your `dx` and `dy` MUST NOT be `0` and `0`, respectively. This is for example.



# 2D Translation

## Module III: 2D Translation

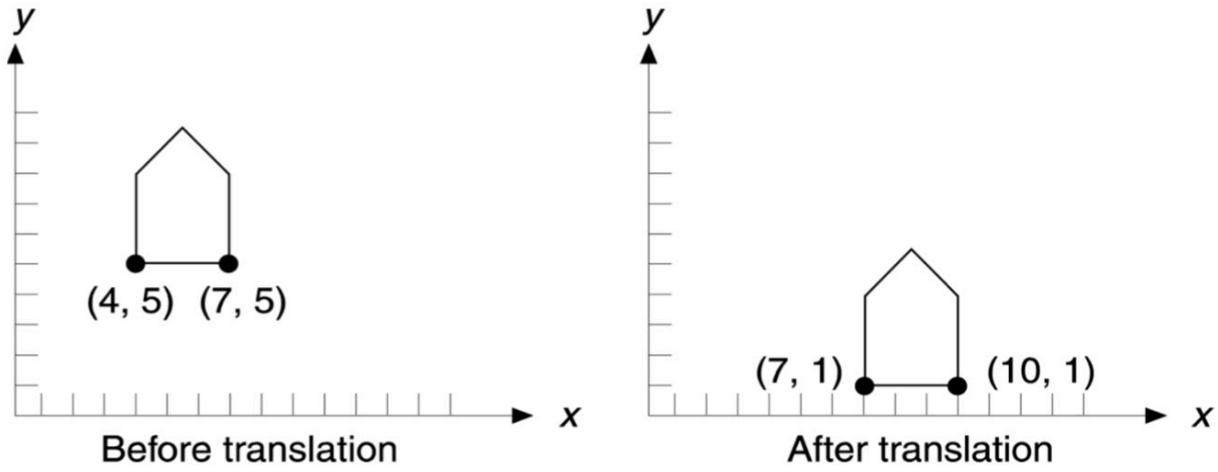
- A translation transformation changes or performs a rigid motion of points to new locations.
- That means we calculate:

$$x' = x + d_x$$

$$y' = y + d_y$$

- Defined with column vectors:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} d_x \\ d_y \end{bmatrix}$$



Introduction
to

# 2D Scaling



# 2D Scaling

## Module III: 2D Scaling

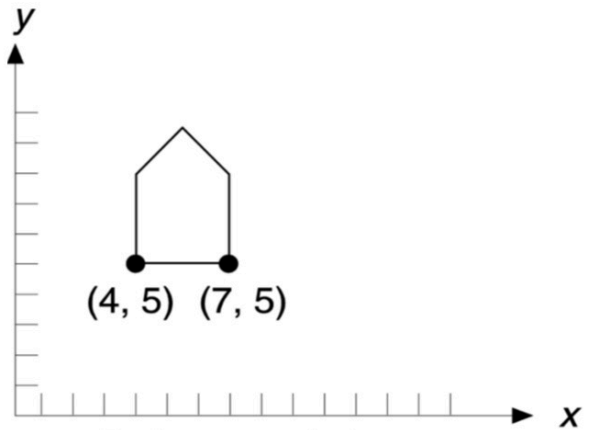
- A scaling transformation alters the size of an object.
- In the scaling process, we either compress or expand the dimension of the object.
- That means we calculate:

$$x' = x \cdot s_x$$

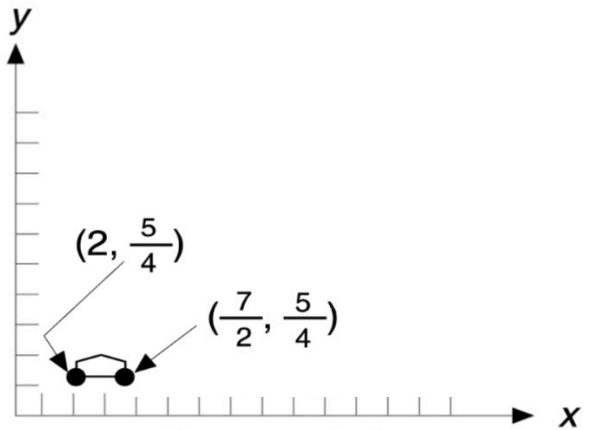
$$y' = y \cdot s_y$$

- In matrix form:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$




Before translation & scaling

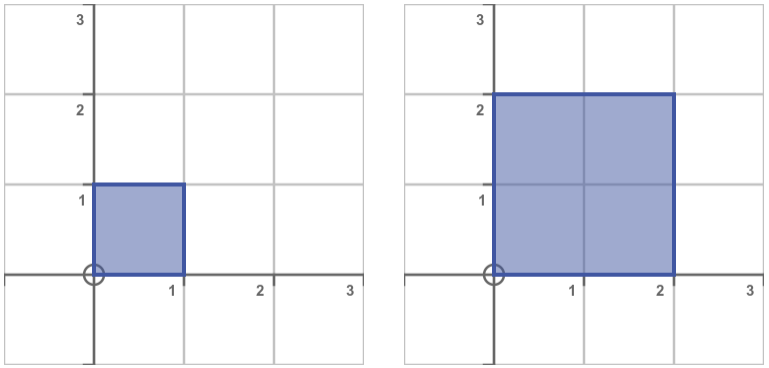



After translation & scaling

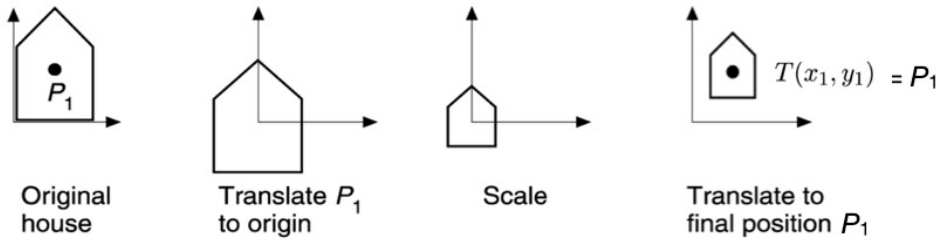
# 2D Scaling transformations

## Module III: 2D Scaling

- There are two approaches to complete the 2D scaling.
- Please take a look at these two approaches:



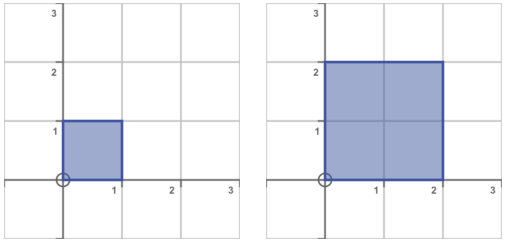

I. 2D scaling around the origin




II. 2D scaling around an arbitrary point (P1)

# I. 2D Scaling at the Origin

## Module III: 2D Scaling



- The useful thing about this is that it works for the other transformations too.
- This time we set u and v to 0 because there is no translation:

$$\begin{pmatrix} s & 0 & 0 \\ 0 & 1/s & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} sx \\ y/s \\ 1 \end{pmatrix}$$

- This form works for a scaling transformation.

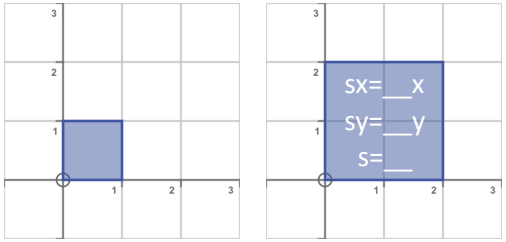


1 minutes



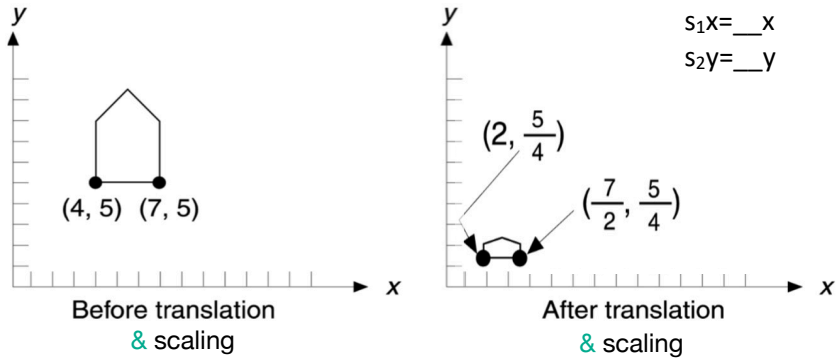
# I. 2D Scaling at the Origin

## Module III: 2D Scaling




- A scaling transformation uses the following matrix:

$$\begin{pmatrix} s & 0 \\ 0 & s \end{pmatrix}$$



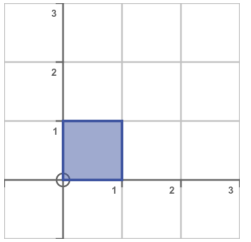

- Here is the effect it has on a vector:

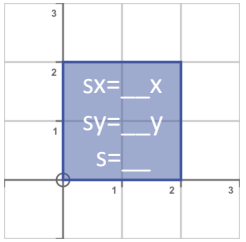
$$\begin{pmatrix} s & 0 \\ 0 & s \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} sx \\ sy \end{pmatrix}$$



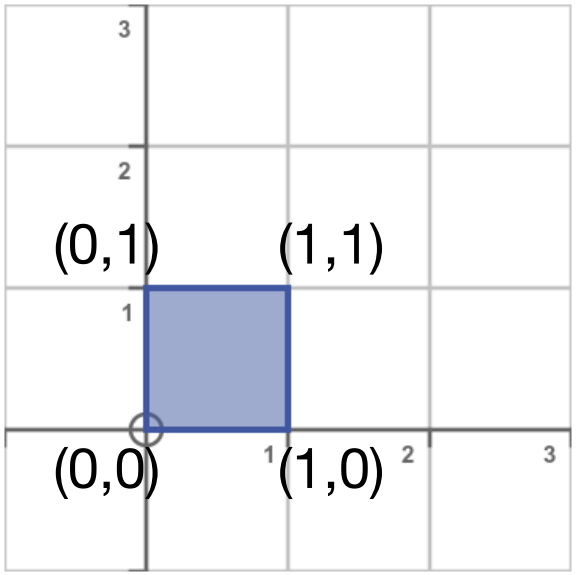
# I. 2D Scaling at the Origin

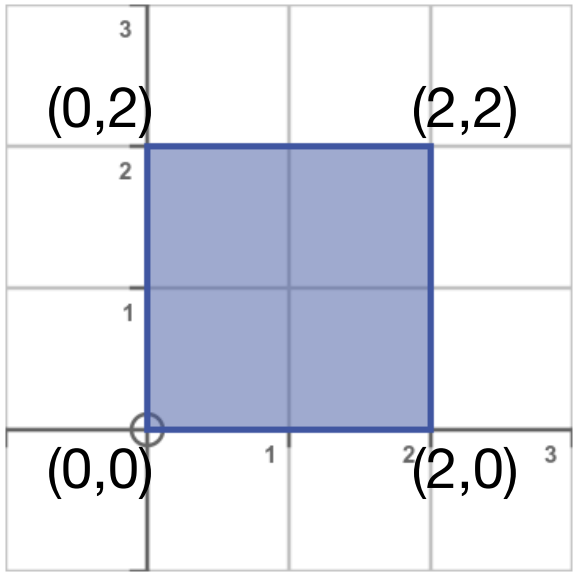
## Module III: 2D Scaling






- If we take a unit square and apply this transform to each of its vertices, we get the new shape on the right called "uniform scaling".
- We have used an s value of 2, which scales the shape by a factor of 2:

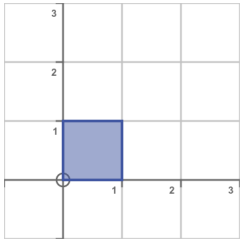


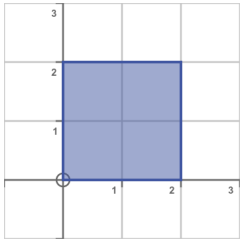




S=2

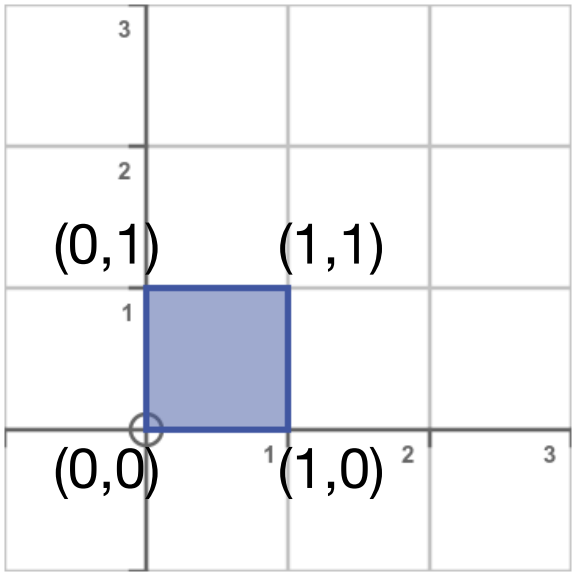
# I. 2D Scaling at the Origin

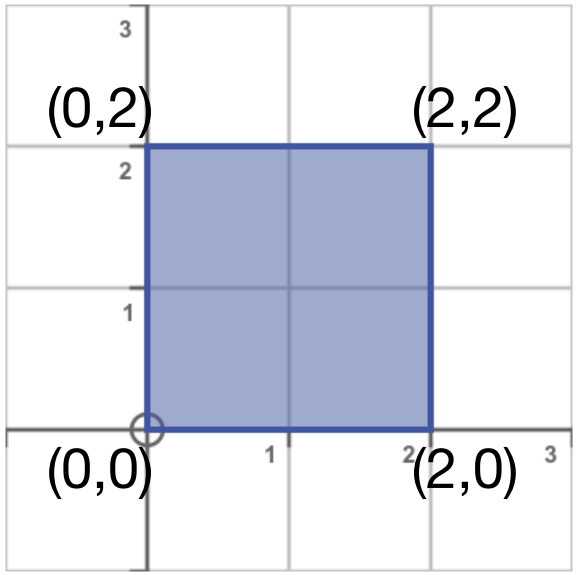
## Module III: 2D Scaling





- This transform scales everything out from the origin; in other words, it has the centre of enlargement of (0, 0).

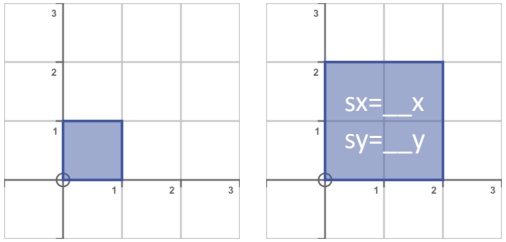



S=2

# I. 2D Scaling at the Origin

## Module III: 2D Scaling

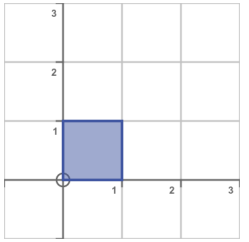


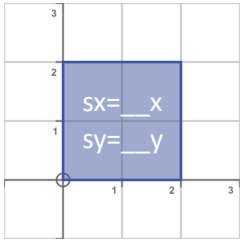

- A stretch transform scales the shape in one dimension (nonuniform scaling) only.
- Here is the matrix transformation:

$$\begin{pmatrix} s & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} sx \\ y \end{pmatrix}$$

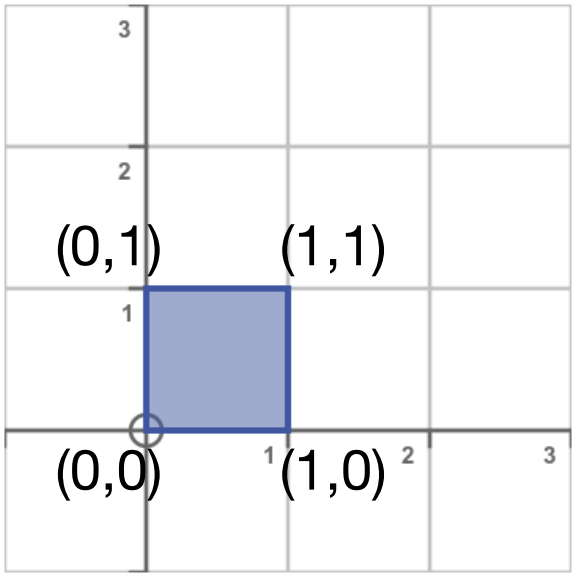
# I. 2D Scaling at the Origin

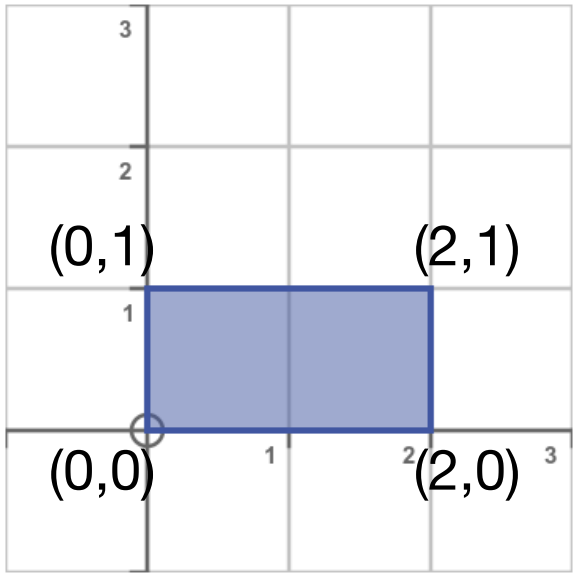
## Module III: 2D Scaling






- This stretches the square in the x-direction: S=2



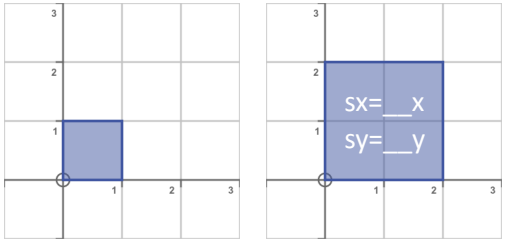




S=2

- A stretch in the y-direction could be achieved by swapping the 1 and s.

# I. 2D Scaling at the Origin

## Module III: 2D Scaling

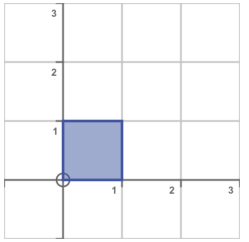


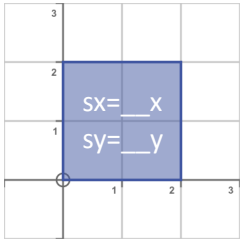

- Finally, a squeeze transform scales the shape by *s* in one dimension only, and by *1/s* in the other dimension (nonuniform scaling).
- Here is the squeeze matrix:

$$\begin{pmatrix} s & 0 \\ 0 & 1/s \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} sx \\ y/s \end{pmatrix}$$

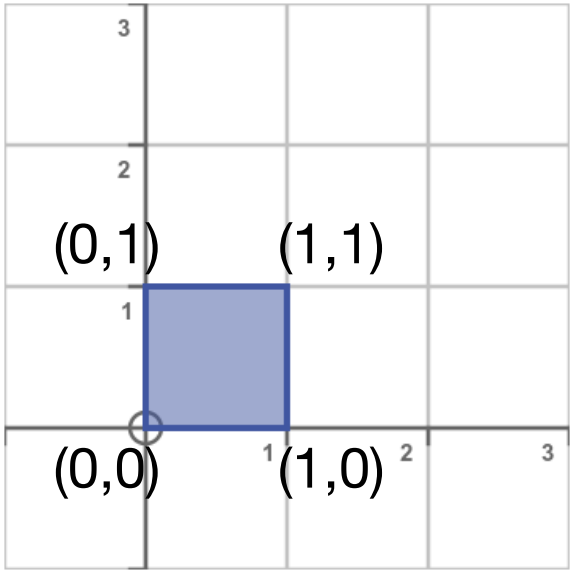
# I. 2D Scaling at the Origin

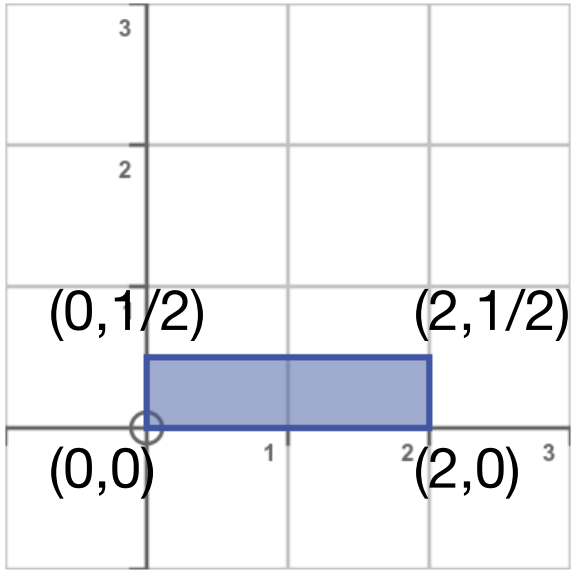
## Module III: 2D Scaling






- Here is the effect: S=2



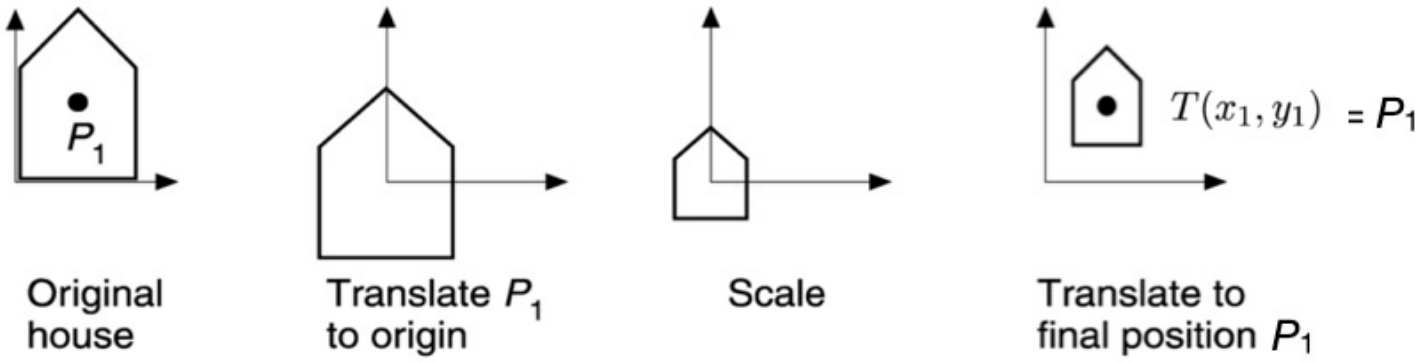

S=2

- A squeeze transformation preserves area. When a squeeze matrix is applied, the width is multiplied by s and the height by 1/s, so the area of the shape is unchanged.

# II. 2D Scaling at an Arbitrary Point (P1)

## Module III: 2D Scaling

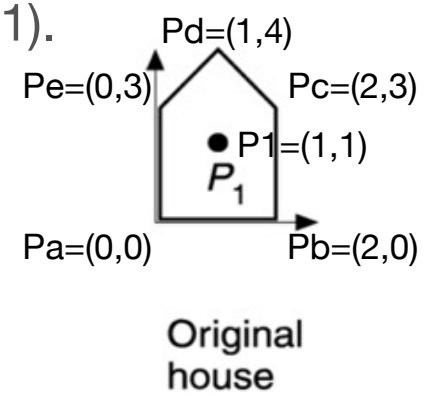
- Scaling an object about an arbitrary point P1(x1, y1) involves a sequence of transformations to achieve the desired effect, as illustrated in this image.

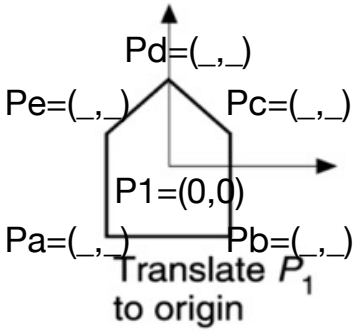


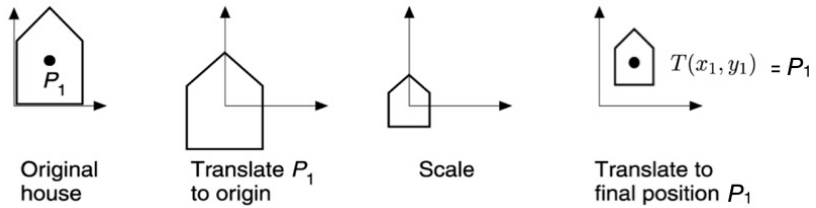
# II. 2D Scaling at an Arbitrary Point (P1)

## Module III: 2D Scaling

- This process ensures that P1 remains fixed while the rest of the object scales relative to it.
- Here's how it's done:
  - Translate P1 to the Origin:
    - First, the object is translated so that the arbitrary point P1 coincides with the origin (0, 0).
    - If P1 is at (x1, y1), this translation is achieved by moving the object by (-x1, -y1).



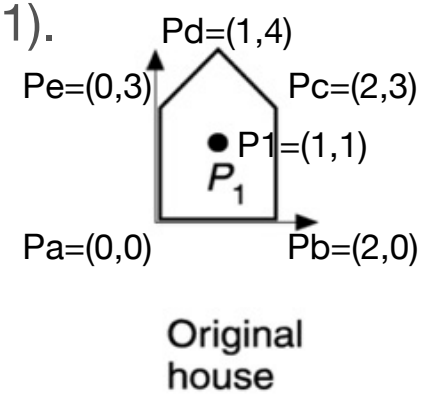



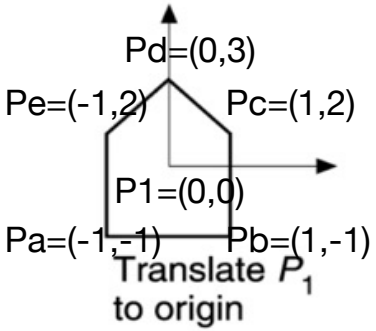



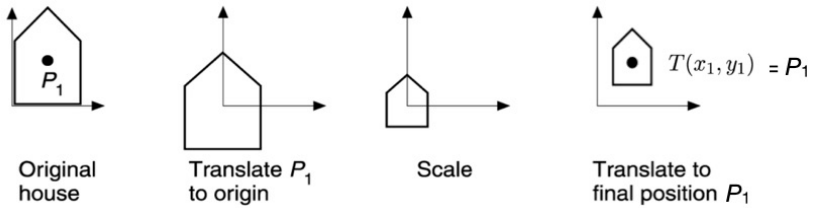
# II. 2D Scaling at an Arbitrary Point (P1)

## Module III: 2D Scaling

- This process ensures that P1 remains fixed while the rest of the object scales relative to it.
- Here's how it's done:
  - Translate P1 to the Origin:
    - First, the object is translated so that the arbitrary point P1 coincides with the origin (0, 0).
    - If P1 is at (x1, y1), this translation is achieved by moving the object by (-x1, -y1).



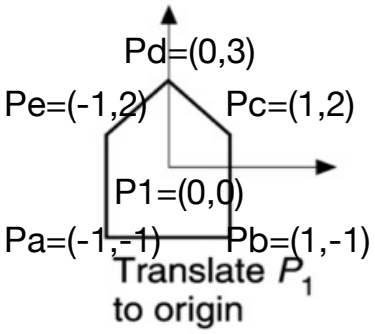



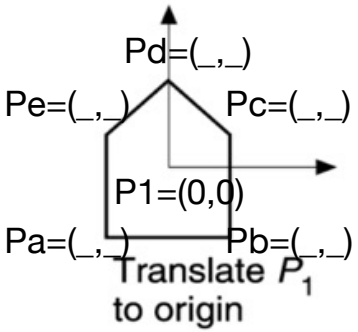



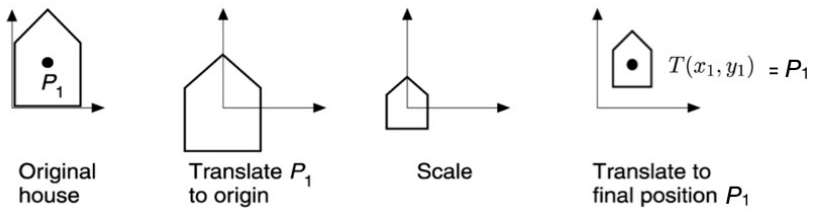
# II. 2D Scaling at an Arbitrary Point (P1)

## Module III: 2D Scaling

- This process ensures that P1 remains fixed while the rest of the object scales relative to it.
- Here's how it's done:
  - Scale the Object:
    - With P1 effectively at the origin, the standard scaling transformation is applied.
    - Each point (x, y) of the object is scaled by factors (sx) and (sy) to become (x.sx, y.sy), "dot product". S=2



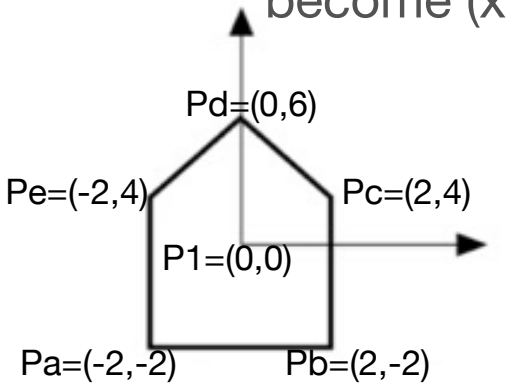



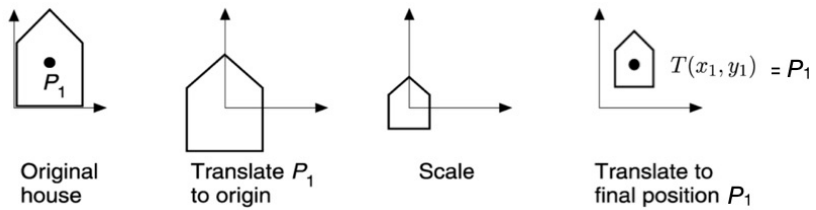



# II. 2D Scaling at an Arbitrary Point (P1)

## Module III: 2D Scaling

- This process ensures that P1 remains fixed while the rest of the object scales relative to it.
- Here's how it's done:
  - Scale the Object:
    - With P1 effectively at the origin, the standard scaling transformation is applied.
    - Each point (x, y) of the object is scaled by factors (sx) and (sy) to become (x.sx, y.sy), "dot product". S=2
      uniform sx==sy
      nonuniform sx<>sy



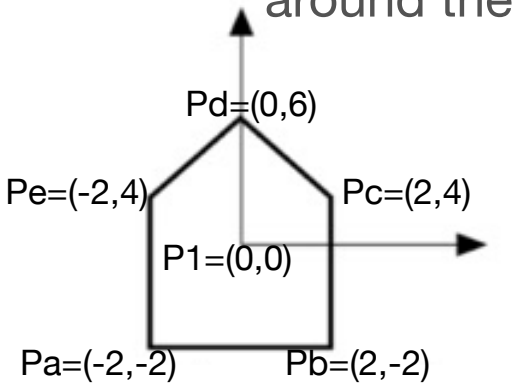



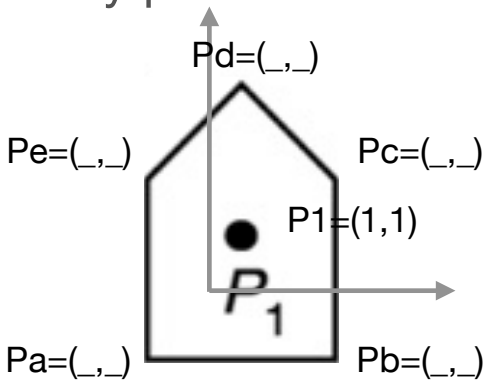
# II. 2D Scaling at an Arbitrary Point (P1)

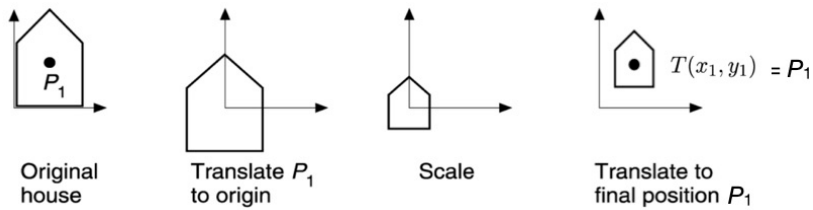
## Module III: 2D Scaling

- This process ensures that P1 remains fixed while the rest of the object scales relative to it.
- Here's how it's done:
  - Translate Back to Original Position:
    - Finally, the object is translated back to its original position by moving it by (x1, y1).
    - This repositions (P1) to its initial coordinates, completing the scaling around the arbitrary point.

S=2



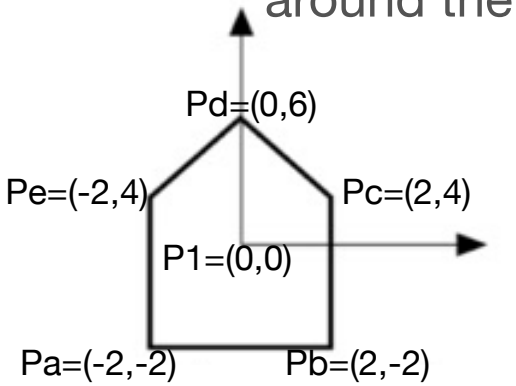



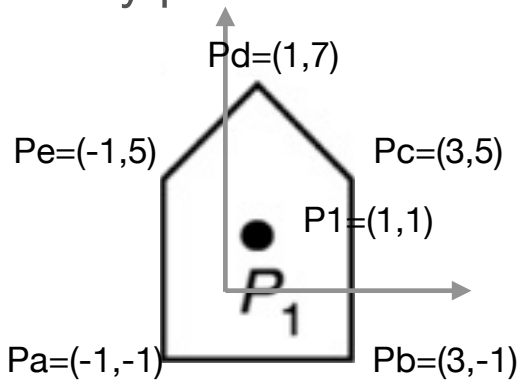
# II. 2D Scaling at an Arbitrary Point (P1)

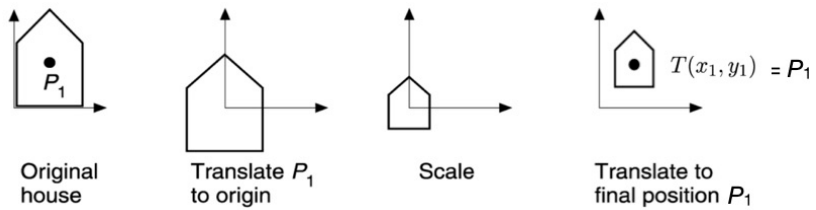
## Module III: 2D Scaling

- This process ensures that P1 remains fixed while the rest of the object scales relative to it.
- Here's how it's done:
  - Translate Back to Original Position:
    - Finally, the object is translated back to its original position by moving it by (x1, y1).
    - This repositions (P1) to its initial coordinates, completing the scaling around the arbitrary point.

S=2



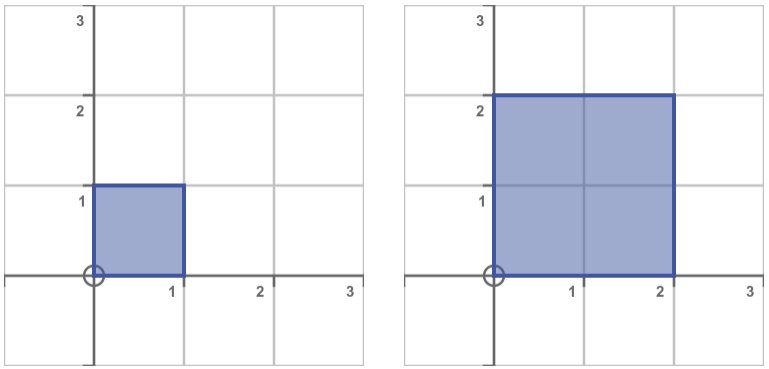



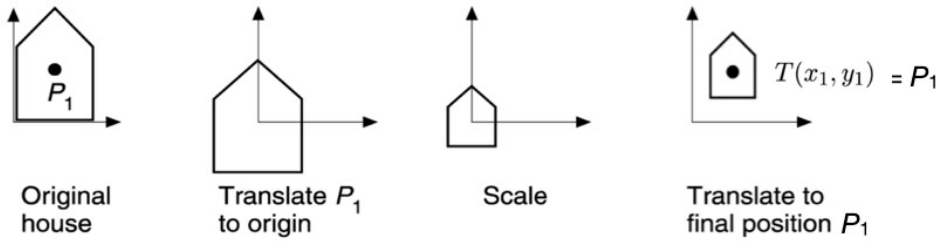
# II. 2D Scaling at an Arbitrary Point (P1)

## Module III: 2D Scaling

- In essence, scaling about an arbitrary point leverages the simpler scaling-about-the-origin operation by temporarily relocating the point of interest to the origin, performing the scaling, and then returning it to its original place.




I. 2D scaling around the origin



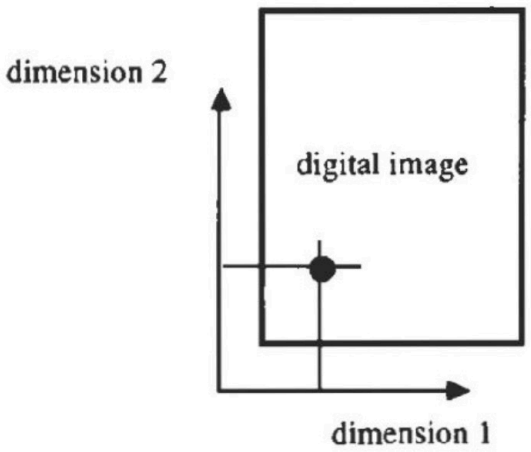

II. 2D scaling around an arbitrary point (P1)

# Animation/Video
# Basic Concept

# Animation/Video Basic Concept

## Module III: Animation/Video

- Definition:

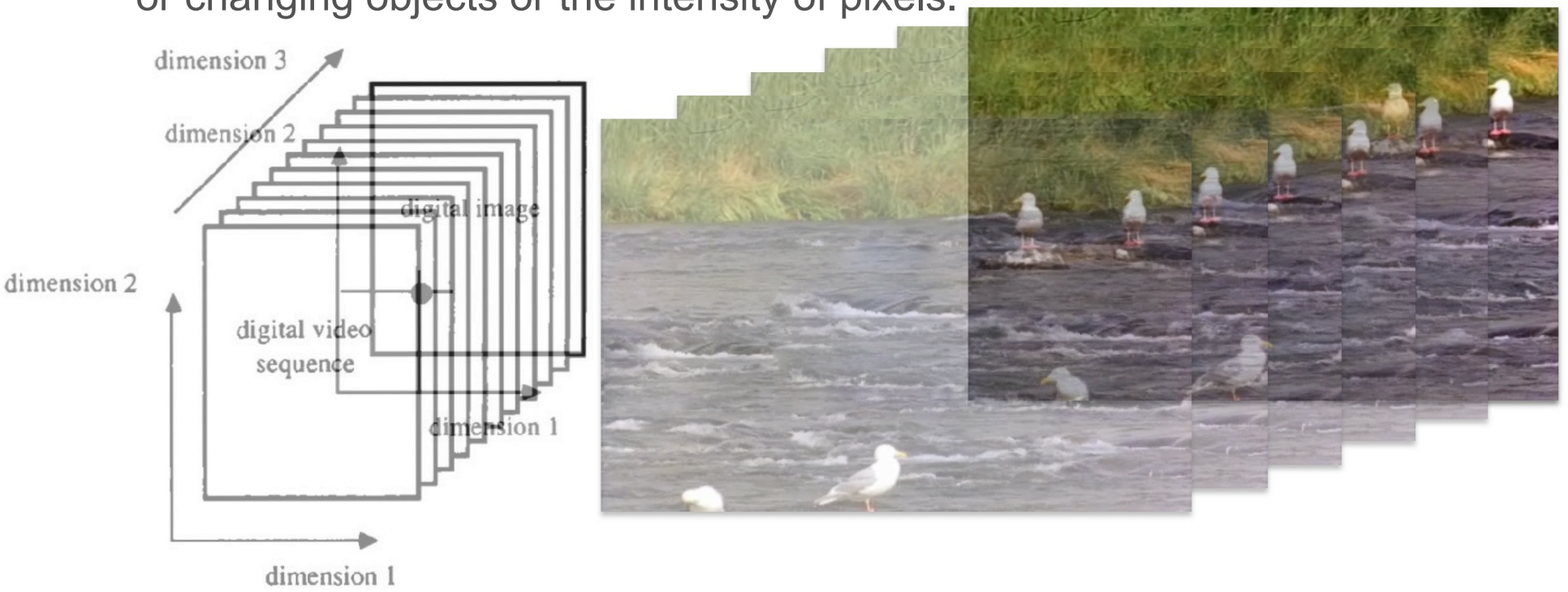





# Animation/Video Basic Concept

## Module III: Animation/Video

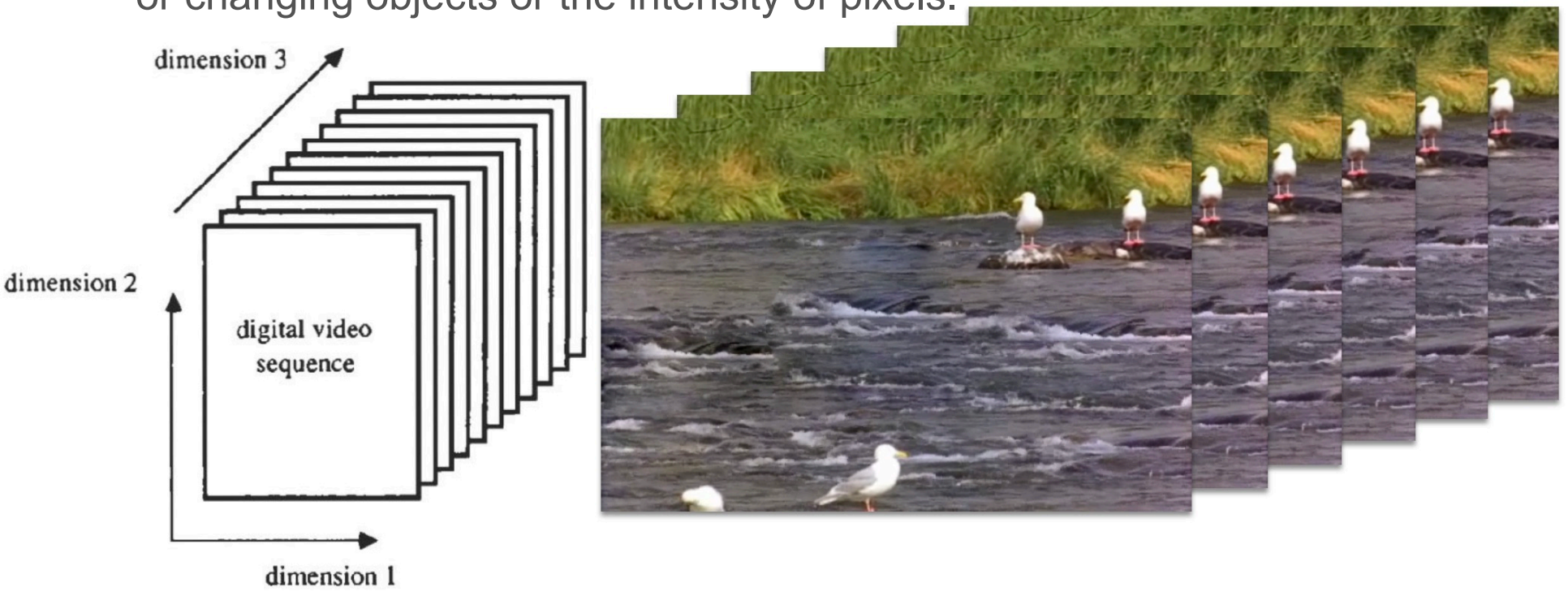
- Definition: Animation/Video evolves with time and generally contains moving and/or changing objects or the intensity of pixels.



# Animation/Video Basic Concept

## Module III: Animation/Video

- Definition: Animation/Video evolves with time and generally contains moving and/or changing objects or the intensity of pixels.



# 2D Animation Techniques

## Module III: Animation/Video

- Two 2D animation techniques will be discussed:
  1. Traditional animation or frame-by-frame technique:
     - The animation was done by hand.
       - All the frames in the animation are to be drawn by hand.
     - Each second of animation requires 24 frames/images (films).



# 2D Animation Techniques

## Module III: Animation/Video

- Two 2D animation techniques will be discussed:
  2. Key framing:
     - The **computer** automatically **fills in the missing frames** by smoothly interpolating between the key points.

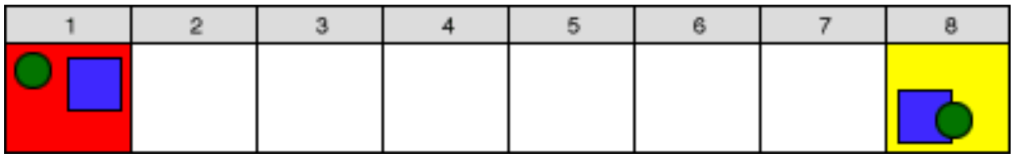



**Figure: Frames drawn by user**

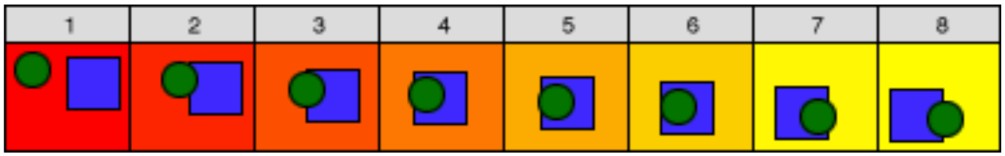



**Figure: In-between frames generated by computer**

source: "Computer Graphics," www.tutorialpoint.com, Tutorial Point (I) Pvt. Ltd.

# Motion Images Display
## on Colab
### Step-by-Step

# Preparing and opening video files

## Module III: Animation/Video

- Import Library:
  - cv2: OpenCV for image and video processing.
  - numpy: for calculations on arrays

```
import cv2
import numpy as np
from google.colab import drive
from google.colab.patches import cv2_imshow
from IPython.display import display, HTML
import ipywidgets as widgets
from IPython.display import clear_output
import time
```

  - google.colab: for Google Colab
  - ipywidgets: to create Widgets in Jupyter Notebook
  - time: timing management

# Open Video

## Module III: Animation/Video

- Mount Google Drive: mount the video path in Google Drive

```
# Step 1: Mount Google Drive to access the video file
drive.mount('/content/drive')
```

- Define video path: define the video path in Google Drive

```
# Step 2: Define the path to your video in Google Drive
video_path = '/content/drive/My Drive/MP4used/11_001.mp4'
```

- Open the video file: create a VideoCapture object for video reading

```
# Step 3: Open the video using OpenCV
cap = cv2.VideoCapture(video_path)
```

# Video Availability Verification

## Module III: Animation/Video

- Video opening verification: an error message occurs if a problem

```
# Check if the video was opened correctly
if not cap.isOpened():
    print("Error: Could not open video.")
else:
    print("Video opened successfully.")
```

- Preparing for displaying read-only images

```
# Step 4: Prepare for frame processing and widget display
frame_count = 0
ret, prev_frame = cap.read()
```

  - `frame_count` to count the number of frames.
  - `Read()` reads the first frame from the video.

# Read the First Frame

## Module III: Animation/Video

- 1st Frame Availability Check: an error message if a problem

```
if not ret:
    print("Error: Could not read the first frame.")
else:
    print("Processing frames for animation...")
```

- Create Image Widget:

```
# Create an ipywidgets Image widget for displaying frames
initial_image = cv2.imencode('.png', prev_frame)[1].tobytes()
image_widget = widgets.Image(
    value=initial_image,
    format='png',
    width=prev_frame.shape[1],
    height=prev_frame.shape[0],
)
```

- Convert the first frame to PNG format.
- Create a widget to display the image.
- Set the widget size to match the video frame size

# Video Displaying

## Module III: Animation/Video

- Displaying Widget: shows Image Widget in Notebook

```
# Display the image widget
display(image_widget)
```

# Frames Display

## Module III: Animation/Video

- Check for the last frame (user-defined value)

```
while frame_count < 100:
    ret, frame = cap.read()
    if not ret:
        print(f"End of video or error after {frame_count} frames.")
        break

    # Convert the current frame to bytes and update the widget
    _, buffer = cv2.imencode('.png', frame)
    image_widget.value = buffer.tobytes()

    # Optional: Add a small delay to control animation speed
    # time.sleep(0.04) # Uncomment and adjust for desired speed

    frame_count += 1
```

  - reads one frame at a time from the video (up to 100 frames).
  - stops the loop if the frame reading fails or the video ends.

# Frames Display

## Module III: Animation/Video

- Check for the last frame (user-defined value)

```
while frame_count < 100:
    ret, frame = cap.read()
    if not ret:
        print(f"End of video or error after {frame_count} frames.")
        break

    # Convert the current frame to bytes and update the widget
    _, buffer = cv2.imencode('.png', frame)
    image_widget.value = buffer.tobytes()

    # Optional: Add a small delay to control animation speed
    # time.sleep(0.04) # Uncomment and adjust for desired speed

    frame_count += 1
```

  - converts the current frame to PNG format.
  - updates the image in the widget with the new frame (from buffer).

# Frames Display

## Module III: Animation/Video

- Check for the last frame (user-defined value)

```
while frame_count < 100:
    ret, frame = cap.read()
    if not ret:
        print(f"End of video or error after {frame_count} frames.")
        break

    # Convert the current frame to bytes and update the widget
    _, buffer = cv2.imencode('.png', frame)
    image_widget.value = buffer.tobytes()

    # Optional: Add a small delay to control animation speed
    # time.sleep(0.04) # Uncomment and adjust for desired speed

    frame_count += 1
```

  - Speed control (optional): You can adjust the animation speed by uncommenting this line.

# Frames Display

## Module III: Animation/Video

- Check for the last frame (user-defined value)

```
while frame_count < 100:
    ret, frame = cap.read()
    if not ret:
        print(f"End of video or error after {frame_count} frames.")
        break

    # Convert the current frame to bytes and update the widget
    _, buffer = cv2.imencode('.png', frame)
    image_widget.value = buffer.tobytes()

    # Optional: Add a small delay to control animation speed
    # time.sleep(0.04) # Uncomment and adjust for desired speed

    frame_count += 1
```

  - frame counter value increments

# Ending the Display

## Module III: Animation/Video

- Close connection: Closes the VideoCapture object with a message, informing that processing is complete.

```
# Step 5: Release resources
cap.release()
print("Processing complete.")
```



'/content/drive/My Drive/MP4used/11_001.mp4'

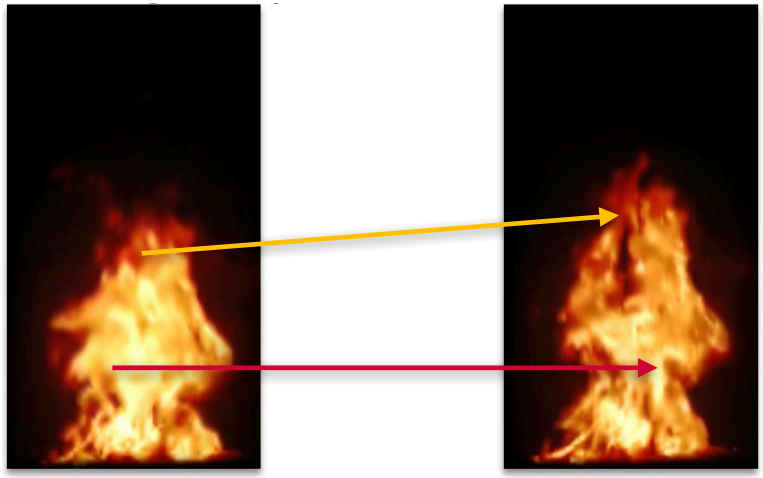
# Optical Flow

## Motion Vector Field with Optical Flow

# Optical Flow

## Module III: Motion Vector Field

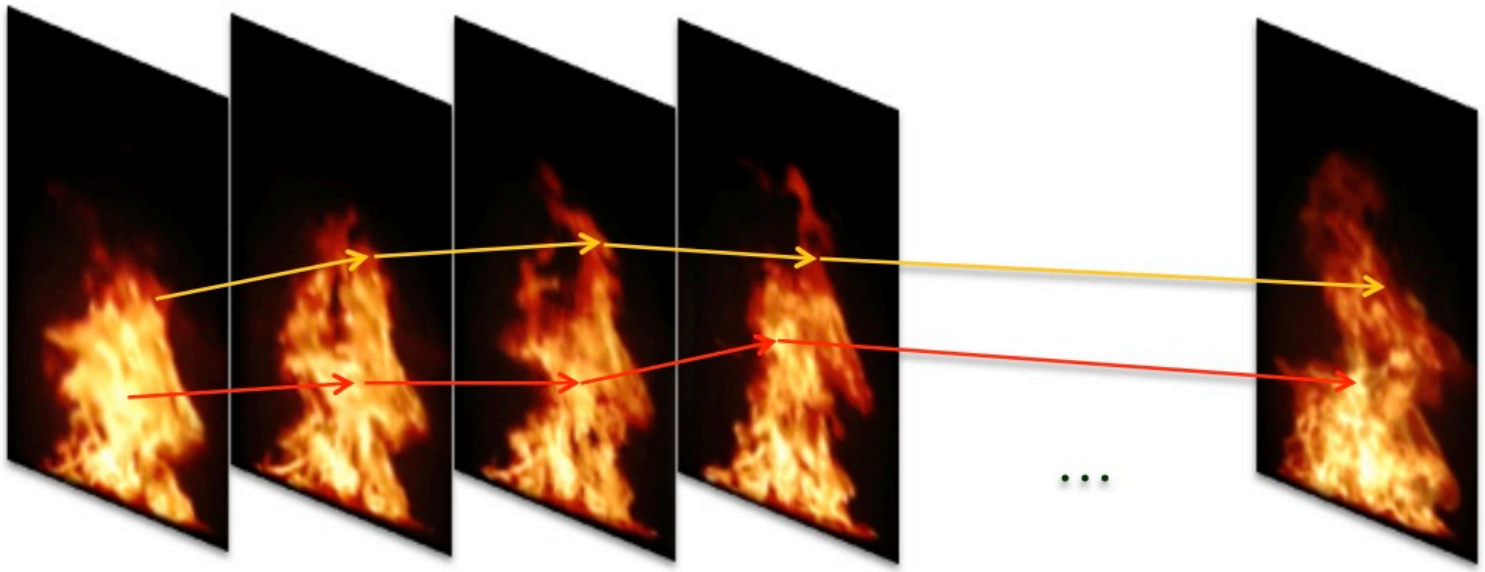
- Optical flow is a motion estimation calculated from local derivatives in an image sequence, or where pixel intensity changes occur in adjacent images (image 1 and image 2, or frame 1 and frame 2).
- In 2D, the intensity of pixels in adjacent images (image 1 and image 2) changes.



# Optical Flow

## Module III: Motion Vector Field

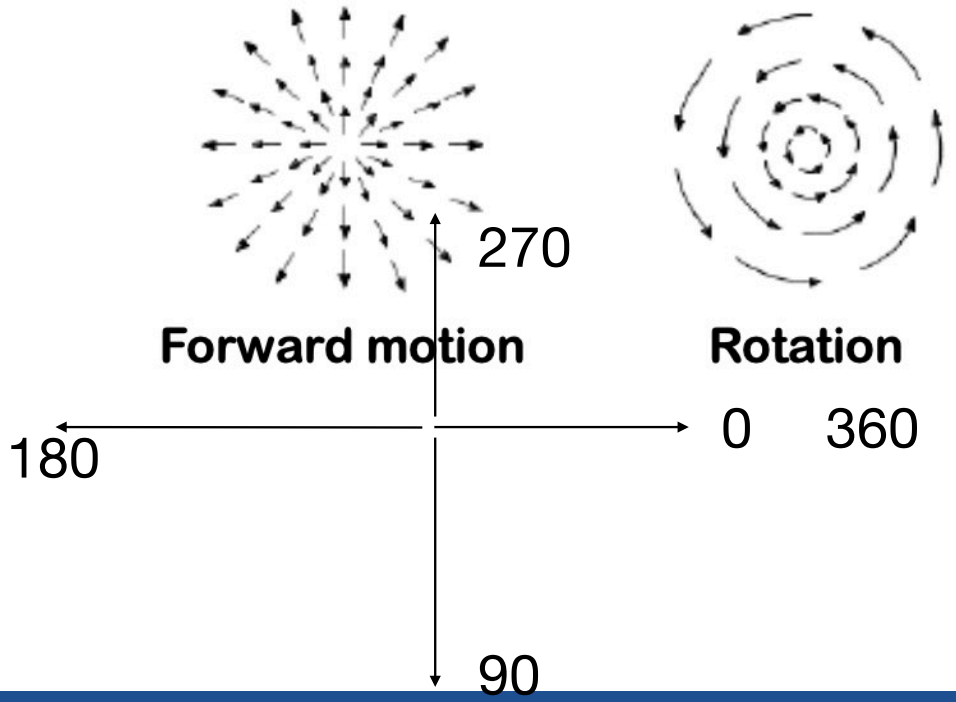
- In 3D (video/animation), the intensity of pixels in adjacent images (image 1 and image 2, image 2 and image 3, image 3 and image 4, …) changes.



# Examples of Optical Flow

## Module III: Motion Vector Field

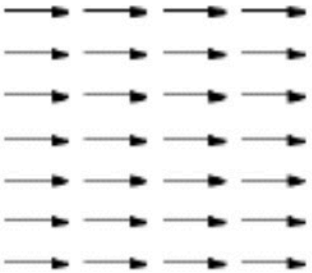
- Motion patterns



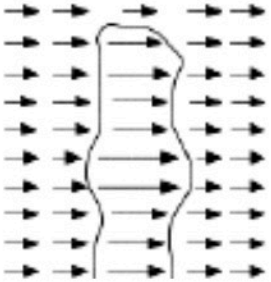

Forward motion



Rotation



Horizontal Translation



Faster moving

# Farneback Optical Flow

## Motion Vector Field

### Step-by-Step

# Preparing and opening video files

## Module III: Farneback Optical Flow

- Import Library:
  - cv2: OpenCV for image and video processing.
  - numpy: for calculations on arrays

```
import cv2
import numpy as np
from google.colab import drive
from google.colab.patches import cv2_imshow
from IPython.display import display, HTML
import ipywidgets as widgets
from IPython.display import clear_output
import time
```

  - google.colab: for Google Colab
  - ipywidgets: to create Widgets in Jupyter Notebook
  - time: timing management

# Open Video

## Module III: Farneback Optical Flow

- Mount Google Drive: mount the video path in Google Drive

```
# Step 1: Mount Google Drive to access the video file
drive.mount('/content/drive')
```

- Define video path: define the video path in Google Drive

```
# Step 2: Define the path to your video in Google Drive
video_path = '/content/drive/My Drive/MP4used/11_001.mp4'
```

- Open the video file: create a VideoCapture object for video reading

```
# Step 3: Open the video using OpenCV
cap = cv2.VideoCapture(video_path)
```

# Video Availability Verification

## Module III: Farneback Optical Flow

- Video opening verification: an error message occurs if a problem

```
# Check if the video was opened correctly
if not cap.isOpened():
    print("Error: Could not open video.")
else:
    print("Video opened successfully.")
```

- Preparing for displaying read-only images

```
# Step 4: Prepare for frame processing and widget display
frame_count = 0
ret, prev_frame = cap.read()
```

  - `frame_count` to count the number of frames.
  - `Read()` reads the first frame from the video.

# First Frame & Grayscale Conversion

## Module III: Farneback Optical Flow

- 1st Frame Availability Check: an error message if a problem

```
if not ret:
    print("Error: Could not read the first frame.")
else:
    print("Processing frames for animation...")
```

- Convert to Grayscale:

```
# Convert first frame to grayscale
prev_gray = cv2.cvtColor(prev_frame, cv2.COLOR_BGR2GRAY)
```

# Farneback Parameters

## Module III: Farneback Optical Flow

- Define Farneback parameters and values

```
# Parameters for Farneback optical flow
fb_params = dict(
    pyr_scale=0.5,    # image scale (<1) to build pyramids
    levels=3,         # number of pyramid layers
    winsize=15,       # averaging window size
    iterations=3,     # number of iterations at each pyramid level
    poly_n=20,        # size of pixel neighborhood
    poly_sigma=1.2,   # standard deviation of Gaussian
    flags=0           # operation flags
)
```

  - winsize: Window size for calculating polynomial expansion.
  - poly_n: Area size for approximation using polynomials.
  - poly_sigma: Gaussian kernel standard deviation

- pyr_scale: The scaling ratio of the image in each pyramid layer.
- levels: The number of pyramid layers (the more layers, the larger the movement covered).

# Create Display Widget

## Module III: Farneback Optical Flow

- Grid parameters

```
# Motion vector display parameters
grid_size = 20
vector_color = (0, 255, 0)
origin_color = (0, 0, 255)
vector_scale = 5
```

- Create widget

```
# Create image widget
initial_image = cv2.imencode('.png', prev_frame)[1].tobytes()
image_widget = widgets.Image(
    value=initial_image,
    format='png',
    width=prev_frame.shape[1],
    height=prev_frame.shape[0],
)
display(image_widget)
```

# Frame Display

## Module III: Farneback Optical Flow

- Check for the last frame (user-defined value)

```
while frame_count < 100:
    ret, frame = cap.read()
    if not ret:
        print(f"End of video or error after {frame_count} frames.")
        break

    # Convert to grayscale
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Calculate Farneback optical flow
    flow = cv2.calcOpticalFlowFarneback(
        prev_gray, gray, None, **fb_params
    )

    # Create visualization frame
    vis_frame = frame.copy()
```

# Create Optical Flow

## Module III: Farneback Optical Flow

- Create optical flow along the grid

```
    # Draw motion vectors on a grid
    h, w = gray.shape
    y, x = np.mgrid[grid_size//2:h:grid_size, grid_size//2:w:grid_size].reshape(2,-1).astype(int)

    for i, (xi, yi) in enumerate(zip(x, y)):
        dx = flow[yi, xi, 0]
        dy = flow[yi, xi, 1]

        # Draw origin point
        cv2.circle(vis_frame, (xi, yi), 1, origin_color, -1)

        # Draw motion vector
        cv2.arrowedLine(vis_frame, (xi, yi),
                        (int(xi + dx*vector_scale),
                         int(yi + dy*vector_scale)),
                        vector_color, 1, tipLength=0.3)
```

# Display Optical Flow

## Module III: Farneback Optical Flow

- Widget Update: Updates the image in the widget for display.
- Delayed display.

```
    # Update widget
    _, buffer = cv2.imencode('.png', vis_frame)
    image_widget.value = buffer.tobytes()

    # Optional delay
    # time.sleep(0.04)

    prev_gray = gray.copy()
    frame_count += 1
```

# Ending the Display

## Module III: Farneback Optical Flow

- Close connection: Closes the VideoCapture object with a message, informing that processing is complete.

```
cap.release()
print("Processing complete.")
```



# References

## Reference Materials


- John F. Hughes, Andries van Dam, Morgan McGuire, David F. Sklar, James D. Foley, Steven K. Feiner, and Kurt Akeley, “Computer Graphics,” Third Edition, *Addison-Wesley*, 2014
- P. Joshi, D. M. Escriva and V. Godoy, “OpenCV by Example,” *Packt Publishing*, Birmingham - Mumbai, 297 pages, ISBN 978-1-78528-094-8, Jan 2016.
- MediaPipe: https://mediapipe.readthedocs.io/en/latest/solutions/hands.html, accessed August 18, 2025


Mahidol University

# Motion Analysis

## Simple

### Step-by-Step

# Create Motion Analysis Function

## Module III: Simple Analysis

- Improve the Farneback code by adding a section for analyzing the direction and speed of movement:
  - Add the `analyze_motion( )` function.

```
def analyze_motion(flow, frame_shape, grid_size):
    h, w = frame_shape[:2]
    y, x = np.mgrid[grid_size//2:h:grid_size, grid_size//2:w:grid_size].reshape(2,-1).astype(int)

    # recording all motion vector values in terms of magnitude and direction
    all_dx = []
    all_dy = []
    magnitudes = []
    directions = []
```

# Create Motion Analysis Function

## Module III: Simple Analysis

- Improve the Farneback code by adding a section for analyzing the direction and speed of movement:
  - Add the `analyze_motion( )` function.

```
    for xi, yi in zip(x, y):
        dx = flow[yi, xi, 0]
        dy = flow[yi, xi, 1]

        # magnitude and direction conversion
        magnitude = np.sqrt(dx**2 + dy**2)
        direction = np.degrees(np.arctan2(dy, dx)) % 360  # convert to 0-360 degrees

        all_dx.append(dx)
        all_dy.append(dy)
        magnitudes.append(magnitude)
        directions.append(direction)
```

# Create Motion Analysis Function

## Module III: Simple Analysis

- Improve the Farneback code by adding a section for analyzing the direction and speed of movement:
  - Add the `analyze_motion( )` function.

```
    return {
        'dx': np.array(all_dx),
        'dy': np.array(all_dy),
        'magnitudes': np.array(magnitudes),
        'directions': np.array(directions)
    }
```

# Create Motion Analysis Function

## Module III: Simple Analysis

- Improve the Farneback code by adding a section for analyzing the direction and speed of movement:
  - Improved the code content in the motion calculation loop.

```
while frame_count < 100:
    ret, frame = cap.read()
    if not ret:
        print(f"End of video or error after {frame_count} frames.")
        break

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    flow = cv2.calcOpticalFlowFarneback(prev_gray, gray, None, **fb_params)

    # grid point for demonstration
    h, w = gray.shape[:2]
    y, x = np.mgrid[grid_size//2:h:grid_size, grid_size//2:w:grid_size].reshape(2,-1).astype(int)

    # analysis
    motion_data = analyze_motion(flow, gray.shape, grid_size)
```

# Create Motion Analysis Function

## Module III: Simple Analysis

- Improve the Farneback code by adding a section for analyzing the direction and speed of movement:
  - Improved the code content in the motion calculation loop.

```
    # Calculate statistics for features of interest
    avg_magnitude = np.mean(motion_data['magnitudes'])
    avg_direction = np.degrees(np.arctan2(
        np.mean(motion_data['dy']),
        np.mean(motion_data['dx'])
    )) % 360

    # display
    vis_frame = frame.copy()
```

# Create Motion Analysis Function

## Module III: Simple Analysis

- Improve the Farneback code by adding a section for analyzing the direction and speed of movement:
  - Improved the code content in the motion calculation loop.

```
    # small vectors (details)
    for xi, yi in zip(x, y):
        dx = flow[yi, xi, 0]
        dy = flow[yi, xi, 1]
        cv2.arrowedLine(vis_frame, (xi, yi),
                        (int(xi + dx*vector_scale), int(yi + dy*vector_scale)),
                        vector_color, 1, tipLength=0.3)

    # analysis display (2 frames calculation)
    cv2.putText(vis_frame, f"Avg Speed: {avg_magnitude:.2f} pixels/frame",
                (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255), 2)
    cv2.putText(vis_frame, f"Avg Direction: {avg_direction:.1f} degrees",
                (10, 60), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255), 2)
```

# Create Motion Analysis Function

## Module III: Simple Analysis

- Improve the Farneback code by adding a section for analyzing the direction and speed of movement:
  - Improved the code content in the motion calculation loop.

```
    # overall summary (2 frames analysis)
    center_x, center_y = w // 2, h // 2
    avg_dx = np.mean(motion_data['dx'])
    avg_dy = np.mean(motion_data['dy'])
    cv2.arrowedLine(vis_frame, (center_x, center_y),
                    (int(center_x + avg_dx*vector_scale*3),
                     int(center_y + avg_dy*vector_scale*3)),
                    (0, 0, 255), 3, tipLength=0.5)

    # update Widget
    image_widget.value = cv2.imencode('.png', vis_frame)[1].tobytes()

    prev_gray = gray.copy()
    frame_count += 1
```

# Direction

## Module III: Simple Analysis

- Mean Direction (Degrees): Calculated from arctan2(`dy`, `dx`) and converted to degrees.
- Direction Interpretation:
  - 0°: Movement to the right
  - 90°: Movement downward
  - 180°: Movement to the left
  - 270°: Movement upward

# Magnitude

## Module III: Simple Analysis

- Vector size (pixels/frame): Calculated from sqrt(`dx^2 + dy^2`)
- Relative speed:
  - < 0.5: Very slow
  - 0.5-2: Slow
  - 2-5: Moderate
  - 5: Fast

# Motion Analysis

## Module III: Simple Analysis



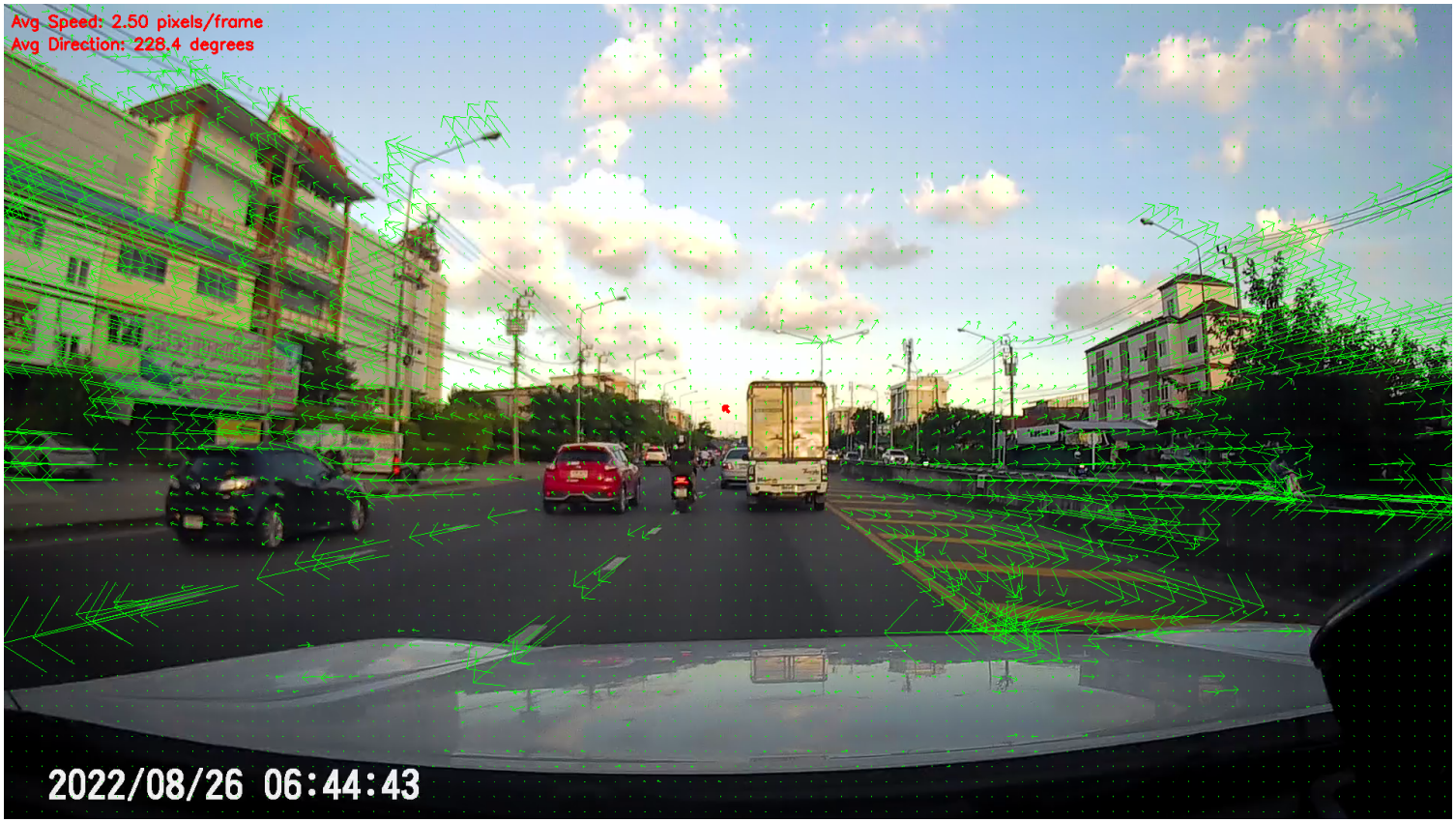
# Practical Application

## Module III: Motion Vector Field

- Primary Motion Detection:
  - Checks which direction the object is moving.
  - Checks for significant movement.
- Abnormality Detection:
  - Checks when the average speed exceeds a specified threshold.
  - Checks when the direction of movement deviates from the normal pattern.
- Flow Analysis:
  - Analyze fluid movement.
  - Study the main flow directions.

# Motion Analysis

## Module III: Speed & Direction



# Motion Analysis

## Module III: Fluid/Gas/Smoke Movement



# References

## Reference Materials


- John F. Hughes, Andries van Dam, Morgan McGuire, David F. Sklar, James D. Foley, Steven K. Feiner, and Kurt Akeley, “Computer Graphics,” Third Edition, *Addison-Wesley*, 2014
- P. Joshi, D. M. Escriva and V. Godoy, “OpenCV by Example,” *Packt Publishing*, Birmingham - Mumbai, 297 pages, ISBN 978-1-78528-094-8, Jan 2016.
- MediaPipe: https://mediapipe.readthedocs.io/en/latest/solutions/hands.html, accessed August 18, 2025


Mahidol University